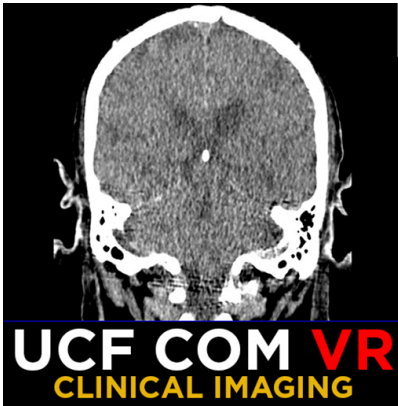




Clinical Imaging CT VR

Group 4 Members:

Arianna Ramirez Oquendo

Ash Hutchinson

Kyle Kratt

Matthew Eisenberg

Michael Biskup

Thomas Winslow

Zoe Schlesinger

Sponsor:

Melissa Cowan, MA

Assistant Director, Instructional Technology

Educational Technology Dept.

Melissa.Cowan@ucf.edu

Table of Contents

Clinical Imaging CT VR.....	0
Table of Contents.....	1
Executive Summary.....	7
Introductions.....	8
Arianna Ramirez - Project Manager: Unity Developer / Scrum Master.....	8
Zoe Schlesinger - Frontend: Unity Developer / UI Developer.....	11
Kyle Kratt - Frontend: Integration / VR Interactions.....	14
Michael Biskup - Frontend: CT Scans / 3D Models.....	16
Thomas Winslow - Frontend: CT Scans / 3D Models.....	18
Mathew Eisenberg - Backend: Drag & Drop / Database.....	20
Ash Hutchinson - Backend: Database / UI Developer.....	23
Problem and Solution.....	25
Societal Value.....	27
Educational Value.....	28
Legal, Ethical, and Privacy Issues.....	29
Security Issues.....	29
Legal Issues.....	29
Ethical Issues.....	30
Project Characterization.....	31
Minimal Viable Product.....	31
Accessibility and User Options.....	31
3D Models.....	32
Features and Interactions.....	32
Backend.....	34
Guidelines.....	34
Stretch Goals.....	35
AI Integration.....	35
Virtual Pen.....	36
Difficulty.....	36
Assessment.....	37
Multiple Environments/Rooms.....	37
Other Medical Imaging Forms.....	38
Additional Hand Tracking Elements.....	39
Quiz Library.....	39
Additional Platforms.....	39

Epics/User Stories.....	52
Epic 1.....	53
Epic 2.....	53
Epic 3.....	54
Epic 4.....	55
Requirements.....	56
Concept of Operations.....	57
Design Documentation.....	58
Tech Stack.....	58
Frontend.....	58
Backend.....	60
Administration.....	61
Hardware.....	63
Research.....	63
Unity Learn.....	64
Meta Horizon.....	66
Other VR Apps.....	67
Dr. Hemo's Great Escape.....	67
Cardiac VR.....	70
Embedding The Best Medical Practices.....	72
Development Process.....	80
Initial Development.....	80
3D Model Design Process.....	81
Development Issues.....	81
Design Summary.....	89
Block Diagram.....	89
Use Case Diagram.....	90
Activity Diagram.....	91
Entity Relationship Diagram.....	92
Detailed Description.....	94
Accessibility.....	94
3D Models.....	97
The Segmentation Process.....	115
Preparing models for VR.....	117
Labeling and Annotation within 3D Slicer.....	119
3D Models in Unity.....	120
Setting up the Model.....	120

Displaying the Model.....	122
Renders Container.....	123
3D Model Scripts.....	124
3D Model UI Buttons.....	128
Frontend.....	139
Backend.....	144
Level Design.....	151
Tutorial Room.....	152
Lab Room.....	153
Quiz Room.....	155
Room Design Changes.....	156
Layout.....	158
Initial Layout.....	158
Style Guide.....	165
Introduction.....	165
Project.....	165
Asset Naming.....	167
Scripting.....	167
Custom Tools.....	169
Build / Prototype / Test Plan.....	171
Testing Strategy Overview.....	171
Phase 1: Internal Prototyping and QA (June–August).....	171
Phase 2: Controlled Pilot Test with Medical Students (September–October).....	172
Phase 3: Iterative Improvement and Final Testing (October–November) 173	
Design.....	174
Interactions and Interactables.....	177
Administration.....	178
Budget/Assets.....	178
Backlogs.....	179
Final Summer Sprint.....	180
Product Backlog.....	181
Sprint Map.....	183
Milestone Chart.....	186
Conclusion.....	186
Summary.....	186

Results Analysis.....	187
Final Performance Analysis and Graphs.....	191
Insights Gained.....	192
Frontend.....	192
Version Control.....	194
Building the APK.....	198
Notes for Future Developers.....	201
Ideas for Future Developers.....	202
Student Testing Feedback.....	207
Acknowledgements.....	209
Sponsor Assistance.....	209
Additional Assistance.....	209
Facilities and Equipment.....	210
Bibliography.....	210
Appendix.....	212

Executive Summary

The project that we created is a virtual reality (VR) application designed to teach first-year medical students how to read and annotate CT scans. Within the app, the user will be placed in an interactive clinical imaging suite where they will be able to view normal and abnormal CT scans through moveable slices on a board and detailed 3D models. The students will have the ability to measure the 3D models and CT scans. The users are also able to view and manipulate 3D renderings of isolated organs and images of cadaveric dissection tissue. On top of the CT scan images and 3D model viewer, the team will be implementing quizzes and a drag and drop game, provided by the UCF College of Medicine, inside the virtual environment as an additional study tool for students to practice with. Throughout the development of this project, accessibility has been kept in mind at all times, particularly by adhering to the UCF Virtual Accessibility Policy, to ensure we reach the widest possible audience.

To accomplish this project, we will be using Meta Quest 2 and 3 headsets provided by the University of Central Florida College of Medicine. For software, development for the VR app will primarily take place using Unity 6 and OpenXR for frontend development and Visual Studio Code and Azure Devops Git Repo as a cooperative coding environment. Any 3D model creation/ changes that need to be done will be done using TotalSegmentator and Blender. Additionally, the team will be using 3D Slicer for rendering the CT scans and providing a 3D model and a slice through the isolated organ. The slice through the organ is displayed as an image that the user can control the placement of through the model using Sagittal, Transverse, and Coronal planes. Incorporating a backend will utilize Azure-hosted .NET/SQL Database, and Rest Controller.

The purpose of this project is to give first-year medical students the ability to study reading and identifying CT scans, identifying abnormalities in CT scans, and practice solving patient problems in a fully immersive VR environment. This will impact medical students, who are currently restricted to studying organ models only during lab times and typically without abnormalities such as medically implantable devices, and gives these students the ability to study at their own pace and on their own time using a virtual environment and virtual tools.

Introductions

Arianna Ramirez - Project Manager: Unity Developer / Scrum Master

Motivations

My personal motivation for this project mainly stems from my background in Biology and my interest in virtual/ augmented reality. I was studying Biology, specifically on the nursing track, for two years before I decided going into computers was a better option for me. I really enjoyed everything I learned, Biology and Anatomy were always a huge interest of mine and I've always liked the idea of mixing technology and medicine. So when I saw the presentation for this project I knew it would be my top choice. Another reason I have a big motivation for this project is because I have never worked on any kind of Virtual Reality software development projects, so I'd like this to be a learning opportunity for me, especially since virtual reality seems like it's going to be the next big thing in the technological world. I believe the College of Medicine students can learn a lot from interacting with virtual anatomical models. I know when I was studying Nursing I felt I had to learn as much as I could during lab times and my

camera roll was constantly filled with pictures of medical models because without the professor or a TA present I would not have access to the models or cadavers. This app will allow students to study and practice without feeling restricted to limited lab times.

Original Ideas

This project is a huge opportunity to learn with the current minimum viable product, but to add to those requirements, I think it would be good to consider implementing different kinds of practice games like a matching game in order to help the College of Medicine students practice identify human anatomy accurately. Another practice game could be a game that shows healthy and unhealthy models and asks the students to identify the unhealthy models and why, similar to a "Spot It" game. It may even be possible to have the student find the specific abnormality and circle it/ point to it on the model and check to see whether they pointed it out correctly. Besides this idea, another could be implementing AI produced definitions for abnormalities. An example would be if a student does not know why a specific model/ scan would be considered unhealthy we can find a way to have AI explain it so that students can learn more while inside the interactive environment. Otherwise, if I cannot find a way to implement this kind of AI, then I can just input the definitions or reasoning myself for students to reference and study from.

Individual Responsibilities

My role in making Clinical Imaging CT VR is project manager and scrum master but I also worked with Zoe Schlesinger to complete the rooms. Mine and Zoe's plan was to create the layout and rooms such as the Tutorial Room, the CT Viewing Room, and the Quiz Room, while also leaving space for any future development for future teams. It was our responsibility to ensure the scaling of the rooms is accurate. Also for an intuitive flow

between the rooms so the users do not feel lost or overwhelmed. Lastly, we are responsible for the addition of robust walls, objects, and proper lighting to make the scene feel alive while also making sure users cannot clip their heads through any walls or walk through any objects.

Proper scaling is going to be one of the most important and most difficult portions of our responsibilities. This includes: making sure the headsets adjust to each user's height, making sure any tall users can walk through hallways and doorways without clipping through any walls, placing any buttons and objects at a responsible height, and keeping objects at a realistic size. To achieve this me and the other level designers were constantly testing the objects we place in the environment with the headset. Through the Unity Designer it is hard to get a real feel for the scale, so instead of relying on the headset simulator we made a habit of checking through the headset. We each vary in height so we will be good testers for the scale of each object to ensure they are not out of reach or unrealistic sizes inside the environment. The flow between rooms is also an important task. If there is improper flow then the users will get lost or dislike the design, and then likely end up not using the project. Since this is intended for medical students to use in place of a lab, the plan for each of us level designers was to try our best to mimic a medical lab setting. Additionally, the layout is meant to be as user friendly as possible, starting each new player off at the tutorial section for them to get a good feel and understanding of the tools and mechanics before moving onto the CT Viewer and Quiz rooms.

As project manager some of my main responsibilities included proper communication between our sponsor, our professor, and my team. It was usually my responsibility to inform the sponsor of our weekly progress or to ask the professor any questions my team and I wanted the answers to. Additionally, it was also my responsibility to reach out to any event

coordinators such as the STEM day director or the Showcase coordinator since my team participated in both events. I also had to ensure my team had a good understanding of goals and deadlines so we remained on track for project completion. For this I also maintained the Jira sprints and constantly updated them with big deadlines or individual responsibilities for the members to reference. Finally, since I was project manager I also did my best to help other members when needed.

Zoe Schlesinger - Frontend: Unity Developer / UI Developer

Motivations

My name is Zoe Schlesinger and my motivation for joining this project is because I like how it combines the medical field with computer science. When I first came to UCF I was deciding whether I should go the medical route or the computer science route so when I saw this project that combines the two I was very interested in joining for my final project here at UCF. I believe it's also fitting because when I was looking into the types of jobs Computer Science majors could go into I was interested in the medical robots where they perform surgery. Although this is not exactly in that same branch it still reminded me of my initial thoughts and I thought it would be an interesting project. There were a couple of medical related projects presented but I felt this was the project most aligned with what I wanted to accomplish in my final year at UCF. Specifically, I wanted to go out of my comfort zone and work with something that I hadn't before. In this case it is working with VR, I've always wanted to learn and play around with VR but hadn't had a chance so I thought this would be an opportunity to learn and expand my skills. Additionally, many of my friends are going into the medical field and will be going to med school which this project is oriented towards so I also felt drawn to it when I saw that. I feel like this will be a project that

will be very useful for med students and it'll be cool that they get to actually use our work.

Original Ideas

Although I'm sure my ideas will evolve as I get more information about the design and layout of the project, one of the ideas I have is having different ways of the student looking at the scans. There is a lot that goes into studying CT scans so I believe it would be beneficial if the student is able to have a casual, untimed session where they are able to study and look at the scans with no pressure and then have a mode where they get a certain amount of time to look at the scan before they must answer a question or questions. I also think it would be helpful for them to be able to choose to look at and be quizzed on only normal scans, only abnormal scans, or a mix of the scans. It might also be interesting to have a mode where they must look at two scans and choose which is abnormal. Additionally, I think it would be wise to have a tutorial "level" that walks the user through how to use the controls and the way to study the scans as not everyone would be equipped with the knowledge to use VR or our program. Another idea would be to have a learning area where the student can follow along to a voice narration to learn how to read the scans or learn the organs. It might also be good to have a pathway for students to follow along, like an "easy", "medium", "hard" and the student can learn about the scan then study it on their own, and then answer questions or do a drag and drop quiz.

Individual Responsibilities

For this project, my role is within the frontend, doing level design alongside my other groupmate, Arianna. This role includes tasks like deciding and creating the room layouts, ensuring wall functionality, and designing the interior of the rooms including where user interactions should go and where environmental objects are needed. Additionally, it falls to us to

think about other design choices like color schemes, background audio, and lighting. Lastly, across the whole application it is important that we have accurate scaling not only to realistic standards but across each room.

While we have not gotten to the actual interior designing of the application, I have created many resources for us to reference once we begin the work. This includes Figma's for the overall room layouts, a storyboard showing interior design, and a list of possible assets we might need. Our sponsor has emphasized the importance of planning what we want to do and how we want it to look before we get started because it creates a good reference for us to use and no one is confused on what is going on within the group. The one aspect of the application I did start this semester was creating the actual layout of the room in Unity. Using the cube object I was able to create the walls for each room including door holes for where we want to place the door asset. I was also able to include the floor and ceiling and ensure everything was functional so the user would not be able to clip through any wall.

Next semester, I will be focused on designing the interior of the rooms. Placing objects meant for user interaction where they need to be and filling the rest of the empty space with environmental objects like plants, cabinets, and desks. I believe it is important for the area to look as realistic as possible for the user as well as ensuring it does not feel too cluttered or too sparse. Included in this is the scaling of each object and making sure it is imported to application and scaled properly. Our sponsor mentioned this as an issue in some of her past projects so I want to make sure to test everything as I go so there is no need to go back later and fix a bunch of stuff.

In the second semester I also took on the role of UI developer to help Ash with the UI interface of the quizzes as well as the new login room we

had developed. This included ensuring the colors had enough contrast for accessibility as well ensuring the UI would accept variations of questions and answer choices. For the login I created the separate scene and ensured a spatial keyboard would work to fill in the inputs.

Kyle Kratt - Frontend: Integration / VR Interactions

Motivations

My name Kyle Kratt and I've personally been using VR for many years, and so I've always had an interest in the fine details that make programming VR games different from traditional games. I also felt that having that experience gives me a decent understanding of what makes a decent VR UI and overall user experience - I hope that already having an implicit understanding of what options are available to improve accessibility, such as smooth vs snap turning or distance grabbing, will help us fine tune our final product. Besides that, it also feels like an opportunity to attempt a project well outside of anything I've done before, but which still feels reasonable in scope. I'm certain that at the end of this project, regardless of my role, I will have learned plenty about not just VR, but also 3D graphics, combining the various layers that will make a project like this possible, and working together as a team to create something on a larger scale.

Original Ideas

I still feel like we have a lot to discuss with our sponsor in terms of specifics, but as I see it now, this is going to be a study aid for medical students. I don't think that we will want a distracting environment or theme - the emphasis should be on the various assets that our sponsor will provide. These assets will come in a variety of forms - audio samples, images, videos, and 3D models - the main benefit of the VR experience is definitely

going to be the ability of the user to manipulate and view the 3D models, but VR will also allow us to have realistic scaling for our images, allowing us to implement rulers using actual units, which is more difficult using regular monitors. There is going to be some form of quiz, probably using a random selection of assets, combining multiple choice questions and applying labels to images and 3D models. It might be useful to have some form of analytics tracking student responses, allowing students to see their own problem areas and instructors to see commonly missed questions.

Depending on how our assets are organized, we might want to label and separate the various assets by category, allowing students to study specific categories of assets and helping the analytics to determine which categories students are struggling with, rather than just specific problems. I think an important aspect of this project will be figuring out a standard format and categorization system that will enable instructors and future developers to create libraries of additional assets that can be easily added into the application. This will ensure that this application can be maintained and used well into the future.

Individual Responsibilities

My role will be working with the backend and VR functionality. I'll be primarily focused on ensuring that all of the VR functionality works as intended - grabbable objects, moveable UI elements, getting buttons and menus to function with both controllers and hand tracking. I'll also be working with the frontend designers to make sure these elements will work functionally.

A key responsibility for me will be making sure that the CT scan experience is fully functional. This will require allowing multiple control schemes - users should be able to use either controllers or hand tracking to navigate the CT scans, and controllers should function both from a distance

and from up close. The user should also be able to navigate a CT scan either by selecting a point on a different scan axis, or by using a slider to navigate between slices for a given axis. A key part of the functionality is making sure that navigating a CT scan in a given axis should update both the current slice for the 3D model as well as the other axes (which will each have their own panel).

I'll also be responsible for getting the frontend quiz elements to interact with our database. This will involve some C# scripting in Unity to connect our frontend elements with our backend APIs, as well as additional C# .NET scripting in our backend.

Michael Biskup - Frontend: CT Scans / 3D Models

Motivations

My name is Michael Biskup, and my motivations for this project come from my own personal interest in applying my experience in computer science within the medical field. Most of this interest stems from a course I took in bioinformatics algorithms during my junior year. This course not only gave me an interest in the applications of computer science within medical research but also felt to an extent more significant than any other projects I have worked on, given what contributions and advancements in this field that can lead to. Given this prior interest from the course (and admittedly, some inspiration from recently watching House M.D), I felt as if this project was a great choice. Even without the context of the prior interest, I think the premise of making a Clinical Imaging Virtual Reality Application for students is a great idea, and it allows me to gain more experience in VR implementation along with Unity of which I have some previous experience (fairly good poker game and an undeniably less impressive flappy bird

game). But combining my previous interest in this topic along with the factors that make this project enticing, this is something that I feel highly motivated to complete.

Original Ideas

Given the nature of this project being a learning app using various renders and real-world CT scans, it gives us a fairly big canvas on which to add to. My first impression was to use the different resources and assets that we have to create environments in which the student could study the materials in broader terms. Within this environment, I believe it would be beneficial to also allow students to narrow down the context of what they are studying to target areas that students might need more help with. Think of it somewhat as a sandbox environment where students could study the different 3D models, and it would give relevant information. This would provide a great way for students to study the materials before going into the environments where they would be tested via the practice questions. The nature of how these questions would be asked is broad in the current proposal, so with more context, it will be easier to narrow down what is necessary. However, I believe that different instances should each ask questions on CT scans within similar areas and provide a few methods of quizzing the students. Whether it is identifying major organ structures or comparing normal and abnormal CT scans. For the voiceovers, I don't think implementing the AI voice overs will be difficult however, I'm conflicted on whether it is the right idea given how lifeless they can sound. Despite this bias, I will acknowledge that training the model to say the lines is probably more convenient than recording voice-overs for every single piece of text.

Individual Responsibilities

My role will be working on implementing the 3D models and assets that we will be using. This encompasses streamlining the CT scans into 3D

models as needed and incorporating viewing and user interaction with the models. This will include handling format conversions and optimizing the models for our application using relevant tools. I will make sure the models are utilized to give the best educational value. As Cross support I will also help out with other roles as needed. This part of my role will be more adaptable and allow me to contribute to the project as any issues may arise.

Thomas Winslow - Frontend: CT Scans / 3D Models

Motivations

My name is Thomas Winslow, and my inspiration for choosing this project was to utilize my skills in computer science to align with my interests in the medical field. Among all the project options, this one stood out the most. I truly believe it has the potential to benefit not only my learning but also students using the Clinical Imaging VR game by helping them better understand CT scans. Creating this project in VR was a smart choice. So much of the medical field relies on hands-on experience, and VR provides a way to simulate that, especially in cases where displaying and manipulating 3D models of isolated organs would be impractical in real-life scenarios. My oldest sister is a Physical Therapist, and my younger sister is a Physician Assistant. Seeing their journeys firsthand, I know how challenging the medical field can be. It would be really meaningful to me to help students understand better what they are working with within the field, as well as anyone else interested in learning what medical students do. While I have limited experience with Unity, mostly from their online courses and from a few 2D games, I'm confident that my team and I can create a fully functioning VR game. On a side note, one of my favorite TV shows is House M.D., which makes me extra enthusiastic to create a project like this. One

idea I have for the project would be to incorporate a Dr. House AI that can voice-over what the player is doing, and help them towards a solution.

Original Ideas

With how big this project is going to be, I know there are a lot of different ideas we can take towards creating the optimal outcome. An idea I have is to put the player in a school classroom or next to a surgery table to examine organs, as well as answer multiple-choice questions either through a HUD or a paper that the user can hold. For CT scans, it would be easier to put them on an interactive paper that can be clipped to a whiteboard for easy visual access and the ability to multitask the 3D organs and multiple-choice questions. This allows lots of interactivity in a realistic environment. I'm visualizing this project to look similar to Surgeon Simulator for VR, where the user is just a pair of gloves for the Quest controllers, and in a similar setting. The player doesn't need a specific model, but it's hard to say what it all needs to look like without beginning the Unity work. The most challenging part would be importing CT scans and 3D models, which requires working with Blender and 3D Slicer. I know this is going to be a really fun and rewarding project, and I hope to turn it into something more than just a VR game.

Individual Responsibilities

My role will be working with the backend and VR functionality. I'm focusing on implementing CT scans and 3D models into the game. This involves processing DICOM files using 3D slicer to convert and segment anatomical models to be rendered in game. It is important to take into account the size constraints of every model and how to render it for a VR setting.

Mathew Eisenberg - Backend: Drag & Drop / Database

Motivations

My name is Matthew Eisenberg. I chose the Clinical Imaging VR project primarily because I enjoy building projects from the ground up, rather than working with existing codebases. The opportunity to take an abstract concept and turn it into something functional using modern tools like Unity and VR is something I find genuinely interesting. Another major reason I selected this project is that it doesn't involve any tedious LLM training or generative AI work—just basic API integration with OpenAI if needed. As someone who has already worked with VR technology, I was specifically looking for a project that focused on simulation and data visualization rather than artificial intelligence. This project is grounded in real-world use cases, making it far more relevant to the kind of work I want to do in my career.

What interests me most about the project is working with actual CT scan data and converting that into interactive 3D visualizations in Unity. I find the challenge of handling large medical imaging files and rendering them in a resource-efficient way to be a compelling computer science problem. It's a refreshing change from the overhyped focus on LLMs and brings tangible, practical value.

Original Ideas

From a technical standpoint, I'm excited by the idea of integrating backend systems with Unity and incorporating UCF's existing datasets. Creating seamless bridges between technologies—like DICOM files, Blender 3D models, and VR platforms—is something I find technically engaging.

This project also aligns closely with my real-world experience. During my internship, I've primarily worked on backend systems and simulation within a STE (Synthetic Training Environment) context. Improving my ability to integrate and make these technologies compatible would significantly strengthen my position in the job offer I already have lined up. Many of the tools used in this project—such as Unity, Blender, and data pipelines—are closely related to technologies used in STE. I also have experience with Unity VR when it comes to space simulation so I think I can carry over that experience to this project as a project manager.

Individual Responsibilities

I began the project serving as both the Project Manager and Scrum Master, where I was responsible for setting the overall operational structure of the team. This included coordinating weekly workflows, assigning tasks based on team strengths, facilitating daily communication, and ensuring everyone had the clarity and support needed to deliver their work on time.

In parallel with management duties, I handled the majority of the DevOps and CI/CD infrastructure. I created our Azure Pipelines configuration, automated builds and deployments, managed multiple environments, and ensured stable delivery of new features throughout development. This also included troubleshooting environment issues, ensuring consistent build reproducibility, and optimizing our pipeline to shorten iteration cycles.

Beyond DevOps, I worked on the database. I designed the database schema to ensure it was scalable and able to support both present and future feature sets. This involved managing migrations, optimizing queries, enforcing data integrity, and maintaining consistent communication between backend and gameplay systems. My work ensured that game state, user

progress, and other persistent data layers were structured cleanly and efficiently.

I also managed our Discord server, organizing communication channels, moderating discussions, and setting up systems to keep collaboration smooth among designers, developers, and artists. This greatly reduced confusion and centralised communication.

During the second semester, I stepped down from my Project Manager/Scrum Master role. During this phase, I transitioned to database and became responsible for implementing our drag-and-drop quiz system, which required integrating backend logic, UI interactions, object data flow, and validation systems.

Ash Hutchinson - Backend: Database / UI Developer

Motivations

My name is Ash Hutchinson and I chose to participate in the clinical imaging virtual reality (VR) application project because I believe that this lines up with my interest in the medical field. I have been in multiple health sciences classes and have been certified as a Certified Nursing Assistant (CNA) as well as in cardiopulmonary resuscitation (CPR). I was even planning on going into the medical field before realizing that my skill set would be put to better use in computer science. I have never lost my love for medicine or the human body and this project allows me to use my current skill set in computer science while still helping out in the medical field in some way. The goal of this project is to be able to effectively teach first and second year medical students how to properly read and use computed tomography (CT) scans to identify any potential issues in their patients. Not only does this mean that I will be able to use my skills in

coding to help with the medical field, but also in teaching others about it. Because the lessons will take place in virtual reality, I will have the opportunity to learn more about 3D modeling in applications like Blender and 3D development in applications such as Unity. Truly the chance to learn more and participate in this overlap between medical and computer science is a chance that I have taken.

Original Ideas

The expectations presented with this clinical imagining project are quite clear in what our sponsors want, and even though we have not yet met with our sponsor I do have ideas on how to implement what has already been laid out. We are expected to be able to lay out CT scans for the medical students to be able to cycle through and mark. I believe that it could be a good idea to have that plus giving feedback on whether or not their markings were correct, and why the correct markings are correct. Just providing useful feedback to accentuate the learning experience. The program could also store what the students did and send it to their teachers if providing immediate feedback turns out to be implausible, just so the students have a chance to learn and find out where, exactly, they went wrong. I believe that the real trouble will be the file sizes that will be worked with as CT scans are saved in files that can be hundreds of megabytes or even tens of gigabytes of storage. These are huge files and when also needing to slice these images into 3D models, the time it takes to render could impact the performance of the rest of the program. But overall I intend to have this be as usable and helpful as possible for the students that this project is intended for.

Individual Responsibilities

My role will be working with the Frontend on the settings menu and acting as the accessibility manager for the user interface. I will primarily be

making sure that everything in the settings menu will be accessible and in working order for the users to be able to get the most out of it. I will also be participating in design decisions such as color choices, voice overs, and lighting. There have already been steps taken to set up the UI such as planning it out on Figma and taking inspiration from Meta. I have unfortunately been having issues with Unity when it comes to connecting with the team project and with creating things in general so I have not yet been able to actually create anything physically for the UI, but I have managed to find assets for it and further plan out a design that will be appealing and accessible.

Problem and Solution

There were three main problems that we considered and wanted to address through the creation of this project and why we believed it was a good fit as a VR project.

The first problem was we knew the UCF College of Medicine has a database that holds a bunch of patient CT scans. These patients are people who have passed away and donated their bodies for educational and scientific purposes. Every year more and more patients are donating their bodies and so more and more CT scans get added to this database. Unfortunately though, this huge trove of knowledge goes widely unused since it's hard to incorporate so many different kinds of scans into the curriculum and update it each year. We wanted to utilize all this data, so we found a way to take these CT scans and turn them into detailed, color-coded 3D models and upload them into the VR applications as isolatable and interactable learning tools. By doing this, we are able to utilize all this

unused knowledge and at the same time we upload them into the app without the need to tether it to a computer.

That brings us to the second issue, we want to build confidence among first-year medical students to read and interpret CT scans. Maybe they do not have that confidence built yet because they learn at a slower rate, or they do not have enough lab time to get a good grasp and understanding of every structure in a CT scan. To build this confidence they would need a studying tool, but a lot of the studying tools for these students are online applications that require very strong computers such as 3D Slicer. The point of this VR application is to be a hands-on studying tool for as many students as possible and we knew, unlike the average computer science student, they would not have crazy strong computers with unbelievable graphics cards to run a heavy VR application off of. So we have uploaded all the models onto the Meta Quest headsets and found the perfect ratio for the students to be able to get a detailed, color-coded 3D model while also having it run purely off the in headset processor all while maintaining a high frame rate to reduce motion sickness as much as possible. Since the UCF College of Medicine provides headsets for students to check out and utilize, we have this application pre-loaded onto each headset, no computer download required. They can take as much or as little time practicing reading CT scans with the help of 3D models, taking quizzes to test their knowledge, or enjoying a drag and drop game. With all these tools we hope that the first-year medical students can become more confident reading and interpreting CT scans.

Lets say the students are already confident, they know exactly what they are looking at when reading CT scans. With that in mind, one of the main focuses for this VR application was to allow first-year medical student practice identifying abnormalities in CT scans. With the current UCF COM curriculum first-years focus on reading normal CT scans so they may fail when it comes to them identifying an abnormal scan. They may see a black

spot above the heart and not realize they are looking at a pacemaker. With that knowledge, we have found a way to take the provided CT scans and convert each abnormality into a 3D model and we made it so it can be isolatable from the rest of the body so the students can look at the 3D model, identify what they are looking at, then find it on the CT scans. We hope with this they will get some more practice knowing what medically implantable devices look like in CT scans.

Societal Value

The primary purpose of this CT VR app is to help first-year medical students practice analyzing CT Scans. While students from other years or even non-medical users might find it useful, the app is designed with the assumption that users already understand basic medical terminology and are familiar with CT scans.

As education evolves, so have learning methods, especially with the rise of new innovative technologies. Integrating technologies and education is not a new concept but using Virtual Reality (VR) as a learning tool has remained relatively rare. With this app, we aim to demonstrate that VR can be a powerful pathway for learning and studying, especially for medical students by simulating a professional environment where they can build confidence using tools and analyzing CT scans.

Although there are plenty of resources for students to study CT scans, our app creates a unique experience by putting the user into a clinical imaging suite that resembles the professional setting these students will find themselves in. The user can compare normal and abnormal CT scans, test their knowledge from the questions given, and receive immediate feedback. Additionally, other features will be provided for users such as a ruler to

measure structures. Our sponsor emphasized that in the medical field, measurements should be taken in millimeters, but many students often measure in inches and have to later convert their work to centimeters. Therefore, our app will ensure all measurements will be taken in millimeters, which we hope will reduce the amount of times this error is made within their classes.

This app provides a private, immersive space where users can engage with CT scans, answer questions, participate in a drag and drop game, and explore material at their own pace as often as needed. This can prove beneficial to those who need more time to study or those who prefer a more hands-on learning approach. Our hope is medical students can use our app to practice the skills they need in their classes and clinicals, helping to reduce errors and build their confidence.

Beyond this specific application, we hope our VR app will showcase VR's potential as a learning tool. While this app focuses on CT scans, there are many other areas where VR could be applied. Even within the medical field there could be VR apps for exploring other types of medical imaging such as MRIs, X-rays, and more. Apps like these would offer valuable studying opportunities for students.

Educational Value

CT scan interpretation is a core essential to clinical decision making in emergency medicine, surgery, neurology, oncology, etc. It is important that a student knows exactly what sort of data they can interpret from a scan, how it's used, and the best practices for the field they are going into. Essentially, medical students and trainees should not only recognize what

types of data can be extracted from a scan, but also understand how to interpret the data.

CT imaging is often taught through textbooks or on slides that a teacher presents. One of the major drawbacks being that there is limited scan exposure for the student, like not having an axial field to learn rotations on. While these are still good learning tools, oftentimes students don't get a clear image of what it's like to be working with CT scans and models in the field. Oftentimes, the learning is done on the job where sufficient feedback may not be provided. This creates a gap between academic knowledge and its practical application to the field. Many CTs offer rich spatial and psychological data which students can struggle to grasp without hands-on experience. This is where our VR game becomes a major learning experience for students.

Our VR game will simulate CT viewing and analysis and provide an interactive and low-risk learning experience that mirrors the best practices in radiology and clinical reasoning. On top of this, our diagnostic questions will provide a meaningful way to ensure the student is learning, and give them the ability to see where they might be struggling.

Legal, Ethical, and Privacy Issues

Security Issues

During the creation of the Clinical Imaging VR application, there were points where potential security concerns needed to be taken into account and addressed. Because our project is intended for the use of UCF students that are in the UCF College of Medicine, the security risk that the project faces is minimal, however this is not the case for UCF. Due to using confidential student information for login and result return purposes, if our

data is breached, the personal data of UCF students will be put at risk. To minimize this, there are no names or any contact details in the user table in the database as well as securing any SQL communications with the database to prevent SQL injections. To make sure that our code is secure, the UCF IT department has reviewed the backend code before allowing it to be put on their systems.

Legal Issues

The Clinical Imaging CT Virtual Reality application that we are building will only be using readily available open source resources and assets and resources and assets that are provided by the UCF College of Medicine. Anything that is used in our build that is not open source and not provided are assets that are made by the team itself. Music and other audios are incorporated into this project as well. The audio that has been selected as background music is royalty and copyright free. Additionally, the audio that is used for the tutorial has been written and generated by the team. Due to these precautions, there should be no legal repercussions that emerge as a result of using someone else's work. There is also a legal and ethical concern about the use of real CT scans and images of real cadavers that is addressed in the following paragraph.

Ethical Issues

Because this is an educational application made in collaboration with the UCF College of Medicine, any CT scans that are incorporated in the project will be real CT scans of real patients or cadavers. Showing these images could be a cause for concern from an ethical standpoint. Any CT scans or cadaver images that are used have been provided to us by the UCF

College of Medicine and thus have already garnered any required permissions by patients or family members for use for educational purposes and does not break any HIPAA laws. Additionally, no personal information such as name, age, etc. is given to the team and will not be used in the application. As a precaution against potential misuse of the provided images and scans, UCF College of Medicine only provides headsets and app usage to UCF students who have signed proper forms regarding HIPAA violations upon becoming a student.

Project Characterization

Minimal Viable Product

The Minimum Viable Product (MVP) for our project will be an interactive virtual reality clinical imaging suite application built for the use of Meta Quest 2 and Meta Quest 3 headsets. The application is designed to assist in the learning of first-year medical students by allowing them to practice their skills in analysis and annotation of CT scans in a simulated clinical environment. The core features of the MVP include:

Accessibility and User Options

This clinical imaging virtual reality application is meant to cater to as many medical students as possible. With this in mind while creating this application the team will be adhering to the UCF Digital Accessibility Policy. Since this policy was mainly written with a web interface in mind and this is a virtual reality environment there may be certain aspects that the team will not be able to adhere to but the team has still found creative implementations to allow this product to reach a wide audience.

Some of the accessibility features will be color blindness options and avoiding important information being color coded unless the user chooses. It is also important we ensure all text and objects properly render in the virtual reality environment at a high frame rate and resolution. Any important audio scripts for the users will also have a written version within the application for users to read instead. In the same vein, any text in the application will have an audio option and will be in a font that is easy to read for all users.

In order to make this virtual reality application a more comfortable environment the team will also be implementing some options for the users that may experience motion sickness. Some ideas will be snap turning or a teleport movement option so that the screen does not move too much and cause sickness. If the user does want a free fly/ movement option we can implement a tunneling vignette which can also greatly reduce motion sickness.

3D Models

UCF College of Medicine has created anatomical 3D models for previous medical virtual reality apps so they will be provided for the Clinical Imaging VR application. For the room the team will also be using the provided models to simulate a lab environment. The team may need to alter the provided models to allow real-time manipulation of renders to display slices of CT scans. This will be done with Blender, an application for creating and altering 3D models, and TotalSegmentator, a 3D slicer AI add-on for segmenting anatomical structures. Should any specific models need to be made then they will also be made with Blender.

Features and Interactions

To make this Clinical Imaging Virtual application run as intended and succeed in helping medical first and second year students in practicing CT scan recognition, there needs to be certain features implemented. One of the main features is to support the viewing and scrolling through different normal and abnormal CT scans and anatomical cadaver images, which will be provided to the team by the UCF College of Medicine. Another feature will be the ability to display certain 3D models of organs in an isolated and non-isolated manner for students to have the options to study the organ alone and how it interacts with other organs around. Additionally, we will have modeled and isolatable medically implantable devices, as that is one of the main focuses of this application.

Interactions will help students to study anatomy and CT scans with features such as drag and drop games and a virtual ruler to measure structures. Also implementing colored hotspots to indicate anatomical structures and for highlighting and identifying abnormalities in CT scans. On top of those, labeled menus can help the students in understanding their options and interact with the application easier.

Besides models and interactions for the users the team will also be implementing multiple-choice quizzes, provided by the UCF College of Medicine, to practice and study with. There will be setting options for students to make adjustments in, such as, turn on/off AI narrations, adjust audio, quit, options for motion sickness, etc.

To ensure all users understand and can use the app and all its features properly the team will be implementing an introduction/ tutorial section upon opening the app. This will be in the form of written instructions placed on a wall above the spawn room, and in the form of audio narration to guide the user through the VR experience as they walk through the rooms. Additionally, the team will include a section that will be a credits page for the team and the team's roles.

Backend

For this application, the backend handling needs to be efficient and secure as this will be hosted on UCF servers. For the .NET API, we have followed REST API practices for data communication. The server and database are both intended to be hosted on

For this application we need to have a strong backend, the minimal viable product for the backend is building the required capacity. There will be a REST API for data communication. The deployment system will be an APK to UCF College of Medicine web server. Containerization will be done through Docker for its efficiency. As either a stretch goal or future expansion there can be a web panel interface.

- Spring Boot or Express

Guidelines

Our sponsor, Melissa Cowan, has previously made virtual reality applications with other senior design teams for the College of Medicine

medical students to practice with. Since we are making another application for the College of Medicine we were provided a style guide document that the team will be following while coding so that it stays similar to the other virtual reality applications already made. Additionally, we will be adhering to the UCF Accessibility Guidelines when creating the user interface and interactions for the lab.

Stretch Goals

AI Integration

Potentially adding AI voice overs for the different anatomical parts in the 3D models or for the quizzes to help with accessibility. This can either be done as pre-generated audio files and creating colliders with the audio scripts attached, or by using the Microsoft Azure Unity SDK which has already been made and is compatible with VR devices.

As a bigger stretch goal we can find a model to generate AI quiz questions and responses that correlate with the VR App. Alternatively, we can add a language model that can answer questions for students about CT scans and abnormalities in the CT scans. Although, this might not be possible with our time frame since the current AI models do not read CT scans very well which means we would need to train a model on our own, and that would take a lot of time to complete properly without the AI model constantly making mistakes.

AI Narration Implementation

In our final product we were able to implement the AI narration to guide the user through their experience. We implemented this by creating box colliders throughout the virtual environment and creating an AudioScript that will set off the intended AI narration when the user passes through it. This ensures the correct audio is played to guide the user in the specific room they entered. The audio was written and generated by the team using a copyright free AI generation source. Additionally, we implemented an audio switcher so should the user walk into another box collider to set off a different audio tutorial, the old one will stop and the new one will start so there would be no overlapping tutorial narrations. We have a narration for every room that the user enters to explain a little about what the purpose of the room is and how to operate the activities in that room. This AI narration is able to be turned off or volume adjusted in the settings menu.

Virtual Pen

In addition to the ruler and the labeler which is a part of the minimum viable product, we can add a virtual pen that will allow users to write in the air. This can be helpful for marking points in the 3D model. We can also put a white board in the room for students to write down their ideas when studying the models or when taking the quizzes.

Difficulty

An addition to the quizzes could be to let the users choose how difficult they want the questions to be. Either the team can sort the questions, or we can ask our sponsor to sort out the questions we will be getting from the

College of Medicine by easy, medium, and hard difficulties and make the quizzes that generate in the VR App be based on the chosen difficulty. This will not only allow for a more challenging environment for the students, but it could also potentially give us the ability to reach a larger audience than just first-year medical students. If we had difficulty levels then we could also have some second-year specific multiple choice quizzes.

Although we were not able to implement this in a way for the user to create the difficulty of the quizzes, the admin (like our sponsor) would be able to set and label quizzes if they wanted to have a couple different quizzes each labeled “Easy”, “Medium”, “Hard” so for future implementations we can still have second-year medical student specific quizzes.

Assessment

We can sort the quiz questions we receive from the College of Medicine based on type which we can then use to calculate an assessment on the student’s performance after completing each quiz. For example, if a student struggles with brain abnormalities then the post quiz assessment will indicate to them their score and suggest they study more on the abnormalities that can occur in the brain.

In our final product we included a screen at the end of each quiz which would tell the user out of the total number of questions in the quiz, how many they got correct. Additionally, within each question after they answer they are given a sign where they got it correct or incorrect. This sign occurs both visually (and sometimes with explanation) and with audio (a ding or

buzz). Although this does not directly tell them to practice more on a subject it is still a good indicator based on the quiz topics.

Multiple Environments/Rooms

Another potential addition would be to have multiple rooms in the VR app outside of the required spaces. We can add a tutorial room to explain movement, settings, and how to manipulate the models or take the quizzes. There can also be a credits room with each of our names and roles. Otherwise, we can add doors that cannot be opened yet as a placeholder for future development.

In our final product we did end up having multiple rooms to display a VR controller tutorial and this room also included our credit board. We also added a login scene completely separate from our main building and rooms. In future expansion there could definitely be room to expand to include different activities or even other types of medical images to view.

Other Medical Imaging Forms

This application's main purpose is to display CT scans for medical students to study with. There is strong potential for viewing and interaction with CT scans, and we have identified strong potential for expanding into other forms of medical imaging in future development cycles or stretch goals. From early discussions with the sponsor and our team's personal motivations, it's clear that students could benefit from learning with a wider range of diagnostic tools. Future iterations of this VR platform could incorporate additional imaging modalities such as:

- MRI: Useful for viewing soft tissue, extreme stretch goal.
- X-Ray: Useful for skeletal analysis, a very unlikely stretch goal.
- Ultrasound: Interactive ultrasound training could simulate scanning movement and allow interpretation of dynamic, low-resolution imagery.

While these imaging types are not part of the initial MVP, we have accounted for modular design and extensibility in our Unity application architecture to allow future medical image libraries to be incorporated with minimal disruption. This expansion would follow the same principles outlined in our current plan—focusing on interactivity, clarity, and immersive study tools like measurement, annotations, and AI-guided explanations.

Additional Hand Tracking Elements

While we're going to implement hand tracking for interacting with the CT Scans and UI elements, we still expect users to need to use controllers for features like movement and turning, limiting what a user can do without controllers. It's possible to navigate VR using additional hand gestures, but this would also require additional tutorials and may come off as unintuitive to users.

Quiz Library

Adding a library for the quiz questions and answers where the quizzes in the app can be randomly generated from. This would make it more

randomized than premade quizzes and it would allow for the expansion of quiz questions and answers for further future development.

Additional Platforms

If time permits after the primary development of the VR application, the team may extend the application to support additional platforms beyond Meta Quest 2 and 3. These alternate versions would broaden the accessibility and utility of the training tool for different user environments.

Desktop Application

While the core experience of the Clinical Imaging VR Application is tailored for immersive interaction via Meta Quest 2 and 3 headsets, we recognize the importance of accessibility, versatility, and scalability. A desktop version of the application will be pursued as a post-MVP deliverable to bring the same educational value to environments without VR headsets. This would expand the reach to classrooms, personal study environments, and academic institutions that do not yet support VR hardware.

Purpose & Value:

The desktop version would make the platform accessible to a wider range of users:

- Students without VR headsets can still interact with clinical imaging datasets.
- Faculty can use it for classroom demonstrations or grading.

- Researchers can access anatomical 3D models and CT scan data from standard workstations.
- It enhances equity and accessibility for students who cannot tolerate or afford VR headsets.

Platform & Deployment:

- Target Platform: Windows 10 and 11 (64-bit)
- Build System: Unity 6 Build Target – Windows Standalone (x86_64)
- Deployment Format: Installable .exe package with optional updater/patcher
- Storage: Bundled with local assets (or asset streaming from cloud if necessary)
- Graphics: DirectX 11+ support required, integrated GPU fallback with quality scaling

User Interaction & UI/UX Design:

The desktop interface will adapt the immersive VR elements for traditional 2D displays:

- Mouse/keyboard interaction will replace controller-based motion
- Camera movement and model rotation will be mapped to WASD + mouse or GUI controls
- Modal overlays and UI windows will replace in-world menus

- Hotkeys for toggling between interaction modes (e.g., ruler, slice view, labels, quiz)
- Visual accessibility support such as:
 - High-contrast modes
 - Adjustable font sizes and background dimming
 - Color-blind friendly palettes

Feature Adaptation:

VR Feature	Desktop Equivalent
Hand grab & distance grab	Left-click/drag for selection; mouse scroll zoom
Ray Interactors	Mouse pointer with context-sensitive cursor
Snap turning / teleportation	Camera orbit + navigation keys (WASD)
CT scan slicing planes	Slider UI + mouse-drag handles
Interactive organ models	Drag-to-rotate and click-to-highlight

	interface
Tutorial sequence	Intro slideshow or modal walk-through
Hotspots and labels	Hover or click overlays
Virtual quizzes	Embedded quiz screens + review feedback

Figure 1: Feature Adaptation VR to Desktop

Technical Considerations:

- Performance Scaling: Dynamic asset LOD (Level of Detail), texture compression, and shadow toggle options will help lower-end devices run the desktop version smoothly.
- Asset Handling: Large CT scans and 3D models may be streamed or lazily loaded to reduce memory footprint.
- Multiplatform Feasibility: Though the initial desktop version targets Windows, Unity's cross-platform support makes it theoretically portable to macOS and Linux in future iterations.
- Multilingual Support: Easier to integrate multilingual UI for wider reach, especially since screen real estate is more flexible on desktop than in VR.

Educational Use Cases:

- In-Class Demonstrations: Professors can load cases on-screen and walk students through CT interpretation.
- Remote Access: Students without VR-ready hardware at home can still participate in anatomy labs.
- Accessibility Scenarios: Students with motion sickness or disabilities that make VR use difficult can learn in a comfortable and familiar desktop setting.

Challenges:

- Reduced spatial immersion may impact retention for spatially dependent concepts
- Some features like ruler-based measuring or anatomical plane slicing might feel less intuitive
- The UI needs to balance medical accuracy with ease-of-use across all user technical skill levels

Web Application

To broaden access beyond VR headsets and Windows desktops, the Clinical Imaging VR application can be compiled and exported using Unity's WebGL build target. This allows the same Unity-based experience to be run directly in modern web browsers — making it a powerful, lightweight option for medical students and faculty to access CT scan data, quizzes, and instructional materials without needing to install anything.

This version maintains much of the underlying Unity architecture while adapting UI/UX for 2D mouse/keyboard interaction and optimizing performance for browser constraints.

Platform & Deployment:

- Build Target: Unity WebGL (HTML5 + JavaScript + WebAssembly bundle)
- Compatible Browsers: Chrome, Edge, Firefox, Safari (desktop only; mobile not supported by Unity WebGL)
- Hosting Options:
 - Azure Static Web Apps
 - Azure Blob Storage + CDN
 - UCF College of Medicine Internal Web Servers
- No Plugins or Installs Needed: Fully browser-native experience
- File Size Consideration: Optimized for initial download times (<200 MB target)

Intended Use Cases:

- Preview version for students before diving into the full VR experience
Study-from-anywhere access to quizzes and annotated CT content
- Faculty classroom demonstrations using projectors or screen-sharing
Non-VR fallback for accessibility or hardware-constrained students

Feature Scope:

VR Feature	WebGL
Full 3D VR room	Flat camera view with orbit/zoom controls
Controller input	Mouse + keyboard navigation
Teleportation/snap turning	Click-to-navigate or WASD/arrow keys
Ray interaction & hand grabbing	Mouse pointer clicks, drag & drop
Interactive CT scans	Scrollable CT slice viewer with zoom and pan
3D organ models	Orbit-view static models with basic manipulation
Drag-and-drop labels	Mouse-based drag-and-drop UI
Quizzes	Built-in quiz system with MCQs,

	answer feedback, and scoring
Instructional overlays	Tooltips, panels, and toggles using Unity UI Canvas
Subtitles and audio	Subtitles displayed on-screen; optional background narration
Tutorial/Intro room	Converted to slideshow or guided UI onboarding

Figure 2: Feature Adaptation VR to Web

Performance & Optimization Strategies:

- Texture Compression: WebGL-compatible formats (e.g., DXT5, ETC2)
- LOD Reduction: Simplified 3D models for browser runtime
- Scene Streaming: Load CT models/assets incrementally to reduce memory spikes
- Build Compression: Use gzip or Brotli for minimized file size + loading speed
- Memory Allocation: Fine-tuned WebGL template settings in Unity for memory budgeting

Unity WebGL can't access multithreading, local filesystems, or GPU compute shaders, so some visual fidelity and responsiveness will be traded off for portability and reach.

Limitations of WebGL:

- No native VR support (OpenXR not supported in WebGL)
- No headset or motion controller input
Lower frame rate and memory ceiling (~2GB in most browsers)
- Not supported on mobile browsers
- Longer initial load time (needs splash/loading screen)

Accessibility Considerations:

- Mouse-accessible UI with large hitboxes
- Adjustable font sizes, brightness, and color-blind palettes
- Toggleable subtitles for narration
- Keyboard shortcuts for common interactions
- Optional reduced-motion mode for low-end devices

Mobile Application

While the core functionality of the Clinical Imaging application is optimized for VR headsets and high-performance desktops, mobile support remains a compelling long-term expansion opportunity. Although real-time 3D rendering of anatomical models and CT volume slices is currently

impractical on most smartphones due to hardware limitations, a lightweight mobile version could offer complementary educational tools and asynchronous access to course material.

MVP Feasibility:

At present, the mobile platform presents significant barriers for full-feature parity:

- Limited CPU/GPU capabilities on mobile devices make it difficult to render and interact with 3D CT models smoothly.
Real-time anatomical slicing, raycasting, and shader effects (used for CT navigation and interaction) are not performant or consistent on Android/iOS.
- Mobile UI constraints such as screen size, lack of physical controls, and inconsistent aspect ratios introduce major UX redesign overhead.
- Unity mobile builds introduce new touch input schemes, asset scaling, and testing burdens that are out of scope for MVP timelines.

For these reasons, the mobile build will not be pursued during the primary development phase. However, the architecture is being designed with future modularity in mind, allowing for a tiered version of the app to run on mobile devices as a complementary tool.

Post-MVP Lightweight Mobile Version:

A future mobile companion app could focus on non-3D features, helping students engage with the material while commuting, studying offline, or preparing for class without needing a VR headset or desktop.

Potential Supported Features:

- Quiz Practice Mode:
 - Multiple-choice questions drawn from the backend question bank
 - Adaptive quiz difficulty or randomized sets
 - Review of past attempts with feedback and explanations
- 2D CT Image Viewer:
 - Static cross-sectional images (no dynamic slicing)
 - Simple pinch-to-zoom and swipe-to-scroll interface
 - Overlay of annotations or highlights for abnormalities
- Instructional Content Library:
 - Embedded short videos, audio narration, and labeled diagrams
 - Searchable anatomical glossaries
 - Text walkthroughs of procedures or scan interpretation
- Offline Mode:
 - Cache educational content and quizzes for use without internet
 - Sync performance data back to backend when reconnected
- Push Notifications:
 - Daily quiz reminders

- Study tip alerts
- “CT Scan of the Day” challenges

UI/UX Considerations:

The mobile experience must be heavily simplified and redesigned to ensure usability and accessibility:

- Large buttons and mobile-friendly layouts
- Toggleable contrast modes and font size controls
- Touch-first navigation (no drag-and-drop, use tap/hold instead)
- Reduced reliance on spatial metaphors—no freeform 3D navigation

Development Platform & Tools:

- Unity Mobile Build Targets:
 - Android (primary)
 - iOS (secondary, pending Apple TestFlight and deployment procedures)
- Backend Integration:
 - Same REST API as VR/Desktop/WebGL versions
 - Lightweight endpoints optimized for mobile bandwidth and latency
- Authentication:
 - Optional: UCF SSO integration or local guest mode

- Deployment Channels:
 - Google Play Store (initial beta via closed testing)
 - Apple App Store (TestFlight → public release)
 - APK sideloading option for UCF-controlled devices

Long-Term Opportunities:

If the mobile version gains traction, it could evolve into a full educational companion with more features:

- Personalized progress tracking with backend syncing
- Leaderboard or class-wide challenge support
- Augmented Reality (AR) support to place anatomical models in real-world environments (stretch goal)
- Integration with UCF COM course schedules, linking content to specific lessons or exams

*Note regarding additional platforms:

These platform extensions will only be pursued if the core VR experience is complete and stable. Priority will remain on ensuring the VR app fulfills its educational goals before expanding to non-VR interfaces.

Epics/User Stories

Epics are the big goals we aim to accomplish for the app and the user stories are the smaller steps we need to accomplish in order to complete the

bigger goal. Since our primary users for our app are medical students, our user stories will be in the voice of medical students. For our app we have 4 epics that we have decided make up the core features of our app.

Epic 1

Description: Help the user learn how to use the basic mechanics of the app before continuing to the main area.

User Stories:

- As a medical student, I want to complete a tutorial, so that I can understand how to complete basic interactions like picking up objects, swiping, and moving.
- As a medical student, I want to repeat the tutorial actions as many times as I want, so that I can feel comfortable navigating within the main area.
- As a medical student, I want an option in the menu, so that I can revisit the tutorial if I need to.
- As a medical student, I want an option to skip the tutorial, so that I can go straight to the main content if I'm comfortable navigating

Epic 2

Description: Allow the users to examine a variety of CT scans and 3D models.

User Stories:

- As a medical student, I want to view both normal and abnormal CT scans, so I can examine the difference between healthy and unhealthy anatomy.
- As a medical student, I want to rotate and zoom in on CT scans, so I can examine the structures more closely.
- As a medical student, I want to view 3D models of isolated organs, so I can understand the shape and position of the structures.
- As a medical student, I want to view both the CT scans and the 3D models, so that I can see the connection between the 2D scans and the 3D anatomical structures.
- As a medical student, I want to be able to see the different slices of the CT scan at once, so I can get an in-depth understanding of CT scans.
- As a medical student, I want a reference to the plane I am in within the body, so that I can orient myself properly.

Epic 3

Description: Provide study tools for the user such as labeling, measuring, and practice questions.

User Stories:

- As a medical student, I want to label structures on CT scans and 3D models so that I can practice anatomical identification.
- As a medical student, I want to use a virtual ruler to measure structures so that I can practice taking accurate measurements.
- As a medical student, I want to measure in centimeters so that I can adhere to medical standards.
- As a medical student, I want to answer practice questions about a CT scan or 3D model so I can test my understanding.
- As a medical student, I want immediate feedback after answering a question so that I know if I was correct and why.

Epic 4

Description: Provide a menu for users to customize their experience and ensure accessibility.

User Stories:

- As a medical student I want a menu so I can see all the customizations I can do easily and I can exit the game.
- As a medical student, I want to adjust the brightness of the screen so I can see comfortably.
- As a medical student, I want to adjust the sound of the game so I can hear comfortably.

- As a medical student, I want to hear background music or audio so it feels like a realistic clinical setting.
- As a medical student I want to be able to turn on or off the subtitles so I can understand instructions better.

Requirements

Prior to running the application ensure you have a working Meta Quest headset that is set up and connected to your Meta profile. This application should be downloadable from the UCF College of Medicine. If you are not downloading and using the application directly from the Meta Headset make sure your Meta Quest headset is connected to a PC that at minimum meets the requirements listed below. VR applications need to have a stronger than average PC to run properly and since this is a Clinical Imaging VR app with lots of models and images there is a higher demand for storage than normal, so ensure you have the space on your system before attempting to download. Finally, make your tracking space free of any obstacles when using the application, or if you prefer, you can play this application while seated.

- Compatible Headset - Meta Quest 2 or Meta Quest 3
 - Recommended Headset - Meta Quest 3
- Operating System - Windows 10 or Windows 11
 - Mac compatibility may be added in the future but currently only Windows devices are compatible with the application.
- Graphics - NVIDIA GTX 1660 / RTX 3050

- It is recommended to use at least a NVIDIA RTX 3060/3070 or AMD RX 6700 XT+ for the best experience but it can still run on the above mentioned graphics.
- CPU - Intel Core i5 or AMD Ryzen 5
- Memory - 6 GB RAM

Concept of Operations

The CT VR app is designed to help first-year medical students practice analyzing CT scans in an immersive, interactive environment. The user will need to login as an assigned user or as a guest when they first enter the application. After that, the game will begin with a short introduction scene where the player will be able to familiarize and orient themselves within the Virtual Reality world. After orienting themselves, the player can move on to the main room where they can examine normal and abnormal CT scans, measure different structures, and test their knowledge with practice questions.

The introduction scene will not only introduce what this app is but how to use it. We understand that VR is not a common learning resource and it takes a while to adjust within the VR environment. Therefore, before the player can get to the main room with the CT scans, we will add in a tutorial scene where the player can learn how to interact with objects. For example, learning how to pick up and put down the ruler, how to swipe through images, and how to answer questions. The player can stay as long or as short as they need in this scene based on their own comfort level and should they ever need help, we will provide a tutorial option in the settings menu.

In the main room of the app, the player can examine a variety of normal CT scans and abnormal CT scans. Our plan is to incorporate different modes for the player to pick from: one where the player can just study and look at the CT scans and another mode where there will be questions for the player to test their knowledge about the specific CT scan they'll have open. The practice questions will be in various formats: multiple choice, labeling, or even short answer. The player will need to use the CT scan and other tools such as the ruler to answer these questions. Once the player answers the question and submits it, they will receive immediate feedback. If they get the answer correct they can move on to the next CT scan or question, but if they get the answer incorrect they can either try again or check the answer. The correct answer will only be shown once they get it correct or if they check the answer, the answer will appear along with a short explanation. Additionally, we want to incorporate the "plane of the body" into the app so players can orient themselves to where they are looking at in the CT scan. It is our intent that students will be able to use this app to study and practice the things they have learned in their classes.

Design Documentation

Tech Stack

To deliver a responsive, immersive, and scalable clinical imaging VR application, the following technologies and tools will be used throughout development:

Frontend

Unity 6

The VR application will be using the latest version of Unity 6 - currently 6000.1.10f1 - for its faster rendering and the best OpenXR support. This will be where the virtual reality lab environment will be designed. All the scenes, animations, and interactions will be made using Unity software. Meta also has SDKs for Unity that allow us to use its existing components. Unity makes it easy for us to deploy our application to multiple platforms.

Blender

Blender is a program used to create 3D models. The team will be using Blender to create or alter any 3D models either not provided by the UCF College of Medicine or to change any previously provided 3D model assets. This includes medical tools, organs, or items required for the lab environment.

3D Slicer

3D Slicer is an application used to view CT scans in the axial, sagittal, and coronal planes, and is also used to convert those CT scans to 3D models. This serves as the model we'll want to use for our own CT scan viewer - we'll want to have similar implementation, UI design, and capabilities - the idea is that our application should provide a similar experience, only using VR functionality.

OpenXR

This is a standard for virtual reality applications and is already built into Unity. It will be providing important API's so the VR app can run properly, such as the headset and controller tracking. OpenXR being the standard means it works across different platforms, including different Meta Quest Headsets. Should this VR app grow beyond just Meta Quest 2s and 3s then the use of OpenXR APIs will not hinder cross functionality.

Visual Studio Code

The recommended coding environment for Unity logic is written in Visual Studio Code. To mesh better with similar projects and with Unity's source code we have chosen to use C# as our primary coding language. We will be utilizing this to write scripts and interactions, as well as allowing the frontend to make API calls to the database in order to load quiz questions and return student answers. The College of Medicine has given each of us licenses for Visual Studio Professional, which allows us to use Azure DevOps resources such as repos and pipelines.

Figma

Since this will be a virtual reality environment and Figma is used for 2D UI planning, we will be using this for the very first rough draft. This will be utilized for planning the initial layout of the lab and for planning the look of the UI elements, such as the controls, quiz sections, and options menu. We can also utilize Figma to directly create various UI elements and export them to Unity.

Backend

Microsoft Azure

The UCF College of Medicine uses Microsoft Azure for its SQL databases and server hosting. We'll use this to host our backend and to store quiz questions and answers. It's a natural choice for working with .NET as well, allowing us to integrate our backend and frontend easily since both will be making use of C#. Included in our Visual Studio Professional subscription is a \$50 monthly Azure credit, which among the seven of us will be more than enough to enable us to run our servers for testing, considering how little data will need to be sent back and forth.

Administration

Jira

Jira is a website that is used to facilitate the various aspects of AGILE programming by giving us a way to easily manage all of our administration tasks for the project. It allows us to create and manage sprint maps for our project and keep a running backlog for everything. We are also able to assign tasks to specific members and keep track of everything that we need to get done for the project and where we should be at a certain point in time for it.

Azure Devops Repos

Our version control will also be hosted on Azure, since the UCF College of Medicine has licenses for working with Azure Devops. While we're building our application, we'll be using an Azure git repository to make push and pull

requests and keep track of version history in case a change needs to be rolled back. This also allows us to view the repos for the CardiacVR and CardiacAR apps, which helps us maintain stylistic consistency between our apps.

Microsoft Teams

We'll be using Microsoft Teams for sharing files for various resources among our team and with our sponsor, as well as for communicating asynchronously. The channel also gives us access to the files and resources shared by several other project teams who have worked on College of Medicine projects - most notably the CardiacVR, CardiacAR, and Dr. Hemo's Great Escape apps. This allows us to view previous versions of each app, some of the assets they've used, as well as the design documentation they've submitted for their senior design courses.

Zoom

In order to facilitate better communication with our sponsor, we are using Zoom as our primary means of hosting our weekly sponsor meetings. This allows us to meet with our sponsor on video calls, and make use of features like screen sharing to share visuals of our current development progress.

Discord

We're currently using Discord for the majority of our in-team communication, including voice calls for our weekly team meetings and while

collaborating on code or assignments. This also serves as a secondary repository of resources, sharing collaborative documents and helpful links to resources useful for learning about the various concepts we're working with. We're also able to use Discord to connect with other groups working on Senior Design projects for the College of Medicine, such as the team currently working on Dr. Hemo's Great Escape.

SideQuest

SideQuest allows us to load APKs to the Meta Quest headset without needing to publish the application on the Quest store first. This is indispensable when developing and testing for the headsets. It has also enabled us to try out previous College of Medicine projects on our own devices.

Hardware

Meta Quest 2 and 3

The Quest 2 and 3 are standalone VR headsets from Meta. We've received development headsets from the UCF College of Medicine for testing and deployment. These headsets are capable of running standalone VR apps, including ones sideloaded as APKs, although they're also able to connect to PCs through Meta's Link and Air Link, which enables us to test our application through Unity without redeploying the entire application. Since we'll be using OpenXR, the application will easily run as either PCVR or standalone VR for testing.

Research

For the initial research Melissa Cowan, the sponsor for this project, gave the team two links to teach each member about using Unity, using Meta, and using Quest headsets. The first was Unity Learn pathways which had tons of options from teaching the basics of Unity, teaching C# which is the coding language Unity uses for VR development, and a pathway for VR development in Unity. The pathway that Unity uses to teach VR development guides users into creating a couple of initial VR worlds in order to give developers a feel for what can be made. The second was Meta Horizon developer which, similar to Unity Learn, teaches about VR development in Unity. Meta Horizon is a lot more in depth, going into what SDKs are and how to use them, teaching users about how to use spatial audio, and of course going over the basics like creating small interactions and animating to make the world come to life.

Unity Learn

The Unity Learn VR development pathway served as a solid introduction to creating virtual reality experiences using Unity. The path started by learning the Unity interface, scene creation, and then basic interactions with objects. This was helpful for those of us on the team who were new to Unity or C# programming and gave us a structured and self-paced learning environment.

As the pathway progressed, it shifted to more specific topics, such as setting up XR plugins, configuring VR rigs, and handling user input from headsets and controllers. One of the most useful aspects was how hands-on the Learning Path was. We were guided through creating a couple of small

VR worlds. These worlds served not only as technical exercises but also as creative prompts, giving a sense of the types of experiences that can be crafted in VR and what we can innovate into our project. They included placing interactable objects, setting up teleportation systems, and fine-tuning environments to be immersive and intuitive.

In order to give users a good understanding of the tools and concepts Unity learn has a variety of learning aids including videos, loosely guided tasks, and quizzes. The biggest part of the pathways was giving the users the creative freedom to make something. Below is a VR room a team member created as a “task” from a part of the pathway. The room itself is the only thing the step-by-step guide has you set up, it's up to the user to decorate, change the lighting, camera, background, and edit however they like.



Figure 3: Ari's Room - Image made by a team member during the Unity VR Pathways (taken by Arianna).

By the end of the Unity Learn pathway, the team had gained a strong understanding of the Unity development environment, the core concepts of VR design, and how to begin building responsive and engaging virtual worlds.

Meta Horizon

The Meta Horizon Develop for Unity lessons offered a deeper, more technical dive into VR development tailored specifically for Meta Quest headsets. It started with a review of the basics—setting up Unity projects for VR, importing the appropriate SDKs, and preparing a project for Meta Quest deployment—but quickly expanded into more advanced territory.

A standout aspect of the Meta Horizon lessons was the emphasis on Meta’s SDKs (Software Development Kits). These tools are essential for tapping into the full range of Quest hardware features, such as hand tracking, passthrough, and controller-based interaction. The lessons went in-depth into how to implement these tools effectively, providing code samples, walkthroughs, and real-world examples.

Another key component was spatial audio, which greatly enhanced immersion in VR. The lessons explained how to use Unity’s audio tools in combination with Meta’s spatial audio SDK to make sound directionally accurate and responsive to player movement. This added a whole new level of realism to the VR experiences being developed.

Beyond that, the Meta lessons also included topics like performance optimization, interaction scripting, and animating objects and avatars to

bring a VR world to life. Compared to Unity Learn, Meta's resources assumed a bit more technical familiarity, but they offered critical knowledge for building polished, performant VR experiences specifically for the Meta ecosystem

Other VR Apps

Our team reviewed Dr. Hemo's Great Escape and Cardiac VR, which were developed by the previous Senior Design teams. These projects proved to be great references for the team as they helped us to understand the capabilities and the constraints of the VR development lifecycle, and helped to make our own goals for our project. Both games showed a clear understanding of Unity-based development, different ways in which the UI and interactivity work, environmental design, storytelling, and the general path in which the game is played. Being able to test the game gives us the ability to play-test something similar to our own game, so we could pick things we liked and also find bugs or quirks that we would do differently.

Dr. Hemo's Great Escape

We were really excited to play this game when we were given access to Meta Quest 2s. Designed to be an educational escape room experience, this game gave us a clear outlook on how our project might develop, and also gave us more insight into the bugs/quirks that will appear in our game.

One of the first things our team found was that clipping out of bounds was surprisingly really easy to do. The Player only had to stick their head through a wall or misalign themselves when walking near a wall, and the

game would reset them to the spawn. Even though this mechanic was technically a failsafe so the player wouldn't be falling forever, it can quickly become frustrating as it breaks any immersion the player could have. It's a small reminder for our team that even though minor bugs may occur they can have drastic player disruptions. Encountering a bug in a VR game is significantly worse than on any console/PC game, since VR games strictly rely on immersing the Player in the VR world.

Dr. Hemo's Escape Room would spawn you in a small lobby with interactables like a vending machine, tables, plants etc. The way the Player progresses from this room to the actual game is via an elevator. We discovered that, as cool as elevators are, they introduce a new set of challenges, both for developing, and for the Player to use. As the Player, having an elevator be on the top floor and clipping back to spawn would be incredibly annoying and tedious. As the elevator would ascend or descend the building, different sprites of walls or environments would clip through the door. Looking back, we wondered if a simple staircase could have done the job better, with fewer frustrations. That being said, we can also understand the appeal of having an elevator in a game like this. As a doctor in a 40 floor building, it wouldn't be expected to take the staircase everyday to the 39th floor. And more importantly, elevators are really cool! They are visually interesting and if implemented appropriately, can add a lot to a game. It's amazing that the development team chose to learn more advanced scripting and event handling to implement this feature. In many ways the elevator serves as both a cool design feature and a great learning opportunity.

The game also dove deeper into multiplayer design and a real-time feedback system that allowed players to interact more naturally with the

environment and with others. This was an eye-opener for our team, since it highlights the complexity of syncing all the player interactions in the game. This is something that may seem easy at first but becomes a crucial feature in any multiplayer setting. It developed more ideas for our team to think about possibly adding this feature to our game, even if it's not on the MVP. It also helped us to think about how we would sync the player movements and record them to appropriately address our MVP.

The biggest takeaway from learning about this game was the importance of goal setting during our project's development. On paper, it's easy to implement ambitious features or complex environments. But until we can sit down and actually begin the development, it's incredibly difficult to be realistic with what features will actually work. It showed us that starting small and actually understanding and developing what's on our MVP is essential. Most of our research has been laying the foundations and giving ourselves ample room to expand the game. We learned that many features that at first glance may look simple, actually introduced several layers of complexity. Other features we didn't talk about like the computer screen quizzes, seemed difficult to implement at first, but when broken down seemed to become slightly more manageable.

Overall, Dr. Hemo's Great Escape served as a blueprint and a warning. From this game we were able to individually shape our expectations and see what was possible to implement. It emphasized the importance of balancing being creative with doing what is practical. We know now to anticipate many MANY bugs along the way. Each bug or quirk in the game was a massive learning moment for our team that we can now carry forward to our project. From this, we will be staying flexible with our work, to adapt to the

inevitable surprises of game development and build the game from the ground up.

Cardiac VR



Figure 4: Screenshot from the Cardiac VR App

This game dove deep into 3D design implementation of hearts with interactive flashcards to teach different aspects of the heart and how it works. It served as a meaningful reference for us, particularly with importing and showing 3D models and managing object interactivity. Per our sponsor's expectations, our app should be similar to this in terms of design and usage, so it's important that we study it closely in order to ensure there's a consistent feeling and quality. Even though there were some flaws and limitations, the game still offered lots of educational value and allowed us to discuss how we would implement this into our game.

One of the best things this game had to offer was how it implemented 3D design in a meaningful way. The senior design team was able to create an explorable space that mimicked a real world doctor's office with rooms and a receptionist. The actual doctors rooms were designed as they would be in real life, with a bed and several monitors. On the beds were heart models that accurately showed the different areas of the heart and also offered different ways to visualize the heart such as increasing heart rate or showing how blood was pumping. As we're learning to import 3D models into our own project, being able to see how the assets behaved in this game gave us clarity on potential upcoming problems.

The UI for this game was polished and very well done. The entire experience was highly influential for how we want our game to look. The visual hierarchy, way buttons were laid out, and way feedback systems worked with the player set a standard for our project in how we want to design similar structures.

However, there were some issues with the app that we can hopefully learn from when designing our app. Noticeably, the world scaling was off, and instead players' heights are adjusted so that most objects remain at eye level. This might not be noticeable for some users, but it can lower immersion and potentially increase virtual sickness. There were also some instances where UI elements could be placed closer to the items they interact with, to help alleviate confusion. Finally, the distance grabbing would have been more useful if it didn't teleport the items to your hand - in an application like this where you're viewing models and charts, you don't want a grab to constantly reset what you're looking at.

Embedding The Best Medical Practices

Our game will model and reinforce real clinical standards that students can learn in a fun environment. These are some ideas for the best practices we wish to implement in our game:

"ABCDE" approach to Trauma Scans with Comparison

What it is:

In CT imaging, clinicians follow protocols like the ABCDE method (Airway, Breathing, Circulation, Disability, and Exposure) to ensure that no detail is missed. Similarly, this can be applied to cross-sectional imaging which requires comparing the axial, sagittal, and coronal views to localize pathology.

Possible Gameplay Implementation:

- Tutorial will introduce the ABCDE approach with a checklist or guided overlays
- Progressive Unlocking will allow the student to correctly identify findings from axial views before moving to sagittal or coronal planes
- Timer to simulate a real world setting where the player can get an idea of how long it takes to diagnose under pressure.

Why it Matters:

Systematic approaches like these will help the student in reducing their diagnostic errors in the field. Embedding them into play should help the student routinely discover diagnoses with the best approach.

Clinical Reasoning

Reviewing History -> Ordering Imaging -> Interpreting Information

What it is:

Clinicians can synthesize clinical history, symptoms, labs, and choose the right imaging modality before they are analyzed in context.

Possible Gameplay Implementation:

- Clinical scenario question where the student must choose the right imaging modality. It will look something like this:

Patient Case:

A 62 Year old male shows up to the emergency room with acute shortness of breath and pleuritic chest pain.

Vitals: HR: 106 BPM RR: 25/min BP: 126/82 O2 sat: 93%

Past medical History: Hypertension, recent hip surgery

Physical Exam: Mild tachypnea, clear lungs, no wheezing or crackles, calf is tender and swollen

Question: What is the most appropriate imaging modality to investigate the patient's symptoms?

Answers : Chest X-ray, CTPA, Lower extremity Ultrasound, V/Q scan

Why It matters:

Understanding the right imaging to use on a patient is VITAL to the diagnosis procedure. Teaching players to connect the imaging to real patient data makes the game more realistic and a better learning experience. We want the player to fully understand the concept, not just understand pattern recognition.

Team-Based Decision Making

What it is:

Diagnosis nowadays relies on a strong team of individuals to work together and form critical decision making. This is done so that the case is easier to diagnose as there are multiple angles in which the diagnosis can go. Because this isn't a multiplayer game, it will be slightly different.

Possible Gameplay Implementation:

- Dialogue from pre-recorded messages or an AI can give information that can be correct or not entirely on-the-nose, and the player can form a diagnosis based on the information.
- If questions are too hard we can implement another clinicians take on the data in the form of a hint.

Why It matters:

Confidence in diagnosis grows when clinicians can take from multiple viewpoints. Simulating this dynamic can accurately represent what the student will experience in the field.

Pattern Recognition

What it is:

It is important that the student can interpret scans by understanding what subtle patterns to look out for. Though this takes many years to build, we want our game to further the students' exposure to the pattern building process.

Possible Gameplay Implementation:

- We would expose the player to normal scans involving a diagnosis, and move to abnormal cases where it may be harder to diagnose.
- Compare and learn process, where multiple CT scans can be viewed comparing normal scans vs. abnormal

Why It matters:

Diagnostic fluency improves as students repeatedly expose themselves to normal scans and edge cases. We want our game to provide the volume and variation that other sources are unable to provide in this game-like setting.

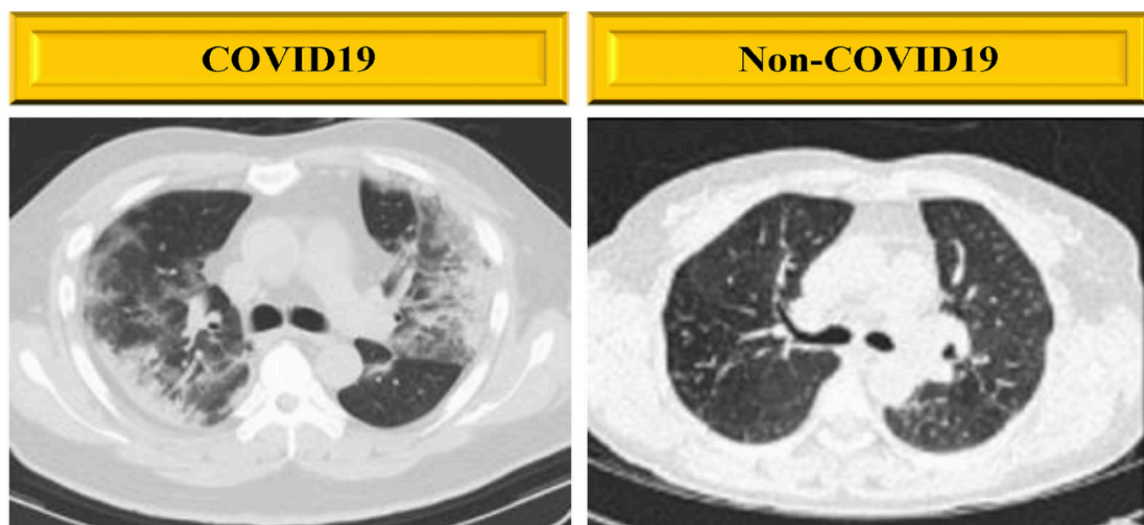


Figure 5: Patterns in Covid-19 CT Scans

Error Recognition

What it is:

A crucial part of becoming a good clinician is finding, understanding, and avoiding errors. In the field it is easy to make errors such as:

Anchoring bias (locking in on a diagnosis too early) or accidental blindness (missing obvious signs when distracted)

Possible Gameplay Implementation:

- Inserting misleading or irrelevant information into a patient's history.
- Provide post-case feedback on missed information
- Having multiple-choice questions with multiple correct answers

Why It Matters:

Error awareness is crucial to expert-level clinicians. We want to be able to train students to pause, double-check and challenge their assumptions so that they can learn the more methodical and cautious approach. This is something that books and classrooms cannot teach.

Time Sensitive Imaging

What it is:

In emergencies there is a very limited window in which the diagnostic procedure can take place. Immediate imaging is crucial.

Possible Gameplay Implementation:

- Time sensitive cases (Ex: CT head scan for stroke - 30 minutes)
- Clock implementation to pressure the player

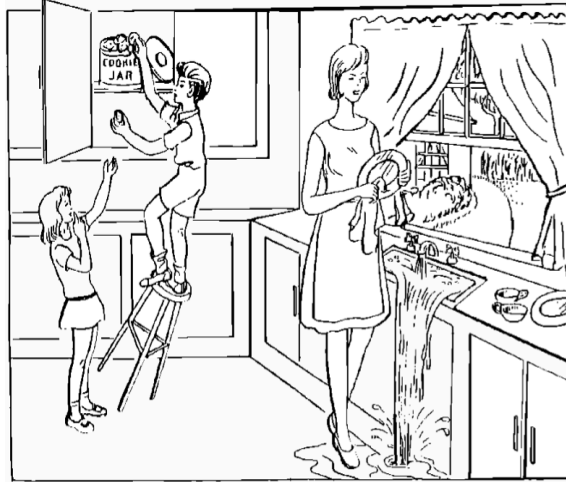


Figure 6: Example Image used with the NIH Stroke Scale

Integrate with NIH Stroke Scale-based Questions

What's wrong with this picture? Level of consciousness:

0 = Alert and responsive

1 = Arousable to minor stimulation

2 = Arousable only to painful stimulation

3 = Unarousable or reflex responses

Why It Matters:

Students can learn the urgency of imaging in cases where delays will affect patient outcomes. This reinforces their judgment under pressure.

Image-Guided Interventions and Follow-Up Protocols

What it is:

CTs often guide to procedures like biopsies or post-op assessments. Understanding how CTs integrate into longitudinal care is essential.

Possible Gameplay Implementation:

- After the student is presented with a scan, follow up questions can be: “Would this patient benefit from a lung biopsy?”
- Including cases with showing progression in scans (Tumor growing)

Why It Matters:

Reinforces the role of imaging in the entire process instead of just the diagnosis. In the real world, a CT scan is just part of an ongoing process, not a one-off task.

Utilizing Clinical Decision Rules (CDRs)

Why it is:

Many best practices use CDRs to guide their decisions.

Possible Gameplay Implementation:

- Questions like: “Does this patient meet criteria for CT head?”
- Decision Trees appear after answers for visual learning

Why It Matters:

Encourages judicious imaging and accountability.

Development Process

Initial Development

After completing the Unity Learn Pathways, going through the Meta Horizons Developer lessons, and watching Youtube tutorials to have a better grasp on the technologies and how to use them to create the CT VR Application, the team started development. Before the Azure Devops was set up, the level designers, Zoe and Arianna, each had their own Unity files and discussed which sections they would be in charge of designing and creating in Unity so as to avoid overlapping. After the Azure was set up and running a combined Unity project was made and all changes were pushed and merged into a single scene which will act as the main file for the project.

The layout was initially pitched to the sponsor in Figma and after some adjustments was recreated in the Unity Project. The layout was made with boundaries in place to stop users from traveling through walls or the floor. The 3D models and textures provided were then uploaded into the project to make the lab scene. XR package was downloaded and set up in the project, the team used a "VR Device Simulator" in order to view the project as if a headset was being used, but without a headset and instead with a mouse and keyboard. The "camera"'s tracking origin must be set to "floor" in order to account for a person's height while using the headset. Changes to buttons are done through XR Input Interaction and changing the scripting.

3D Model Design Process

The design for creating the 3D models begins with obtaining the patient datasets from our sponsor Melissa Cowan. We received 5 datasets in the form of DiCOM files which typically range from 90-180mb of raw CT data and patient metadata. Because we're working with the Meta Quest 2 and 3s, there were limitations on how big these files could be, and we decided to strip them of 90% storage they take up.

Development Issues

Setup

During the development process there were countless issues that arose and needed to be fixed before proceeding with development. The first issue was during the setup of the Azure Devops and Github Repos themselves. Before fixing this issue the team members were each working off their own Unity project files which was not ideal because merging the projects together would cause more issues. During the merge, since all the packages in the different Unity copies did not match each other, there were conflicts. After downloading and updating all the used packages into the shared Unity project file the conflicts were resolved.

3D Models and Textures

Another problem faced was when uploading the 3D models and textures from UCF College of Medicine, certain 3D models would not allow the textures to be placed on them. It was found that the issue had to do with how the models were uploaded. If its parts were split into different components then it would not combine with any textures properly. If the model was one single component then the texture could be placed with no issue. This was resolved by adding a “Materials” component to each piece of the 3D model and then placing the texture on each “Materials” component.

Continuous Integration

As part of the Clinical Imaging VR project’s infrastructure, a robust CI/CD pipeline was established to streamline the build and deployment process. This system was critical to ensuring consistent integration and rapid feedback during development, particularly as multiple branches and developers were actively working on Unity-based code for a virtual reality environment. The approach taken by the team emphasized automation, build efficiency, and scalability, even during the early stages before institutional resources were fully accessible.

Multi-Branch Development Strategy

One of the core practices brought in from internship and co-op experience was the enforcement of a multi-branch development model. Separate branches were created for features, fixes, and experimental work to avoid polluting the main or release branches. This strategy enabled

isolated development, reduced the likelihood of merge conflicts, and made code review through pull requests much more manageable.

Temporary GitHub Pipeline Before Azure Migration

Due to delays in acquiring access to Microsoft Azure DevOps, the team initially adopted GitHub as a temporary remote repository and CI/CD environment. During this period, the priority was to establish an automated build system that would validate Unity projects in headless environments.

The team first resolved licensing issues related to CI/CD usage of Unity. Through research, it was discovered that Unity offers free licenses for educational use. A student license was successfully activated as a temporary measure until the sponsor provided full Unity Pro or Enterprise licenses for production builds.

Build Time Optimization and Performance Bottlenecks

Initial build runs in GitHub Actions revealed a significant performance bottleneck, with average build times exceeding 15 minutes per job. This inefficiency threatened to slow down development cycles and complicate regression testing. As such, optimizing the Unity build process became a high priority.

Unity Docker Image and Preloading Strategy

After investigating performance improvement options, the team opted to preload Unity into a Docker image to reduce setup time during CI builds. The process began with preparing Unity 6 for Linux server environments. Since Unity does not officially provide Docker images with pre-installed editors, the team manually packaged Unity 6 into a .tar.gz archive.

To accomplish this:

- A working version of 7-Zip was first configured within PowerShell.
- The Unity editor installation was compressed into the .tar.gz format using PowerShell scripting and appropriate flags.
- A Dockerfile was authored using a minimal Ubuntu 22.04 base image.
- The Dockerfile extracted the Unity archive and installed all necessary dependencies for headless Unity builds (e.g., libgtk-3, xvfb, mesa-utils).

```
Login Succeeded
PS C:\Users\Matt\ClinicalImagingCTVR\.github> docker build -t ghcr.io/mattlmatthew/unity-6000.1.6f1:latest .
[+] Building 266.9s (8/9)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 439B
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/ubuntu:22.04@sha256:3c61d3759c2639d4b836d32a2d3c83fa0214e36f195a3421018dbaaf79cb
=> => resolve docker.io/library/ubuntu:22.04@sha256:3c61d3759c2639d4b836d32a2d3c83fa0214e36f195a3421018dbaaf79cb
=> => sha256:e735f3a6b70199ad991c5715d965a4d858540eca2be18be0d889698e5a0a3e8c 29.54MB / 29.54MB
=> => extracting sha256:e735f3a6b70199ad991c5715d965a4d858540eca2be18be0d889698e5a0a3e8c
=> [internal] load build context
=> => transferring context: 6.36GB
=> [2/4] RUN apt-get update && apt-get install -y wget curl unzip git libgtk-3-dev libgl1-mesa-dev libglu1-
=> [3/4] COPY Unity_6000.1.6f1.tar.gz /tmp/unity.tar.gz
=> [4/4] RUN mkdir -p /opt/unity && tar -xzf /tmp/unity.tar.gz -C /opt/unity && rm /tmp/unity.tar.gz
=> exporting to image
=> => exporting layers
```

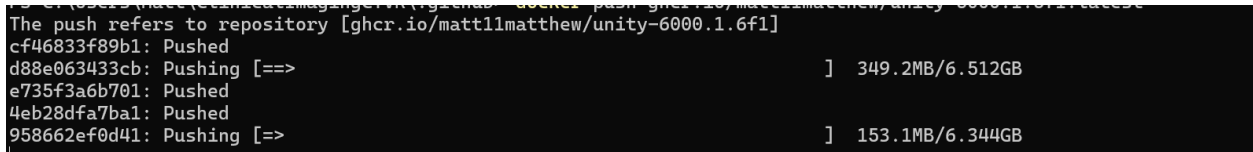
Figure 7: Docker Image Build Process (taken by Matthew Eisenberg)

Private Deployment to GHCR

Once the image was built locally and tested, it was pushed to the GitHub Container Registry (ghcr.io). This required:

- Creation of a GitHub personal access token (PAT) with the appropriate write:packages and read:packages scopes.
- Docker login to GHCR using the PAT.
- Tagging the image according to GHCR format and pushing it to the temporary student GitHub organization.

This private image allowed Unity to be preloaded during CI runs, reducing the setup phase of the build from over 10 minutes to approximately 2 minutes.

A terminal window with a black background and white text. The text shows the output of a 'docker push' command. It starts with 'The push refers to repository [ghcr.io/matt11matthew/unity-6000.1.6f1]'. Then, several lines show progress for different image layers: 'cf46833f89b1: Pushed', 'd88e063433cb: Pushing [==>] 349.2MB/6.512GB', 'e735f3a6b701: Pushed', '4eb28dfa7ba1: Pushed', and '958662ef0d41: Pushing [==>] 153.1MB/6.344GB'.

```
The push refers to repository [ghcr.io/matt11matthew/unity-6000.1.6f1]
cf46833f89b1: Pushed
d88e063433cb: Pushing [==> ] 349.2MB/6.512GB
e735f3a6b701: Pushed
4eb28dfa7ba1: Pushed
958662ef0d41: Pushing [==> ] 153.1MB/6.344GB
```

Figure 8: Docker Publishing Process (Taken by Matthew Eisenberg)

Image Sharing Limitations and Re-tagging

A notable challenge emerged when attempting to share the Docker image across different GitHub organizations. GHCR imposed limitations on private image access controls. To work around this:

- The image was re-tagged under the new organization's namespace.
- It was then pushed again under the shared team-owned GitHub container registry, enabling access through the CI/CD workflows of other members.

Fallback to Unity Cache System

While the Docker solution significantly improved build times, GitHub Actions workflows ran into disk space limitations due to Unity's storage-intensive operations and cache footprint. Given the 14 GB cap for ephemeral runners on GitHub Actions, full builds began to fail under constrained environments.

To maintain reliability, the team temporarily reverted to Unity's cache-based build system. Although slower than Docker-based preloading, the build time was reduced to approximately 5–6 minutes using Unity's `-cacheServerEnable` flag and Unity Builder's optimized actions. This approach proved sufficient for functional production tests and PR validations.



 Update unity-build.yml Unity Build (Pro License via Serial) #6: Commit d47fba8 pushed by matt11matthew	xr-integration	 1 hour ago  13m 32s	...
 Update unity-build.yml Unity Build (Pro License via Serial) #5: Commit 6906467 pushed by matt11matthew	xr-integration	 2 hours ago  13m 41s	...

Figure 9: GitHub Test Action Results (Taken by Matthew Eisenberg)

Repository and Git Ignore Standards

The Unity project was created using the Universal Render Pipeline (URP) to meet cross-platform requirements. A dedicated GitHub repository was created and configured with Unity's recommended `.gitignore` template to avoid syncing unnecessary project files such as `/Library`, `/Temp`, `/Builds`, and user-specific settings.

This ensured lightweight commits, reduced merge conflicts, and improved CI build reproducibility by excluding volatile build artifacts.

OpenXR Integration and Pipeline Configuration

Once the Unity project was configured, OpenXR was manually imported to support Meta Quest 2 and 3 headsets. The setup process included:

- Adding Unity's XR Plugin Management system.
- Installing the OpenXR package via Unity Package Manager.
- Configuring the XR loader and setting tracking origin to "floor" for proper headset alignment.

After integration, several builds were tested through CI to verify compatibility with OpenXR and ensure successful APK output for sideloading.

Migration to Azure DevOps Pipelines

Once Azure access was restored, the team transitioned the build infrastructure to Azure DevOps Pipelines. The GitHub repository was mirrored to Azure DevOps Repos for version control. Existing GitHub workflows were ported to .azure-pipelines.yml format using Microsoft's Unity template as a base.

Key steps included:

- Configuring self-hosted agents with sufficient disk space for Unity builds.
- Integrating Azure Artifacts to host pipeline build outputs.
- Setting up Azure Pipelines variables for secret management (e.g., Unity license, custom parameters).
- Deploying to Azure Storage for APK archiving and internal sideloading.

The team continued to use branch protection rules and enforced pull request workflows to promote best practices, such as code reviews, testing gates, and CI checks before merge approval.

Developer Onboarding and Access Management

With Azure access granted to all team members, accounts were configured with appropriate role-based permissions. A brief onboarding session was conducted to explain Git operations, Azure Boards for task tracking, and the CI/CD pipeline flow.

Each member was guided through:

- Creating a personal fork for feature work.
- Submitting pull requests for review.
- Using branch naming conventions (feature/, bugfix/, test/) to improve clarity.

- Triggering manual and scheduled builds.

Summary and Outcomes

The CI/CD system implemented during development proved critical for consistent, fast iteration in a large Unity project. While several technical challenges were encountered—including licensing, disk limits, and cross-organization registry permissions—the team successfully delivered a fully automated pipeline that supported local and cloud-based testing, APK delivery, and Azure integration.

This infrastructure laid the foundation for continuous improvement, enabling rapid deployment of new features, minimized risk of regressions, and improved code quality through automated checks. It also serves as a replicable model for future Unity VR projects within the UCF College of Medicine.

Design Summary

Block Diagram

The app will use a C# backend to make API calls to the UCF COM Azure SQL database, where quiz questions and student responses will be stored. While most elements of the app will work without needing to be connected to the server, the quizzes will need to make use of API calls to the server in order to function. The app will also likely require that users login with their UCF credentials in order to make use of this functionality.

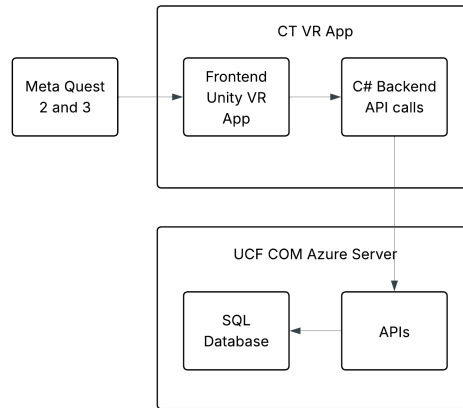


Figure 10: Block Diagram

Use Case Diagram

Students will have the ability to take quizzes and view and interact with various CT scans and models. Instructors will have the ability to view quiz results and add new quizzes, while admins will be able to add new scans and models to the application, which will be pushed as updates.

All of the functionality on the student side will be implemented through the VR application, loaded on Quest 2 and 3 headsets. However, the instructor and admin functions will be managed either by directly managing the Azure database for managing quizzes, or by updating the resources in the application and pushing a new version of the app, which should

automatically update for the headsets where it is already installed.

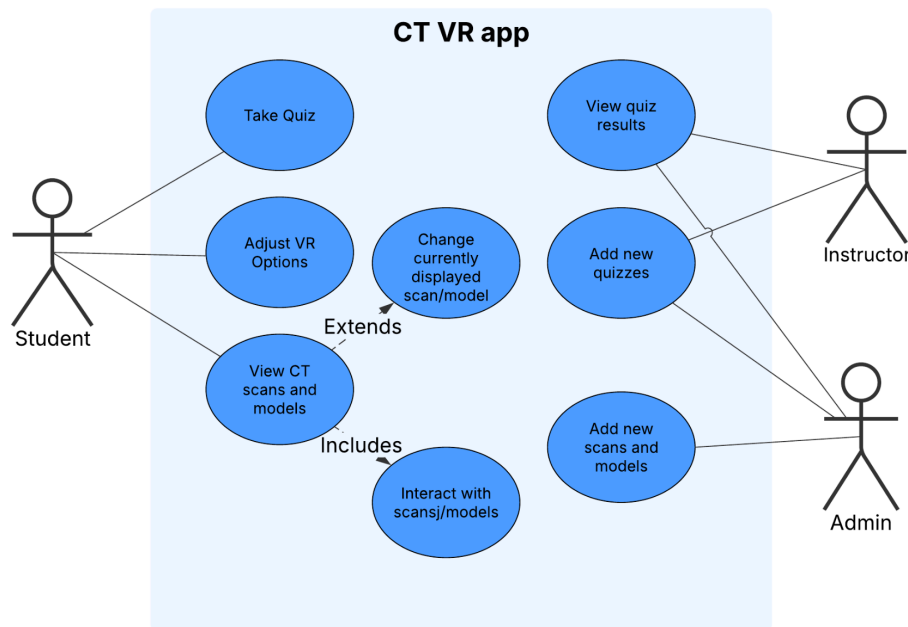


Figure 11: Use Case Diagram

Activity Diagram

The activity diagram demonstrates how the frontend with the user interacts with the backend in order to take a quiz. The quizzes are loaded after the user has logged in and the quiz itself is loaded after the quiz is selected. Further communication only happens when the quiz is completed, sending the chosen answers and marking them against which ones are right and then sending back the user's score. The application will need to communicate with the Azure SQL server through API calls to load the list of

quizzes, the questions for a chosen quiz, the correct answers for those questions, and to store the user's answers and score in the database.

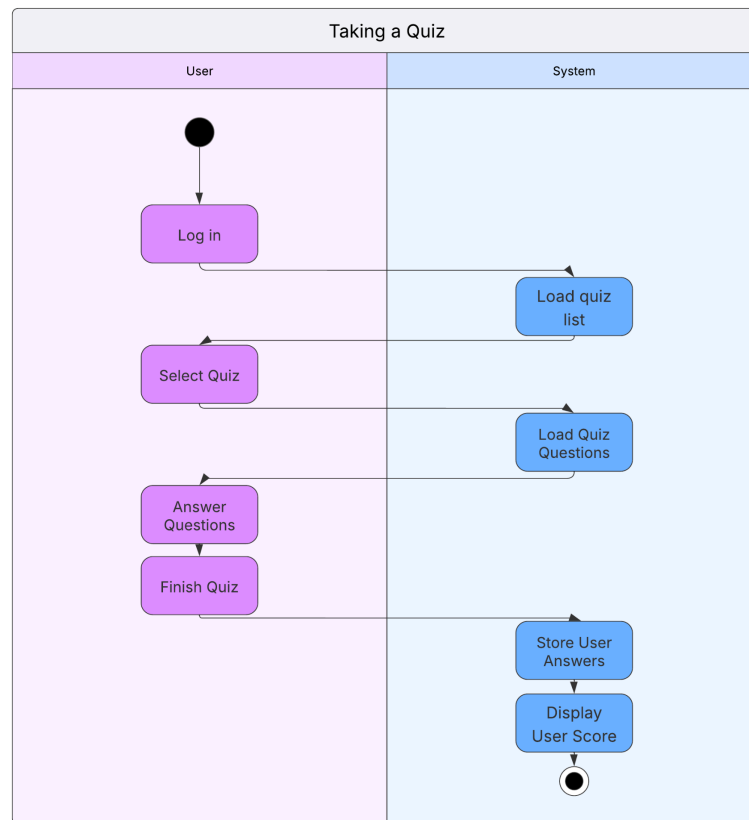


Figure 12: Activity Diagram for taking quiz

Entity Relationship Diagram

The database will store a certain number of quizzes, each with their own ID value. Each will have a name (to be shown to the user) and a number of questions, which can be used to retrieve a given question. Each

time a user takes the quiz, their final score will also be saved, attached to the quiz ID.

Each quiz will have a series of questions, which will be referenced by their quiz ID and question number. They will also have a unique question ID for referencing answer values. Also stored will be the question and answer explanation (if the user gets the question wrong) as strings. Finally, each quiz will contain an answer char (A, B, C, D) containing the answer of the correct letter.

Finally, there will be answers corresponding to each question. Each answer will be keyed by question ID and answer letter. Each will contain the answer description as a string. Each answer will also count the total number of times that answer has been chosen, allowing us to determine commonly missed answers.

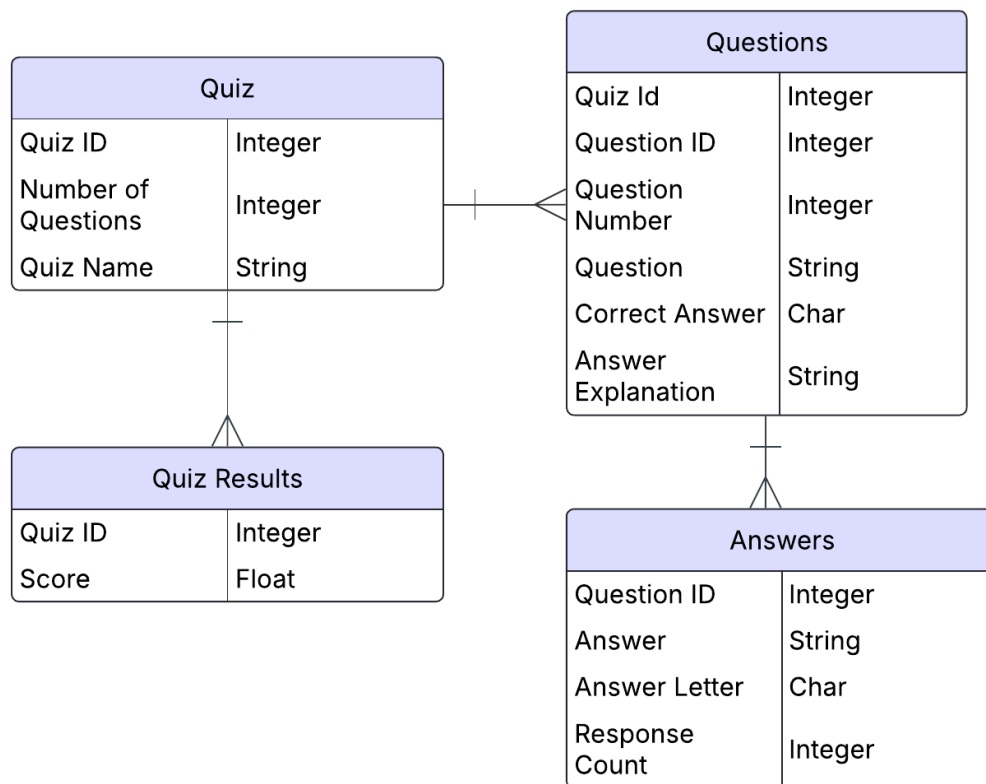


Figure 13: Entity Relationship Diagram for Quiz Database

Detailed Description

Accessibility

Accessibility has been previously gone over briefly. As stated, we will be following the UCF's accessibility guidelines to the best of our ability with virtual reality in order to provide the most enjoyable and smooth experience for all users.

VR Comfortability Settings

This application is intended to be used by first year med students - for many, this will be one of their first experiences using VR. For many people who are unused to VR, it's common to experience VR motion sickness, as the movement in the game doesn't correspond to the movement as experienced by the player when using a controller. Due to the immersiveness of VR, this motion sickness is much more common than with regular flat-screen games. As such, it's important that this application includes features that will lessen the impact of VR motion sickness.

Snap/Smooth turning

When using snap turning, users won't experience turning using their controller's right thumbstick as a continuous rotation, but instead as an instantaneous change in the player's perspective - as a result, players' brains won't be tricked into thinking they are actually turning. By limiting users to fixed turning angles (usually 45°), viewers will instead need to actually move their heads and bodies to make finer changes, which will prevent the disconnect between mind/body motion.

However, for many users who are used to virtual reality, snap turning is annoying and reduces immersion. For this reason, we'll use the industry standard of allowing both methods, with snap turning as the default. Unity makes it easy to swap between the two by including both components, which can be swapped in the options menu.

Smooth/Teleport locomotion

When using teleport movement, users won't be able to smoothly move around using their controller's left thumbstick - instead, pushing the thumbstick forward will show a curved line and reticle projected from their left hand, showing where they will teleport to. By releasing the thumbstick, they will teleport to the location that this reticle is pointing to, with a small period in between where the screen goes black to prevent the player from feeling disoriented by the sudden teleportation. As a result, players should experience less of a disconnect between their actual and perceived movement.

However, teleport movement can break immersion for experienced users, as a normal medical student does not have the ability to teleport. Also, because of the small period where the screen goes black, teleporting can feel annoyingly slow and clunky when trying to make precise nearby movements. Therefore, we'll make sure users can easily swap between the two modes by pressing on the thumbstick, allowing them to use the best of both worlds without needing to go into an unnecessary options menu.

Tunneling Vignette

A tunneling vignette applies a black or darkened visual tunnel around the user's view while they are moving, either through smooth turning, smooth locomotion, or full body movement and turning. By limiting their field of view, it reduces the perceived speed of their motion in-game - thus helping reduce the disconnect between their actual and perceived movement.

Unfortunately for user satisfaction, this feature can be very noticeable and annoying - it's more useful when traveling or turning at high speeds in more action-focused VR games. However, it's an easy feature to include - and while it won't be turned on by default, it will be an available option in the in-game settings.

3D Models

Interactive Features

In order to take full advantage of the capabilities of VR, we want this experience to be as interactive as possible. Users should be able to move and pose elements, snap labels into place on diagrams, and use rulers to measure exact distances on these diagrams, generally trying to match Meta's standard conventions for how these function in VR, if applicable. To do so, we will make use of Meta's interaction SDK components in Unity, as well as Unity's native VR components. Unity's native VR components are found in their XR interaction toolkit, while Meta has cross-compatible components in their Meta XR Interaction SDK Essentials. In general, Meta's components are slightly more feature-rich, especially in regards to hand-tracking, which is why they will be preferred over their XRIT counterparts when possible.

Distance Grabbing

When working with CT scans and other medical diagrams, we'll want the charts to be moveable and poseable, and not subject to normal object physics. For this, we'll want to use Meta's DistanceHandGrabInteractor and

DistanceHandGrabInteractable classes. The user can grab objects from a distance using the grip button of the given hand, grabbing the object pointed to with the hand's Ray Visual. We'll change the default values to make sure that the object does not automatically move towards the user's hands when selected, but instead maintains its position. Once the object is grabbed, the user can move it away or towards themselves by pressing the joystick of the grabbing hand forward and backwards, and rotate it by pressing the joystick left or right. The object will follow the Ray Visual projected from the hand, and will stay in its position once the user releases the trigger. If the user brings certain objects close enough to them, this grab will turn into a regular grab; if they drag certain objects to a socket, the object will "stick" to that socket as described below.

Hand Grab

For certain objects, such as the virtual ruler, when the object is either grabbed from up close or moved into the user's hands using distance grabbing, the object will "snap" into the user's hands as a held object. This will use Meta's GrabInteractable and GrabInteractor classes. As long as the user holds down the grip button of the grabbing hand, the item will continue to be grabbed - if the user pushes forward on the thumbstick of the grabbing hand, the object will return to a distance grab, and if the object is in range of a socket, the object will remain attached to the socket.

Sockets

Sockets are a base Unity component - certain objects will socket into certain other objects, which are socketable. Specifically, certain diagrams will have labels - these labels can be removed and then moved as a quiz for the

user - and if the labels are placed in a socket near the diagram, the label will stick in place and the label will change color to display whether the label is correct.

Buttons

Certain UI elements and the multiple choice quiz will take advantage of buttons. The user is capable of “selecting” the button by pointing their Ray Interactor at the button - while the ray is pointing at the button, the button will light up, and the user can “press” the button by using the trigger on the pointing controller. Alternatively, the user can press the button by moving their hand close to it - the hand will turn into a pointer finger, and if the button is fully pressed the option will be selected. This will use Unity’s `PokerInteractor` and `PokeInteractable` classes.

Navigating CT Scans

This application will make use of several CT scans. These consist of multiple cross-sectional images stacked together. Using VR, users need to be able to navigate between these cross-sections, as well as being able to drag through different parts of a given cross-sectional image.

Each CT scan axis will exist in its own viewport, which the user can move around and reposition using a standard distance grab. While pointing their ray interactor at the viewport and holding the trigger of the controller down, users will be able to switch between cross-sectional planes by pushing and pulling away perpendicularly from the viewport. They will also be able to traverse between slices by using the other axes - by targeting a specific

point on the coronal plane, for example, it will change the target slices for the sagittal and axial planes. Additionally, underneath each CT scan will be a slider, allowing the user to switch between slices for that given plane. Finally, users will be able to zoom in and out by using both controllers at once, moving both controllers apart to zoom out and moving them together to zoom in.

Alternatively, users will also be able to use hand navigation. They will be able to use the slider and cross-axis targeting for intuitive navigation. They will also be able to zoom in and out by pinching and moving their fingers farther apart or closer together. The multitude of options should make navigating the CT scans intuitive and easy, even for first time VR users, and should prime medical students to understand how to work with similar systems in other viewers, like 3d Slicer.

3D Renderings of isolated organs

This application will utilize 3D renderings of isolated organs, which will be provided for us by the UCF College of Medicine. These models will be interactive elements, serving as visual assets that will support the learning experience of first and second year medical students. Each model will be scaled and used to display segments relevant to various CT scans giving students an interactive and effective learning experience.

Unlike the CT scans these 3D models will provide an accurate representation of human anatomy which will be a great supplementary tool for understanding the CT imagery and the relationships between the photos

and the real models displaying a portion of the CT scan. This experience will allow the students to learn and understand the material better than just in a 2D learning environment and allows us to capitalize on the VR aspects of the project. This will create a focused environment for students to study each organ in a detached environment which will allow them to better see the complexities of the different systems in an isolated context. Any bone or tissue structures which would otherwise be a distraction from a point of interest would no longer be of any consideration. This will make the models much more clear and approachable for the students, leading to more room for learning and invoking a sense of curiosity which would enhance the overall experience.

Each organ provided will be an isolated object that pertains to a specific CT scan and will have a focus on isolating it from the human body to give the students an easier canvas to study and identify important features. This means that 3D models will retain anatomic accuracy and have a specific agency in context to the material. Any different anatomical landmarks within scans will be faithfully portrayed to ensure that students will learn from these models. If the focus of a problem was identifying a cadaver with a pacemaker, the 3D model provided would show the key components necessary to identify such as ventricular pathways or the pocket location of the pacemaker in context to the heart and how it connects to the cardiac tissue. The same way in which if the lung was being identified it would display the bronchial trees within the lung structure.

Model Performance and Optimization

Rendering the anatomical models within the Meta Quest 2 environment presents some challenges when it comes to providing smooth performance with fidelity. Particularly the issue arises in finding the balance between graphical fidelity that is essential for understanding the application while keeping a performance that feels responsive and satisfying. If this were not the case the application could feel sluggish, lazy or incompetent which would take away from the experience. This would create a very obtrusive application as the focus will always be distracted by the quality issues. With that being said the application needs to be able to retain a smooth framerate to reduce motion sickness. When working with not only models but the CT scans students need to be able to look at these models and derive a conclusion from what is given. If the renderings are not clearly represented the students may have a hard time relating it to the material or gaining any meaningful perspective from the model. However if the model's fidelity is excessively impressive it might have a significant effect on the performance of the application.

Beyond the scope of maintaining performance however another important consideration is the overall scaling of the model. These models must be scaled in a way to give them educational utility. This means that while they must be accurate the students have to be able to get a scale on the model to identify structures. This could mean making some models bigger or others smaller to maintain this. This means that models will be scaled to give the most clarity and provide sufficient information without overwhelming or confusing students with the scaling of our models.

The idea is to find a balance which will give us the best performance with models that are up to par with what is needed for the students to learn. To ensure smooth performance on the Meta Quest 2 and 3 the 3D renderings of isolated organs will be optimized for VR use, making sure that they don't take up too much space or harm runtime of the application. This could involve reduction of the polygon count where needed or reducing fidelity on models that don't require high detail textures. These choices will be made as needed to give the best visual experience that has an efficient runtime.

Optimization techniques:

- Polygon Reduction
 - As it sounds this would involve reducing the overall polygon count of the 3D renders if their effect on overall performance was significant enough
- Texture Compression
 - If the textures we are using are too high resolution for the needs of the application, texture compression could be a good option. This would involve downscaling the textures to lower resolution where we don't necessarily need significant graphical fidelity
- Culling and Occlusion
 - Create methods to only display and render the 3D models when we absolutely need them, making it so they don't

constantly have an effect on performance. This could involve only displaying sections of the model in which the player can currently see.

With these methods our optimization process will prioritize keeping the features that are educationally relevant if significant runtime issues are prevalent. This means that important structures within the models will keep the highest fidelity while surface level meshes that don't need to be identified can be downscaled. This will keep models visually pertinent without sacrificing key components needed for students to identify structures within the 3D renderings of isolated organs.

To ensure the quality of our application we will make sure to test different features of our application iteratively as the size increases. This will make sure that we keep the quality needs of the application as it expands. This would mean play testing the application to get a baseline on relevant features such as loading times, framerate, and graphical quality. This will not only serve to make sure our application is functional but give us context for how we need to handle the models and features we implement further down the line into this application's production.

Anatomical plane

The anatomical plane is an important concept in medical education which gives a framework to identify relationships between different sections of the human anatomy. These are essentially different planes representing cuts between body sections, essentially cutting them into areas with their

respective axes. Anatomical planes are an important descriptive tool which is why we have decided to implement it within our application. This system will not only serve as a visual guide for students but further serve as an interactive tool for manipulating the models. Especially in the context of our application being a CT VR application, these planes will serve a significant purpose in connecting the analysis between our 3D renderings and the CT scans that will be provided to students within our application.

To help give students a clear understanding of the orientation of 3D renderings of isolated organs, the VR application will be created with a system for visualizing the anatomical plane system in mind. The anatomical plane system is a division of the human anatomy which separates the orientation of the human body. This is a foundational principle for understanding the orientation of the human body and the relationships between the different structures within it. Not only this but it is a standardized concept in medical education. This makes it a great feature to implement within our application, as it will allow them to connect their studies to actual 3D interpretations. This is essential for providing a consistent output for describing the location of different body structures. It also gives us a good context of a standardized position which will make it easier to display different organs. Along with that the anatomical plane serves as a great way to introduce a variety of tools and guides for students to manipulate models. The anatomical plane system will be split into these three sections.

These three sections include:

- Sagittal Plane

- Division of the body into left and right, this is an important section which divides the brains sections and the lungs
- Coronal Plane
 - Division of the body into anterior and posterior (front and back), this is important for understanding the location of organs such as the heart.
- Transverse Plane
 - Division of the body into superior and inferior (top and bottom), this section corresponds to the imaging that you might see within an MRI scan.

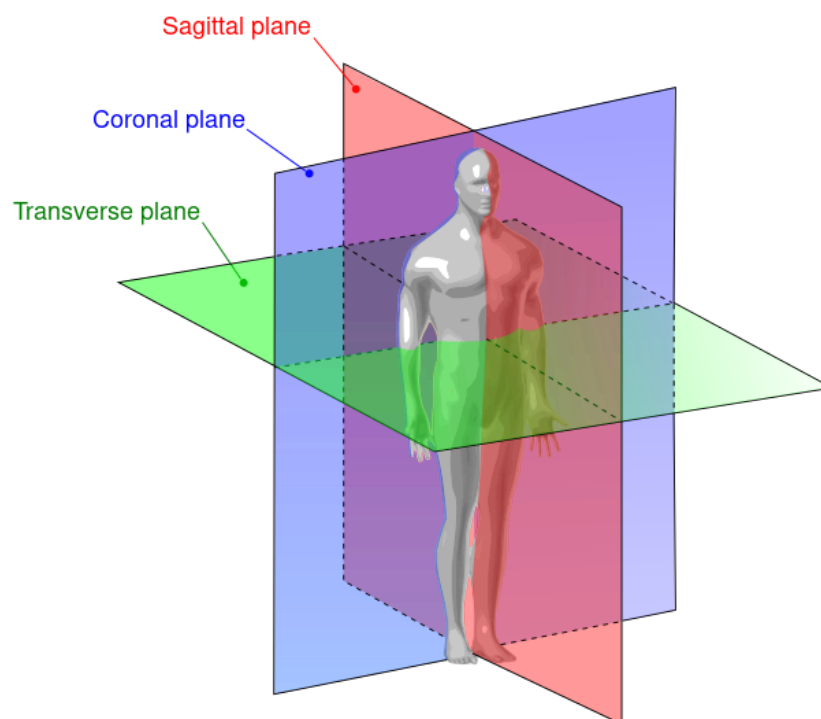


Figure 14: Direction of the 3 planes from LibreTexts, shared under a CC BY-SA License

Inside of our VR application these will be displayed as semi transparent surfaces which section the different models based on their respective axes. In this implementation the user should be able to toggle different planes for clarity or convenience and will essentially give them control over the visual aid during their time on the application. This is not only important for the perspective as mentioned earlier but also gives a quality of life feature which will be great for the first and second year medical students.

These planes will also function as interactive elements for the students to manipulate. With a point of reference as the anatomical planes, students will be able to rotate models or adjust their positions as they see fit. Having these planes gives a point of reference and allows the students to manipulate the models while keeping an understanding into the orientation of the isolated organ.

For a more specific example if a student was manipulating an 3D model of an isolated organ such as the liver, they would be able to see it from the anterior to the posterior such as you would with a CT scan however with a much clearer model. The integration of these anatomical planes serves to directly provide assistance to the students which will not only give them significant conveniences but also keep a more consistent environment for viewing isolated organs. These Anatomical planes give relevant perspective to the CT scans and the physical anatomy of the 3D renderings of isolated organs. These will serve as toggleable guides that overlay the models, this will help the students see how the models fit within the body.

Cross-Sectional Slicing

Cross-sectional slicing will be a significant feature within this VR application, offering first and second year medical students the opportunity to explore the human anatomy from within as they see fit. While the 3D models by themselves offer a great view of the surface of organ structures, they don't display the internal structures of the models such as a CT scan would. This could be an important feature when visualizing human structures with complex internal systems such as the brain where a visual on structures further than surface level is essential for getting the full picture. These features are almost impossible to fully appreciate and make use of from just the outside of a 3D render. Our integration of cross-sectional slicing aims to combat this issue by simulating the creation of cross sections on our 3D models such as you might see with CT scans, where physicians use these various slices of human anatomy to identify and study potential issues.

In our VR application the user will be able to create cross sectional slices by "cutting through" the different 3D anatomical models using intractable planes. These planes will serve the purpose of not only being means of connecting the 3D models to the structures of the human anatomy such as described in the previous anatomical plane section but serve as a way for students to cut cross sections in the different models. This illusion of slicing the 3D models can be achieved in a variety of ways however the method we have decided to take on is to use clipping planes. As the student will move the plane throughout the 3D model it will create the illusion of slicing the model while it will just be hiding the surfaces on the other side of the plane essentially creating the slices visually. This method is not only incredibly simple but is a great way to show perspective and represent the layers within the human body. This is because as the students will move the

sliders throughout the 3D model they are slicing it will mimic the progression that is seen in CT scans. This is a great way to connect it to the other material as it creates a bridge between the two mediums and is quite literally an exact representation of the process of CT scans. This is great for how students will perceive the body in a more interconnected way and reinforce their understanding of how layers are connected.

What makes this such a good system is the control the students will have over the implementation. Students will be able to manipulate different settings while moving the slicing planes such as position and depth. Not only that but utilizing the aforementioned anatomical planes they will be able to slice the models in the orientation sagittal, coronal, and transverse planes directions. Being able to drag these planes to reveal cross sections will give them a great way to see in real time how the anatomy is revealed. This is great for the immersion of the student and will help them actively build a better perspective on the context of the model layer by layer. From an educational standpoint this is a vital tool for giving the students agency in how they approach viewing these models.

Moreover, this will connect how they understand the alignment of the 3D models to the CT scans. This is a 3D imaging VR application for first and second year medical students which makes the educational value provided the forefront of this application. This feature will capitalize on the use of connecting the real world CT scans provided to the 3D models portraying accurate scans of the isolated organs located within those CT scans to give the students the ability to connect one to another. Introducing the ability to slice cross sections will help them develop an understanding better than static images of CT scans could.

Functionality

The cross sectional slicing will be implemented through interactive planes in which the student can move in their desired direction to reveal internal structure of our isolated organ structures. This will be done in Unity through the use of 3D planes which will be manipulated by the students. The main idea of it is to create a plane with a custom shader, most likely one that makes it mostly transparent. This will then be moved through the model obscuring the surfaces in which it clips creating the illusion of creating slices within the model. This could also be done by removing geometry on the other side of the plane. These planes will be toggleable and interactable, giving students the option to pick the orientation of the planes based on the areas of the isolated organ they want to display. This will also allow students to slice the models based on the plane in which they want to view it such as the sagittal, coronal or transverse planes giving students more options for their cross sectional analysis.

The cross sectional slicing method will be designed to be intuitive and not abrasive to the experience. This will be an option that students will be able to enable or disable as it may not always be necessary to see the models in that context. The idea is to have the model represented with the anatomical planes and if the students feels as though cross sectional slicing is needed for further understanding they can enable it and utilize the planes to create cross sections within the model.

We want this to be a nonintrusive feature so when it is toggled off the clean model with the anatomical plane will be displayed and with the option

toggled on the ability to manipulate the model along with the slicing planes will be visible. This will also give more options and variety to how students want to answer the problems provided to them. Some students could look at the CT scans and the isolated 3D model given the context, while others could slice the model to get a better understanding of how it related to the CT scan by slicing and showing the model layer by layer.

To effectively enable the ability to create cross sections, the models will of course have to have content inside and thus be volumetric. This will be achieved via the models provided to us from UCF Medical Department. Some of these models will be derived from the use of plugging real CT scans into the Slicer 3D application and gaining volumetric models of the scans for our use. This will create high resolution models which can then be sliced to accurately display the model and the slices as they would be within the CT scans. This is due to them being derived from the CT scans themselves thus giving us incredible accurate models to manipulate.

This implementation will give these features:

- Create cross Sections
 - Users will be able to move slicing planes along their axes to reveal cuts within the models
- Multi-Plane Slice Manipulation

- Users will be able to move the sagittal, coronal and transverse Anatomical planes to view the model from different perspective
- CT scan Comparison
 - Allow the user to see the context of the model within the relevant CT scan.

The ability to create cross sections within the 3D models provides great educational value to the medical students and creates a more interactive and layered environment to the experience. This also will give more freedom and agency to the students and how they study the material with them controlling their interaction with the models. Not only that but the creation of cross sectional areas is a core tool in creating a relationship between the CT scans and the 3D models of isolated organs. Allowing the students to create slices with multiple planes will give more flexibility into the viewing experience and make it more compatible with different CT scans. As CT scans can be done on different anatomical planes, being able to slice with a different plane or perspective is imperative to getting the best result from the feature. CT scan comparison will show the current slice that the student has the model on visualized in a 2D format on the screen. This is another great method for connecting the context of the CT scans to the 3D model as it will show the exact position of the slice which can be directly compared to a CT scan giving more clarity.

User interaction with anatomical models

Each 3D model will support user interaction through Meta's SDK, this will allow users to rotate, scale, and position different renderings for an enhanced learning experience. This implementation will also support the rulers, labels, and annotations which will be directly implemented into the interface allowing students to further study the models.

3D Slicer

In our Clinical Imaging VR application which aims to provide a more interactive platform for understanding material aimed at first and second year medical students, accurate and engaging 3D models and CT scans are very vital for the learning experience. A key component of this is transforming the raw files in which we are provided, such as the CT scans, into interactive volumetric models in which students can study and manipulate within the VR environment. These models need to be accurate and provide information with sufficient fidelity, to achieve this we will be using 3D slicer which is an open source application for medical analysis and visualization of medical imaging.

We chose this platform because:

- It is open source and free to use
- It supports a wide variety of medical imaging formats
- It includes tools specifically made for segmentation, volume rendering, and modeling
- It has been widely used in both academic and clinical research

3D Slicer is an application aimed at reconstructing advanced image computing challenges with a focus on more specific uses such as clinical or biomedical applications. It was originally created by teams at MIT and is a great tool for transforming medical data into more usable formats. Such as rendering CT scans with volume and segmenting different models. For our specific project 3D slicer will provide us with a path to take the medical CT scans the University of Central Florida College of Medicine provides us and turn them into 3D models that can be used by the students within our application.

Key features we will take advantage of include:

- Volume rendering of CT data
- Model segmentation
- File format compatibility with Unity (e.g. OBJ, STL, NRRD)
- Cross-sectional and 3D viewing modes
- Reducing Polygon count

The files containing CT scans can come in a variety of formats such as dcm or ima files but for the purpose of this project we have currently only received them in the IMA format. From this we take the folder of a CT scan containing what is usually a few hundred images making up the CT scan and plug it into a 3D slicer. From this there are a variety of ways we can derive a volumetric model of the CT scan depending on what we need.

What makes 3D Slicer so important to our project is the fidelity and accuracy of the CT scans. Models derived from 3D Slicer will retain the anatomical accuracy provided within the CT scans while also providing many ways for us to manipulate the models depending on what we need to display within the project. This could be through segmentation which would allow us

to isolate a specific part of the model from the rest, for example if we had a defect in the lungs of a CT scan we would isolate that specific section from the rest of the model and display it. Allow more accurate understanding and getting rid of unnecessary parts which could distract from what we want the user to obtain from the models provided. There are also many tools for cross sectional viewing which make it great for seeing relationships between internal structures.

Some key capabilities of 3D Slicer that benefit our workflow:

- High-fidelity anatomical rendering
- Segment isolation and defect highlighting
- Ability to remove irrelevant data to focus on educational content
- Detailed inspection using cross-sectional views

The Segmentation Process

One of the most important uses in slicer for this project is the ability to segment different sections of the models. This was briefly discussed above however the applications of this are important enough for its own section. In the context of our project segments will be special sections derived from different CT scan models that will provide a specific context to a problem. Such as identifying the difference between a normal and abnormal CT scan. Due to this we will have to use segmentation in 3D Slicer to separate these specific regions of interest from the surrounding tissue. This will be done within the Segment Editor, this editor can be fairly complicated but provides us with many different tools for getting the models that we need.

Within the Segment Editor:

- Segments can be manually drawn on each image slice
- Semi-automated and fully automated segmentation tools can speed up the process
- Tools include smoothing, coloring, fill-between-slices, and 3D rendering
- Manual refinement is possible for more accurate results

Within the Segment Editor each segment can be created manually through a fairly time consuming however effective method of going through each slicer and drawing a boundary of the section you want to segment. There are also some semi and fully automatic methods that can be more or less effective. Inside the segment editor we can also manipulate the segment in many different visual ways such as smoothing the model, coloring it, and level tracing to display the 3D model. For the most part a majority of these features are for very specific purposes and will be on a case by case basis depending on the CT scan we are using. The most important ones will probably be using the fill between slices to render and painting the segment to make it easier to understand.

In our VR application when segmenting parts of a model the quality and ability to understand what we are cutting out always needs to be considered. Especially with the segmented models we expect the students to go over these with more scrutiny as they are going to be identifying things on these more specific portions. Due to this we will have to make sure there aren't any gaps or distortions that could make these models less than accurate. We will have certain quality expectations for each segment and we will make sure to review each we use to make sure every model is up to the quality needed for students.

Quality assurance checklist for each segment:

- Visually inspect for holes or jagged areas
- Confirm boundaries match the intended anatomy
- Remove or refine ambiguous data
- Compare against source slices to verify accuracy

Once we have successfully segmented an organ we will use the built in features of slicer to export it with whatever file format we need it in. This could be displaying it as a nrrd file or an obj file. Currently we are still working on figuring out the best format for displaying the models. Many different methods work when exporting these models to unity so it will eventually come down to which method provides us with the best performance and is the most convenient to implement.

Preparing models for VR

Once the 3D model has been made within slicer there are some considerations before putting the model straight into unity. As previously mentioned the file format and size are a pretty big consideration which is still at our discretion. When making the 3D models we are using what is more often than not a few hundred image files containing segments of scans to create one 3D model. These 3D models can have very large file sizes which makes the concept of optimizing performance an absolute priority when using these models. These volumetric models often have hundreds of thousands of polygons which would not only reduce performance but from some personal testing from our team even induce slight discomfort or motion sickness in some users.

VR model optimization goals:

- Reduce file size where possible
- Simplify polygon counts while preserving accuracy
- Ensure smooth performance on VR headsets
- Minimize motion artifacts or discomfort for users

To address this issue whenever needed we will introduce simplifying the models into our production pipeline for the models. This of course will be a case by case basis as some will undeniably be bigger than others, but the idea is to simplify applicable models before putting them into unity. This can be done through a variety of applications however the most convenient and probably the best would be through Blender. This would allow us to reduce the polygon counts of the models while retaining the accuracy that we need within our models. Putting these into blender would give us a lot of adjacency over the size of the models and allow us to use some aforementioned optimization techniques such as Polygon decimation, smoothing of the surfaces and cleaning the meshes.

Optimization methods using Blender:

- Polygon decimation (reduce mesh complexity)
- Smoothing tools to clean jagged geometry
- Mesh cleanup and repair tools
- Export in optimized file formats

Once the models we will be using are done being refined and are ready to be put into Unity we will make sure to give them consistent naming conventions based on the context of the model. Each model will come with a text file describing the location and implementation of the model. This is important due to the fact that we will be dealing with a variety of models

which will all serve a specific purpose. Correctly naming all of these files and setting each with their own text file will make sure everything is being properly utilized and avoid confusion. Additionally after exporting each model from blender they will be stress tested within our Unity application using the VR to not only ensure framerate but also the quality of the model.

Labeling and Annotation within 3D Slicer

Another component of 3D Slicer is the ability to label and annotate different structures within the CT scans and 3D models. While the 3D models are a great resource we are working with first and second year medical students so in some cases labeling different parts of the models can be important for clarity. Clear labeling of different sections will leave essential visual cues that will help identify and differentiate different regions of the 3D models.

- Labels help identify individual anatomical regions
- Annotations improve clarity for early-stage learners
- Metadata integration allows easier referencing in Unity or VR

Through 3D Slicer we are able to integrate the metadata directly into the models we are exporting. This would let us take different segments of the models and identify each by a specific name, color, or their own boundaries. This is not only important for clarity with the students but also for our group. If we are using these models as seven computer science majors instead of Medical students it is nice to have our data neatly sorted and labeled to avoid confusion.

- Each segment can be:
 - Named: Each can be given a specific name which will help with organization and clarity
 - Color-coded: Different colors can be applied to regions making each distinct
 - Bounded: Clear boundaries for visual separation and reduce the ambiguity between segments
- Helps both student learners and the development team
- Reduces the chance of errors during model use or revision

These labels however can be also implemented in Unity as well and will eventually play a part in our implementation of measuring and identifying certain parts depending on the questions pertaining to the model.

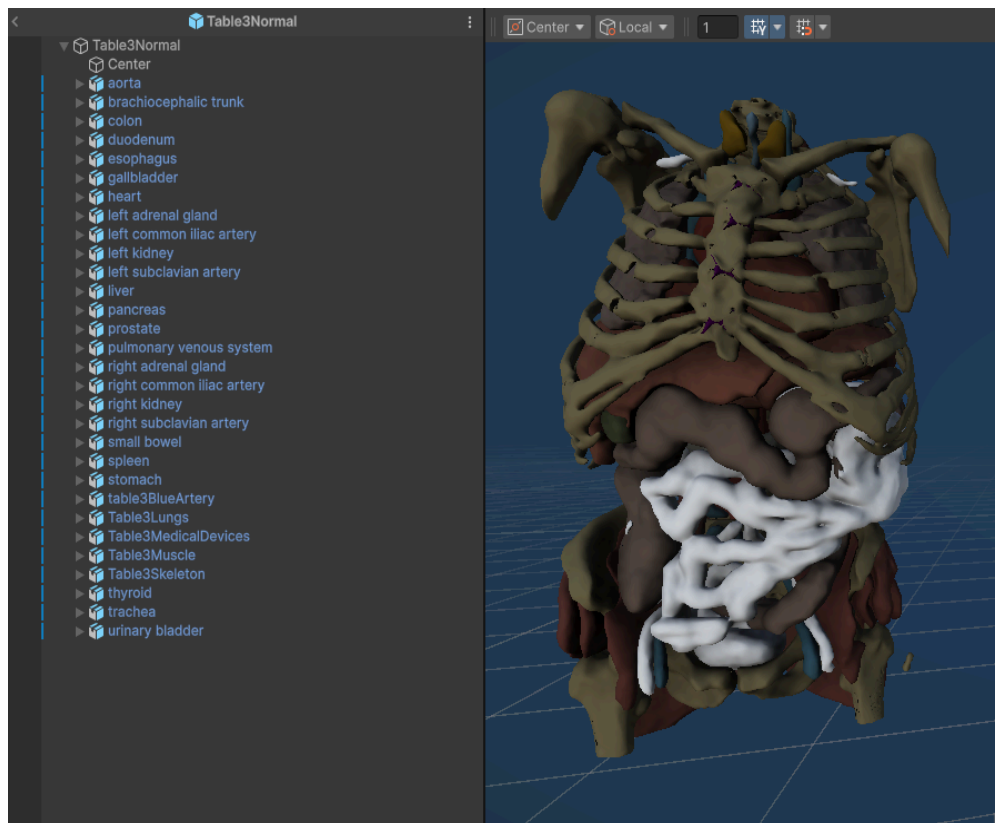
- Label use in Unity may allow:
 - Real-time part identification
 - Interactive feedback for learning
 - Measuring tools within the application

So if we use 3D Slicer or Unity for labeling will depend on the data we are given later however it is a very useful tool that we will most likely use. Especially due to the ease of using built in metadata to provide information for not only the students but for the people working on this project.

3D Models in Unity

Setting up the Model

As stated before and can't be stated enough, there were impossible limitations with rendering in a VR environment. One of the major ways this may be presented is how our models are setup in Unity. Because each of our isolated organs and organ systems need a different color and polygon count to ensure they were represented the best they could be, they each needed to be a separate obj file. This lead us to using this design for our model prefab:



Model Prefab in Unity

This represents how all of our models are modulated, with each separate body part holding a Mesh Filter and a Mesh Renderer inside their respective grp1 child gameobject. The parent gameobject (Table3Normal here) nicely ties all of our .obj files together, and allows us to position and scale it by simply changing the parent gameobject. There are some models (refer to model 20) that needed drastic position changes, we don't know why some models were positioned incorrectly in 3D slicer, but if this approach is followed by a team after us, just know to change the Y positioning manually and it should be fixed. On top of positioning and scaling, the parent gameobject has 3 main components:

- Rigid Body
- XR Grab Interactable
- Box Collider

RigidBody applies the physics we want to our model, the main changes we made to this component were: removing gravity, enabling "is kinematic" to make all of the movement and collisions for the gameobject done through scripts, and lowering the linear and angular damping so the model won't fly off when let go. Some changes can be made to allow appropriate physics such as changes to the colliders so the model won't go through the walls.

XR Grab Interactable allows the user to grab the model, we added the appropriate colliders and changed to dynamic attach, which simply means the user can grab it to where they selected it from. This makes the rotations based off where the user is holding it.

Box Collider is the collider for the model where the user can interact with the model. We wanted to use a mesh collider for better accuracy of the actual model, but because of how the hierarchy is, making the model a mesh collider would have nearly 20 different colliders (for each separate organ) all

interacting on eachother making them fly off into space. The box collider was the easy and simple approach but had its conflicts. The player can move the model when grabbing the air where the box isnt on the model, and our ruler is slightly inaccurate because it measures a box, and not the model itself.

Displaying the Model

We have a normal and Abnormal CT Menu, this is for the player to compare and contrast the differences of a normal CT scan and an abnormal one. Initially, we were not provided with any normal CT scans, so all normal references are based on Table 15, which is a patient with diverticulosis and no medically implanted devices. While this still isnt technically “normal” its the only model we received without a medical device which is the main thing we want to highlight in our CT scans. Our project was originally designed around having two models in the scene, but we switched to a single model about a month before the final APK released. The normal scan still retains the model view and a few unused scripts, left intentionally for potential future adaptation. All newly created models are part of the abnormal side.

Our models start in the CT viewer once a patient is selected, and can be popped out via the pop out button.

Renders Container

This is what holds our model, we have 3 different game objects here:

- Render Interaction
- Planes
- 3DModelCamera

The Render Interaction collider provides what the user will grab to rotate and scale the model when in its popped out view. Most our scripts will ignore this collider, and is disabled when the detach button is clicked. This is intentionally bigger than the model itself, so that its impossible to grab and move the model even though its technically possible with XRGrabInteractable.

Planes holds our planes to be toggled on and off via our popout UI.

Each plane has its respective manager that does the following:

- Listens for the CT slider movement
- Sets its move direction based on its axial view
- Sets the number of slices its passing through

If we wanted a more detailed scan, we can increase the number of CT pictures we have and update here to the correct number of slices. We found 120 slices to be the most optimal.

3DModelCamera is what views our model in our CT viewer, this is rendered in our scene but outside of our environment, its specified to only view the model and can capture nothing else. It is disabled when the model is in its popped out view.

3D Model Scripts

ModelManager

After our prefab is created and in our Assets->Resources->Renders->Prefabs folder, we introduce it to our ModelManager gameobject. This is our script that holds each of our prefabs to be spawned dynamically in our environment. The models HAVE TO BE

SPAWNED DYNAMICALLY for rendering purposes. We found having even two models in the scene would reduce framerate substantially. ModelManager also allows us to set the position of the spawned model, and this can be done in Unity's inspector.

In the ModelManager.cs script itself, when the game is run, *Start()* is run. This makes sure a spawn target is assigned, calculates a modified spawn position, instantiates ALL models at startup, chooses a default model (we did model 16 which is an empty prefab to show nothing at startup) and records the original spawn transform for *ToggleTeleport()*.

The *ToggleTeleport()* function is used to be called for our Pop Out button and teleports the model from outside the environment where our camera is setup to show it in the CT viewer into the room itself. It saves the original position and the previous position to be called back and forth. The button that calls this functions also calls our ToggleModelLayer.cs script to change the layers from UI3DModel -> default, to swap between allowing the camera to view the model and allowing the user to see the model. This is for performance reasons, so when the model is in camera view the player wont have to render it physically in the scene.

ChooseModel(), *PreviousModel()*, *NextModel()*, and *ResetModel()* are relatively easy to grasp, these functions are made to be called for *onclick()* events for our UI buttons to do as they say.

For the majority of our scripts interacting with our models, we will call ModelManager and use *ModelManager.CurrentModel*. This holds the prefab of the current model, which can then be used to find any positioning, sizing, or rotation thats on the selected model.

VRModelController

This script lets the player rotate and scale a 3D model using the joystick on their controller while the model is being grabbed/selected. We have two scripts for the movement, one while the model is inside of the CT viewer, and one when it is popped out (VRModelControllerPhysical). It listens for select/grab events using XRSimpleInteractable for the CT viewer and XRGrabInteractable for the popped out model. When the object is selected, it reads joystick input:

- Left/Right movement rotates the model around the world Y-axis.
- Up/Down movement scales the model within a clamped size range.

It references the ModelManager to always modify the currently active model. It only runs these controls while the object is actively selected.

When the model is popped out, we also set a listener to read any scaling and rotation changes to our planes script so that they can change based on where the model is set.

CTPlaneController

The CTPlaneController manages the movement, orientation, and scaling of a slicing plane used to navigate CT scan data inside the 3D viewer. Its primary function is to synchronize the CT sliders with the physical position of a slice plane so users can scrub through axial, sagittal, or coronal slices of the currently loaded CT model.

There is a lot to say about this script, if there is confusion on how the script works we recommend plugging it into ChatGPT and viewing the flow diagram it generates. The sequence of events looks like this:


```

Participants:
User, UI Slider, CTPlaneController, ModelManager, Slice Plane (sliceObject)

-----

User          -> UI Slider          : Adjusts slider value
UI Slider     -> CTPlaneController : onValueChanged(newValue)
CTPlaneController -> CTPlaneController : UpdateSlicePosition(newValue)
CTPlaneController -> Slice Plane      : Move plane along moveDirection
Slice Plane   -> User                : Updated slice visualization

-----

ModelManager -> CTPlaneController : CurrentModel changed (ResetSlider triggered)
CTPlaneController -> ModelManager  : Request CurrentModel reference
ModelManager -> CTPlaneController : Provide new model's root object

CTPlaneController -> CTPlaneController : UpdatePlaneToCurrentModel()
CTPlaneController -> Model (Renderers) : Read renderer bounds
Model (Renderers) -> CTPlaneController : Return combined bounds

CTPlaneController -> CTPlaneController :
    • Calculate new modelPhysicalSize
    • Set plane rotation
    • Adjust plane scale
    • Recenter initialPosition

CTPlaneController -> UI Slider          : Set slider to midpoint
UI Slider          -> CTPlaneController : onValueChanged(midpoint)
CTPlaneController -> Slice Plane        : Move plane to center slice

-----

User          -> UI Slider          : Continues slicing updates
(Loop repeats)

```

CTPlaneController Sequence Diagram

DistanceMeasure

This script implements the 3D ruler that lets the user point at the model and measure distances between two points using their controller's trigger. It also displays a reticle, draws a measurement line, and shows the computed distance in millimeters. It works something like this:

When clicked, hovering over the model shows a reticle, when the user holds A and drags their ray cast line, a ruler with a measurement will be created.

When another line is being made, destroys the last one.

When the ruler button is turned off, destroys all rulers.

Problems:

Colliders for the renders are very poor, they are all simple box colliders which makes going in depth with measurements very difficult, and many lines will appear floating. This can be fixed with a Mesh collider on the renders, but would add a layer of complexity that the project doesn't need. Furthermore the implications of this strengthen the core concepts of retaining a build that is conceptually simple to build off of and improve, while also keeping the general feel and implementation practices that best suit our team.

Other Scripts

- *ModelTextChanger* changes the text above the CT scans to the model being shown.
- *PlaneToggle*, *ToggleActive*, *ToggleOnOff*, *ToggleUI* toggles their respective gameobjects from a button in our popout UI.
- *ModelSpawner* is used in the CT viewer to set its layer and position for camera view

3D Model UI Buttons

PopOut Button

OnClick():

[ModelManager.ToggleTeleport]

[ToggleModelLayer.ToggleLayer]

Pops out the model and swaps its layer to be rendered by either the Main Camera (player camera) or 3DModelCamera.

Detach Button

Swaps between two modes:

1. Rotation and Scale
2. Rotation and Moveable

Each of these is a onclick() event in our detach button itself.

1.

- Reset the model [ModelManager.ResetModel()]
- Disable Planes [PlanesManager.DisableAll()]
- Switch Model Layer to UI3DModel [ModelLayerManager.ToggleLayer()]
- Enable Scale and Rotate Collider [Interact.ToggleObjectActiveState()]
- Enable Planes Buttons [PLanesButtonsManager.ToggleTarget()]

2.

- Reset the model [ModelManager.ResetModel()]
- Disable Planes [PlanesManager.DisableAll()]
- Switch Model Layer to Default [ModelLayerManager.ToggleLayer()]
- Disable Scale and Rotate Collider [Interact.ToggleObjectActiveState()]
- Disable Planes Buttons [PLanesButtonsManager.ToggleTarget()]

Current problem:

To see the problem yourself: pop any model out of the screen, swap to another model, place this model back in the screen, and finally swap to the previous model: it will be invisible to everything. This happens despite the model being in the correct layer and in the correct position. We have no idea how to fix this, as the player, clicking popout model button will fix issue.

A problem with moving the model is that we cant show our planes on it.

The whole idea is that the planes shouldn't be enabled when the model is being moved, which makes moving the model obsolete in our opinion when the whole idea is to see where the CT view is located in the model. The reasoning behind this decision was that it required immense rendering, requiring a constant Update() to be ran every time the model is moved or rotated, and having the planes adjust their size by running complex mathematical equations to find the new bounds, position, and rotation of the model at every tick is bad for the framerate. We decided in our last week of this project NOT TO TOUCH THE CTPLANECONTROLLER SCRIPT. Every time a single line was changed, the apk wouldn't run.

PopOut Button

OnClick():

```
[DistanceMeasure.ToggleRuler]
```

Toggles the ruler to shown on the model. This is all handled by our right controller which holds our DistanceMeasure script. More details are located in the DistanceMeasure script description above.

CT Color Button

OnClick:

```
[MultiSliceViewer.ToggleAlternate]
```

Changes the CT scans to a colored version to highlight individual body parts.

Patient Buttons (Transfer Functions)

OnClick():

[ModelManager.ChooseModel] (Normal)
[MultiSliceViewer.SetParentFolder] (Normal)
[ModelManager.ChooseModel] (Abnormal)
[MultiSliceViewer.SetParentFolder] (Abnormal)

ChooseModel changes the model prefab we are currently viewing.

SetParentFolder changes the CTslices we reference on the CT viewer screen. We intentionally left the Normal side included even though they are all set the same, we were originally were hoping for more normal scans, but were left with only one. When more data comes in, change these values. For each button on the same model, we call a different prefab that showcases their respective organs or skeletal structures.

Overall having multiple prefabs for the same model is highly unoptimized. In hindsight, having one model shown and disabling the respective body parts would be much simpler.

GridPlanes Buttons

OnClick():

[PlanesManager.PlaneToggle~]

This activates the planes on the 3D model. When the model is in detached view the planes buttons will be disabled. (Refer to detach button for explanation)

CT Viewer

Being a Clinical imaging CT VR application a very important consideration for us was the overall layout and usability of our CT viewer itself. Students using this tool need to be able to compare and contrast normal and abnormal CT scans with complete clarity and full context while also having to do so in a VR environment rather than how they are traditionally in a point and click monitor space. This created an initial design problem of how we can present a large amount of medical information across different CT sequences and perspectives without making this application overwhelming for the first and second year medical students which will be using this.

Before getting into this however we had to solve the issue of how we would get our medical data, when we receive our medical data it typically was in the format of DICOM files. These files do not contain the actual CT images in a format that can be put directly into unity but are rather sections of metadata which describes the physical dimensions, density and all relevant information that is obtained when a CT scan is taken. Because these are structured for medical use and critical analysis rather than for a unity game, we will have to first rely on 3D slicer to process our data.

When first entering 3D slicer we can import in our DICOM dataset, once this is done 3D slicer will automatically organize the CT information to create a volumetric model and create the relative CT sequences. These can be seen in three different views which are as follows

- Axial view (red section) – horizontal slices from top to bottom

- Coronal view (green section) – vertical slices from front to back
- Sagittal view (yellow section) – vertical slices from left to right

It is important that we use each of these views when creating our custom CT sequences as this structure follows the formatting in which these students have been taught so capturing these image sequences is imperative to the design of our application

Slicers allows us to scroll through each sequence and view the CT scans in the model, however it doesn't provide a direct way to extract these images. 3D slicer does however have a built-in Python scripting element which allows us to create a script which can automatically capture all the CT images we might need. Our workflow for obtaining these images involves running a custom Python script which iterates through each of the anatomical views and captures a user defined number of slices in a user defined resolution. Before this is run however the users must define the slice range from which we will capture our images. This step is necessary to be done manually simply because CT scans can contain starting or ending sequences of purely black images as it might capture empty space before getting to the sections of the body. So for example the Axial view might start right off with body captures whereas the coronal view might take 10 frames before any actual relevant information is shown. We do not want to capture these blank frames and as such have to define parameters in which to capture our information based on each axis.

To avoid this problem the user will have to manually scroll through each axis and record the starting and ending positions in the millimeter range displayed on the slicer sliders. This will give us the data for where the

anatomical data starts and finishes which we can input into the script for each view. From here our script will capture 120 images within the range provided for each axis, Axial, Coronal, and Sagittal and saves them into corresponding folders named Red, Green, and Yellow.

The resulting images will be clean and uniform allowing a seamless transition between images and giving clarity and consistency to our sequences. These will give CT images in three different directions which we can then use to construct our CT viewer.

CT Viewer Structure

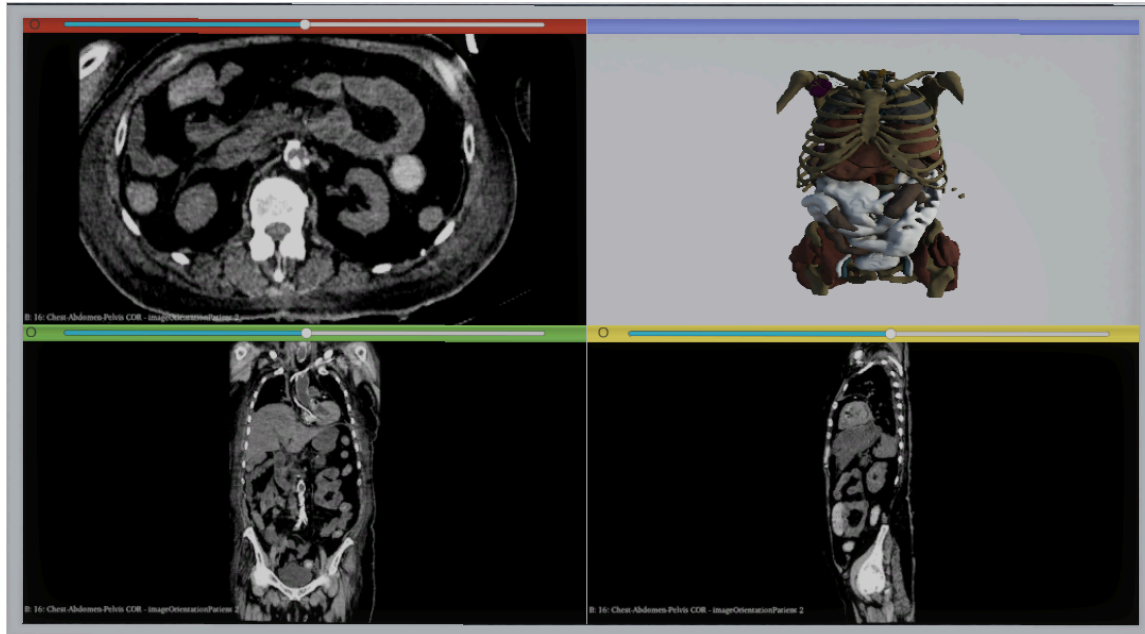
The current structure for our CT viewer in unity is built around a UI canvas that contains all the relevant information pertaining to a model's main views in CT sequences. These show the different perspectives that are obtained when a CT scan is taken of a patient and can be represented in the form of the three standard anatomical axes that are normally used in medical imaging. In our CT viewer we have three quadrants as these views while the top right area is saved for a 3D model view that represents the patient being displayed in the CT sequences. This layout is meant to be familiar and follow a general workflow that is visually appealing while also maintaining ease of use. For this we decided to make the system similar to that of the viewer in CT slicer. This will help students who have already used 3D slicer or other medical applications with a similar structure to orient themselves in our application without the hassle of learning a new system or using one that is inconvenient. Being that students are meant to understand a specific dataset and tie everything together to gain some perspective on not only the model but how the CT scans pertain to what you are looking at, it was imperative that it was structured as such.

From the three different slice views provided students can move through the different CT sequences using the sliders that are provided at the top of each section. The CT viewer was also programmed to use the sliders when students tried to interact with the slider from the image itself to provide more ease of use as the hitbox of the slides without that are much too small. When using a VR headset this was a problem we discovered during development so just a little quality of life feature to make everything easier to use. When moving through a sequence students can get a good feel for the transitions of the body as the slider moves through the CT sequence in the 120 images provided in the slider. This number was chosen specifically due to it providing a smooth transition throughout the sequence while also not taking up too much of our applications space. Normally these sequences could be anywhere from 300+ images which is just unnecessary for what we need. This value could be adjusted in future versions of students or the sponsor feels otherwise.

Students have the option to expand any of the slice views into a fullscreen mode. This will take up the space of the entire ct viewer with just one ct sequence which students can interact with similarly to how they could before. Fullscreen mode allows students to see the image with a bigger perspective which is great for seeing the smaller details. The relative use of this is case by case with some images giving enough context in the regular window without the need for a fullscreen. However, having the option provides a bigger view for CTs which have smaller structures and more subtle features. This is necessary because not all image sequences are made the same due to CT scans coming from a variety of different machines taking it which makes some appear bigger or smaller.

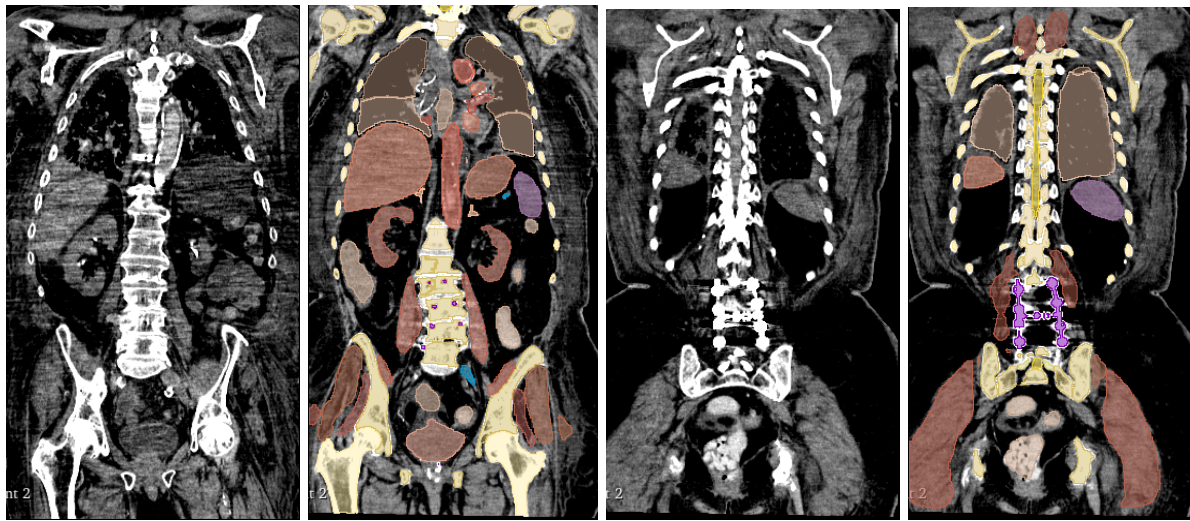
From this fullscreen view students can also measure the CT scans. Due to the fact that CT scans can differ in the physical dimensions which are being captured, we cannot gain relative scale from a pixel to millimeter ratio. To solve this problem we extract the metadata provided in 3D slicer for the relative size of the CT images, this data is very precise with a small margin of error for correct size. This ensures that the measurements students take will be accurate and will correspond with any measurements taken of the model. When a student enters fullscreen it will reference the model being viewed and the color sequence that was selected and set a relative scale for length and height to ensure its accuracy each time as each CT will be different even within the same model.

The relationship our application provides between the models and the 2D ct slice views adds a very important layer of understanding. It is essentially that the students can reference the CT scans and the model in a way that ties everything together. To do this when using the clipping planes provided in the 3D space, it will move the corresponding slice view to the same sequence that is being referenced in the model. This will allow students to either move on the 3D space or move the plane in the 3D space and display on the opposite exactly where you are looking. This real linkage helps students understand how these 2D slices correspond to the actual 3D structure of the body. This combination of all the features in our CT viewer makes our CT viewer an effective learning tool and will be great for helping students not only with finding abnormalities but also help build a bridge between 2D visualization with the context in 3D.



CT Color View

It's from here where the CT color view can be seen, this is activated from clicking the color view toggle on the model UI. The color view uses the data provided from the 3D slicer total segmentation to map all of the colors associated with the segmented organs onto our actual CT images. Now first of all to get this we will have to run our CT script on 3D slicers with the total segmentation model selected. This will create a sequence of CT images exactly like the original just with the color applied on top. We can then have it switch the image being grabbed based on if the color section is toggled or not. Now the CT color view is a very important part of the application because it gives more clarity to the CT images and makes it easier to derive a conclusion from it. Being able to have the organs colored such as they are on the models the students will be looking at gives a great reference point and allows them to identify the exact spot in which they are looking in the context of the model which is great for identifying abnormalities.



Take for example these images as an example, they display a patient who has a medical implant in his back. The two images on the left represent the same region of the body with one being color coded to display organs and medical implants while the other is uncolored. The two images on the right follow the same pattern but for a different region within the same patient.

The color coded views will clearly highlight different portions of the body based on the colors represented in the models. Such as a red coloring for muscles, a beige for bones, and a variety of colors which correspond directly to that of the model. In the left model you can see the pins of the medical device on the colored view which would otherwise be harder to see in the uncolored. On the right the colored view serves to show the full

context of the device which makes it easier to understand both the anatomy and the positioning of the medical device within the body.

Frontend

The user interface for our Virtual Reality application will be designed with accessibility in mind and following the UCF Accessibility Policy as much as possible when it pertains to VR. Some intended additions include a selection of colorblind-friendly color palettes, each having high contrast to allow for easier readability as well as different font options to choose from. Other accessibility options that we intend to include in the settings are adjustable brightness, volume, text size, and control mapping. Because we intend to include audio speech, we will also be including in subtitles that can be toggled on and off.

A major setback in VR is the motion sickness that can occur due to the movement in the app while your body isn't moving at all. Some ways that we can reduce this motion sickness that we are considering using are smooth movement and limited rotation speed. There will also be seated or standing modes to avoid any issues with clipping through walls.

Quiz UI

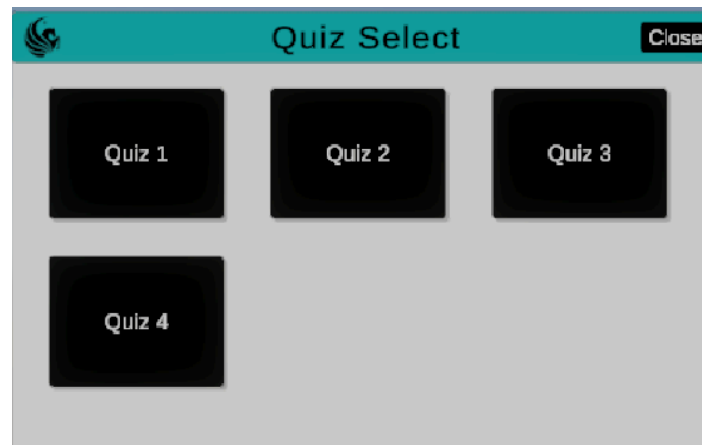


Figure: Display of Quiz Select (not final product)

The quiz begins with a quiz select where the user can choose from various quiz topics to test their knowledge on. The idea is for the Admin to be able to add and remove different quizzes and/or questions as needed. In the above image it only displays basic names such as Quiz 1 or Quiz 2 but in general the Admin would want to label the quiz so the user doesn't have the guess which each is about. The quiz select buttons are within a scroll box container so there is no limit to how many can be inserted, the user would just need to scroll as more are added. The above image displays a close final product but as we decided how we wanted to display the quiz for the users small adjustments were made as well as some color changes.

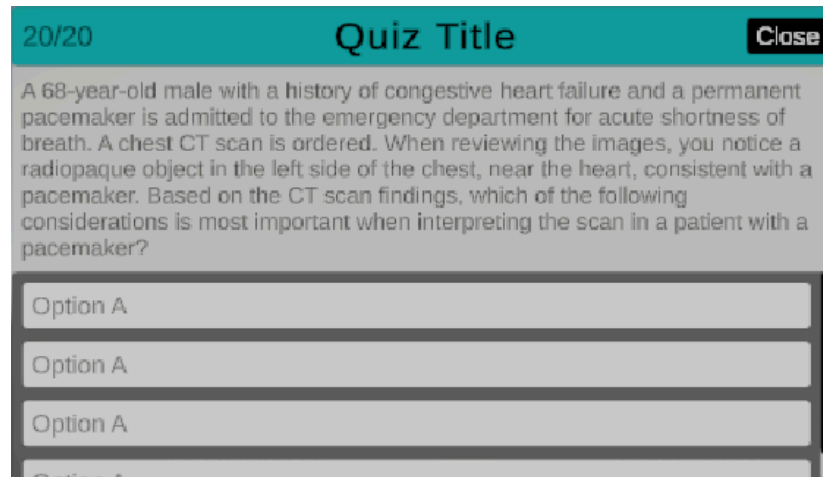


Figure: Quiz UI for Question and Answer (not final product)

Above displays how the question and answer quiz panel appears to the user. In our original concept we planned to have the question and answer in a scroll box container but when given the questions from our sponsor we realized that some of the questions might be really long (as seen in our mock quiz above) and thus the user would need to scroll back and forth if they wanted to read the question or answers again. To combat this we just added the scroll box container onto the answer choices so the question will always be shown.

After the user clicks an answer they are shown a results panel which tells them the correct answer and an explanation of why that answer choice is correct. They are then prompted to continue to the next question. After all the questions have been answered the users results will display and they are given the option to go back to the quiz select panel. These quizzes are untimed and are able to be taken an unlimited amount of times.

Originally we had a laptop for the user to click on and the quiz would pop up but after testing with it we realized it might be annoying for the user to have it attached to their head movements and it would be difficult to control where the user could go with the quiz. After some team discussion

we decided to display the quiz up on the whiteboard in the quiz room. This decision ultimately made the most sense so the user could use the full space of the quiz room and it would be easy for them to see and interact with. This also later paid off as the room looked more cohesive with the drag and drop quiz which is also up on a whiteboard.

Login UI

In the new semester we were able to implement the backend to connect to an admin portal. In order to use it properly we needed to have the user login (either as an assigned user or as a guest) in order to know whether to track their data but also to know information about them and their session. Therefore on the VR side we needed something for the user to actually interact with. So we came up with a login UI and in addition, a welcome screen to greet the user. The welcome screen would give a warning about using VR and then they would be taken to a login screen with username and password input fields.

We wanted to ensure the user had to be forced to choose to login with a user and pass or guest login so this UI was in another scene where the user first landed. They would only be allowed to continue to the main app once they choose an option. This would allow us to track changes to a user and implement session times as well as allowing the admin to make changes for that user.

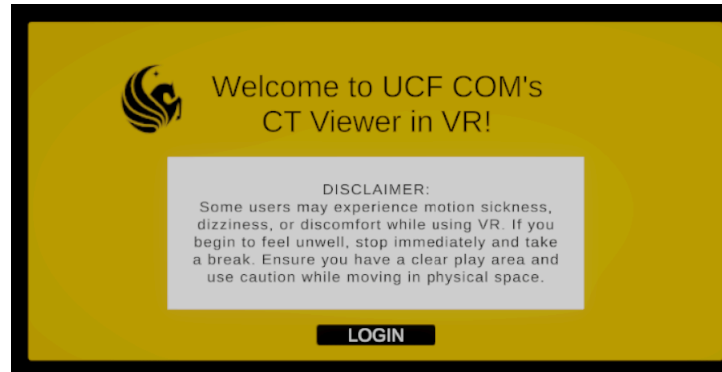


Figure: Display of welcome screen

When the user first opens the application they first will see a little title screen with a login button. Our sponsor suggested having a warning somewhere in the application as not everyone is used to VR and it's important they are aware of the possibility of feeling nauseous. Many other VR apps include a message about dizziness and having a lot of open space so we took inspiration from there. We also wanted to add some UCF theme somewhere in the app since the core of the application does not reflect clearly that it is a UCF COM application.

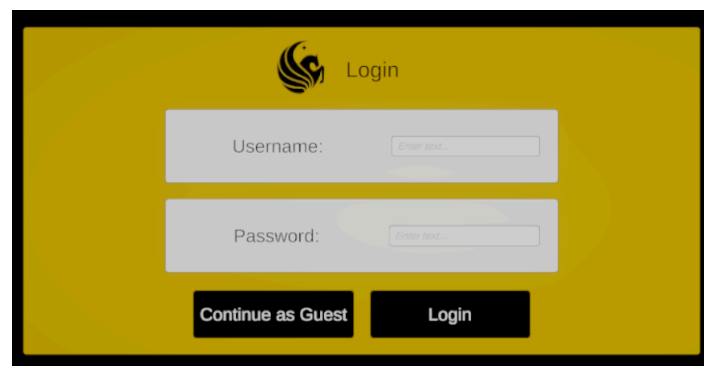


Figure: Display of Login screen

Once the user has read the warning they can click on the login button to be shown a login screen. This is where the user must choose to either login with their credentials or continue as a guest. Again, we went with a UCF theme to emphasize that this is a UCF COM application. The idea is that

an admin (for example our sponsor) would be able to add in credentials for students who should be able to access the application and have their data tracked (for quizzes). The guest sign in is there more so for showing it off to people or testing new features without needing to sign in every time but the intent is for a majority of users to sign in. To sign in, the user just needs to click into the input box and a spatial VR keyboard will appear that they can use.

Backend

For the backend of our project we were given access to a shared backend database and API that the other College of Medicine VR apps have been using. What we were given was out of date and messy. We aimed to update, improve, and clean up the code that we were given and apply it to our project without breaking what was already there. Halfway through our project creation, it was decided that a new database and server would be utilized to host all of the College of Medicine's VR apps. Previous iterations of this had issues and were named in a way that did not promote expansion. Therefore, we had free rein to disregard most of what the other projects were doing as they were needing to be updated by a separate team and could follow what we did for a more dynamic structure.

Unfortunately, due to firewall issues between UCF Main Campus and the COM servers, we cannot host the API on their servers and the admin portal and vr applications cannot access the database. This is something that we have no control over, though we hope for future teams to be able to make the necessary changes to get everything connected once the server is up and running.

Database

The database for the backend was originally shared with the other COM VR apps before it was transferred. A concern was brought up by our sponsor that there would be static fields for the number of answers to a question or question to quiz. We solved this issue by making our SQL database relational. This way, everything is still connected to each other but allows for dynamic growth in the amount of quizzes, questions, or answers you wish for.

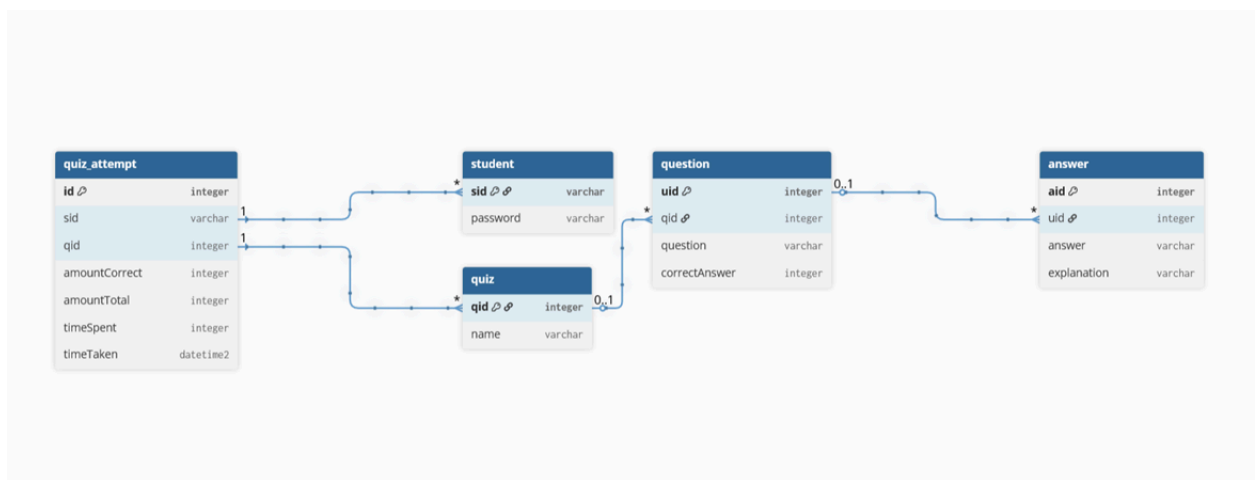


Figure: Database Tables

Each of the tables outside of the student table are specific to the Clinical Imaging VR app. The student table was kept the same as previous groups had it, even now, though there are some columns that could be considered redundant. As shown in the figure, the main components of the student table are the sid and password for login purposes, however, it also consists of classSection, appSettings, and loggedIn columns. These three columns have no current uses, with classSection being the column that has the most potential to be used to separate content by class section. The appSettings and loggedIn columns do not hold any meaningful information and the loggedIn values would not matter due to any difficulties with losing connection to the database.

Each of the Clinical Imaging tables have a prefix of ct in their actual table names in the database. This prefix naming system can be used for the other projects as well, to differentiate which tables belong to which projects in order to avoid confusion in a shared storage space. Another thing to note is that it was decided to utilize varchar instead of nvarchar due to nvarchar requiring twice the amount of data compared to varchar. This is because of nvarchar's ability to use extra characters such as emojis, which are not exactly needed in an academic setting and thus not worth the storage tradeoff.

API

The .NET 8 REST API is designed to facilitate communication between the database and any frontend apps that pull information from it, including the Admin Portal. This API is intended to be used across every College of Medicine VR application and hosted on their servers in the future. Thus, each of the API controllers must be dynamic, allowing for any future updates to take place seamlessly. The controllers are easily readable in their layout and contain a multitude of comments explaining what each function and its parts does.

Something to note is that the API "controllers" are not actually .NET controllers as those require a different formatting than was used. Instead, a minimal API approach was used due to the smaller scale of this specific project and how few controllers in total would be needed. Both approaches get the same job done, though the minimal API approach requires for each "controller" file to be mapped and called in the program.cs file as opposed to just calling all controllers that are automatically mapped.

Due to potential security concerns with the API being put onto the official UCF COM server, extra attention was placed into minimizing security risks. Because we are using an SQL database, we are at risk of an SQL

injection attack with improperly handled SQL communications. An injection attack could lead to private student information being accessed by unauthorized parties. Another serious security concern that was inherited by past groups was the lack of hashing passwords. This is very bad because if someone got access to the table that isn't authorized to be there, every student's account is no longer secure. We did fix this and implemented hashing for the login located in our project's "controller" to fix this issue. It is expected that another final code review will be performed before the API is transferred onto the official servers.

Admin Portal

The admin portal is intended as a way for an admin (our sponsor) to go in and be able to manually adjust the students that have access to class quizzes and track their response data, be able to modify the quizzes, questions, and answers individually, and be able to show student response data. Due to this portion not being a part of the MVP for this project, progress on this has been minimal, though major improvements and adjustments have been made since it was last worked on.

Originally, this website was created using Angular and displayed personal information about students on the student dashboard which was a serious privacy concern. Both of these were changed in the course of updating the website. While the base framework stayed the same, we moved away from Angular to React at the request of our sponsor. As for the privacy concerns, this was changed so that no passwords displayed, even though they are now hashed.

Because this site is only supposed to be viewed by the admins, there is a login screen that is routed to when you have not been verified yet. The design for the login screen was taken from the current login screen that we have for the Clinical Imaging VR app as it is hopeful that the logins can be

linked across each COM VR application and the unity would only showcase this.

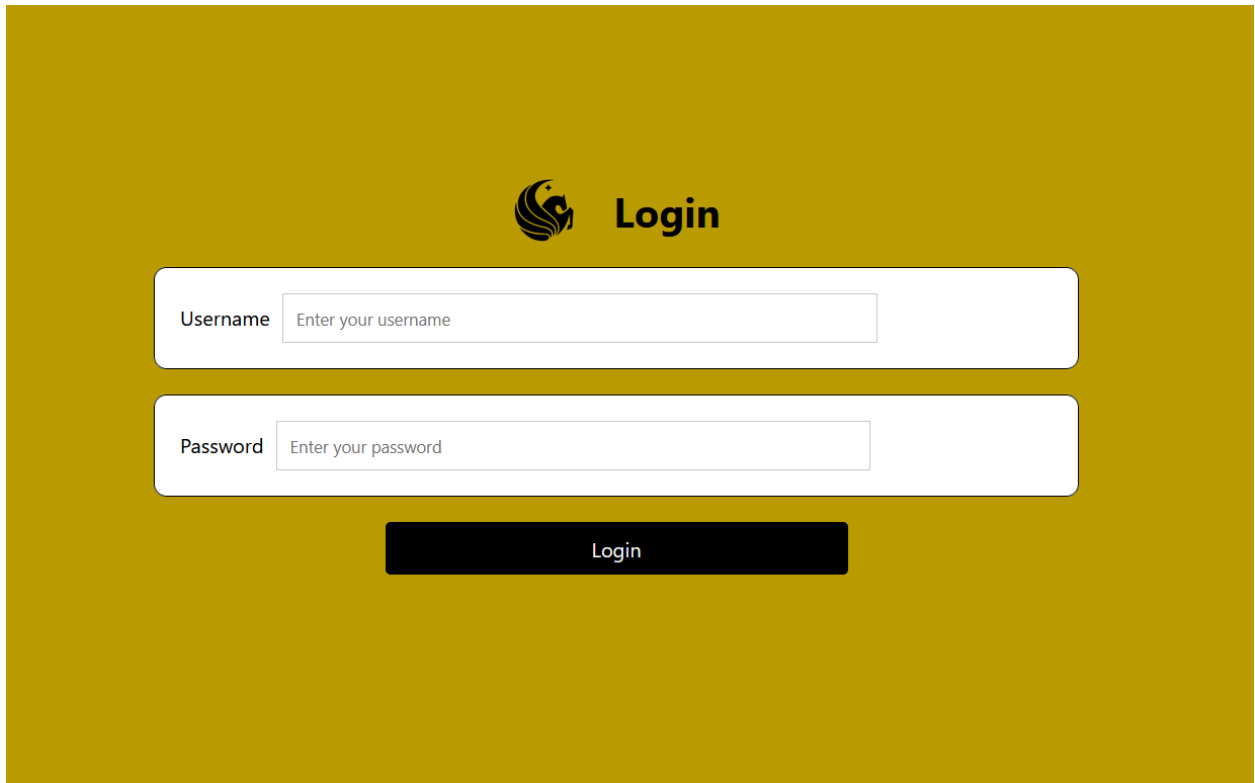
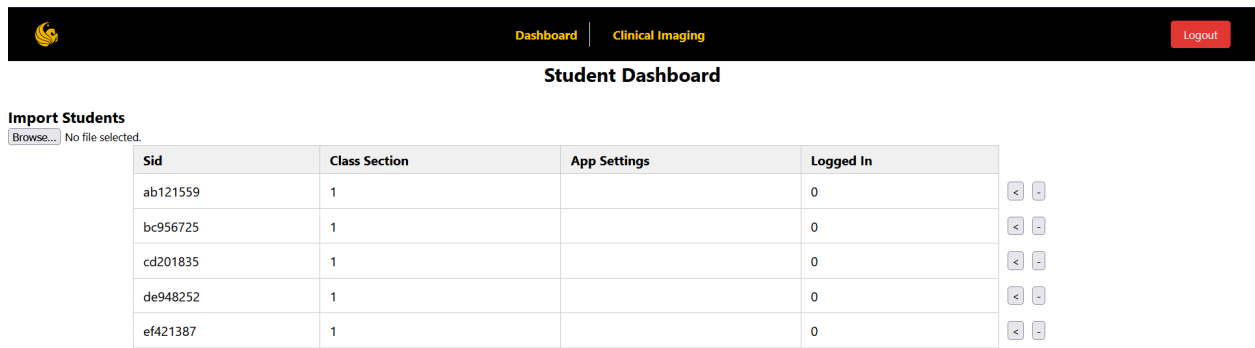
The image shows a login screen with a solid olive-green background. At the top center, there is a black circular logo with a white stylized flame or bird-like shape inside, followed by the word "Login" in a bold, black, sans-serif font. Below this, there are two white rectangular input fields stacked vertically. The first field is labeled "Username" on the left and contains the placeholder text "Enter your username". The second field is labeled "Password" on the left and contains the placeholder text "Enter your password". Below these fields is a black rectangular button with the word "Login" in white, centered text.

Figure: Admin Portal Login Screen

At this point in time, the login does work, though the username and password are hardcoded in the login page file. This can be changed by possibly switching the students table to a users table and adding in an admin column to check if they are authorized, or perhaps adding a separate admin table. The login session of the user is tracked and will stay logged in even if the screen is refreshed until logout. The logout button can be found in the upper right corner of the navigation bar when the user is logged in.

The navigation bar allows the user to navigate throughout the portal. Currently there is only navigation for the student dashboard and clinical imaging pages, though the framework for future projects to be added is in the code. Each project should be it's separate thing in the navigation bar to avoid cluttering it up with each individual pages that can be routed to.

Instead, it was decided to order by application and have a dropdown menu appear when clicked to show all of the pages that are related to that application. The clinical imaging currently only routes to the finished quiz view and unimplemented quiz responses, though the dropdown does work.



Sid	Class Section	App Settings	Logged In
ab121559	1		0
bc956725	1		0
cd201835	1		0
de948252	1		0
ef421387	1		0

Figure: Admin Portal Student Dashboard

On first logging in, the user will be sent to the student dashboard, which allows the user to view what is currently in the student table except for the password. The button at the top allows for a csv comma file to be selected from your files and imported all of the contents in that file and translate it to a readable json structure to be added into the student table. This is currently running correctly, with the sid being linked to nid and the password being set to the student pid. It should be noted that the classSection column of the student table is too aggressive in assigning initial values as each is always set to 1 automatically despite different values being initially added.

The buttons on the side of the shown table in the student dashboard allow for the student class section to be edited (<) or deleted (-). These features on this page currently do not work and need to be reviewed in the API located in our teams files.



[Dashboard](#)
[Clinical Imaging](#)

Logout

Import Quiz

Browse... No file selected.

Type	ID	Name / Text	
Quiz	1	Pacemaker Quiz	< -
Quiz	2	DBS Implant Quiz	< -
Question	2	A 75-year-old patient with Parkinson's disease presents for evaluation. A non-contrast head CT is obtained. You notice a linear hyperdense structure extending into the basal ganglia region with surrounding streak artifact. Which of the following is the most likely explanation for this finding?	< -
Question	3	On a non-contrast CT head, you notice a linear hyperdense structure passing into the basal ganglia with streak artifact. The patient has a history of Parkinson's disease. What is the most likely finding?	< -
Answer	10	Calcification of the basal ganglia.	< -
Answer	11	Deep brain stimulation (DBS) electrode.	< -
Answer	12	Acute intracerebral hemorrhage.	< -
Answer	13	Retained surgical material.	< -
Question	4	When interpreting CT scans in patients with DBS electrodes, which of the following errors is most important to avoid?	< -
Question	5	A CT scan of a patient with DBS shows streak artifacts radiating from the electrode site. Which of the following best explains this phenomenon?	< -
Quiz	3	Hip Replacement Quiz	< -
Quiz	22	Imported Quiz	< -

Figure: Admin Portal Clinical Imaging Quiz View

The quiz view was difficult to organize because of how dynamic in size it is. Eventually I settled on organizing the table in a way that it would operate like a dropdown menu, where only the quizzes are displayed initially and on clicking one, it's questions are displayed, and on clicking one of those, the answers are displayed. It was hard to differentiate which of the information belonged to what type: quiz, question, or answer so I tried to make it more obvious by including the type field and changing the background colors in the table based on which type it is, as well as adjusting the border size. After completing this, I still had an extra column that was only used for the question portion, the correct answer id. I figured I could kill two birds with one stone and deleted the unnecessary column that made it hard to figure out which answer was actually correct, and instead implemented a highlight of the correct answer that made it more obvious.

The buttons to the right of the table operate the same way that they do in the student dashboard, edit (<) and deleted (-). However, these ones do work and have been extensively tested. The edit button opens a popout menu that autofills in the information that is already in the table for the

selected row. From here you can adjust the information as wanted and upon submitting the updated information, it will refresh to show the updated information. The delete button asks for confirmation before deleting the selected row, but upon confirmation does successfully delete it.

The button at the top left of the page allows for a Canvas quiz to be imported. In Canvas, there is an option to export a quiz. Upon exporting the quiz it creates a .zip folder that contains an xml file with the quiz information in it. Once the .zip folder is chosen, the program unzips the folder and searches for the needed xml file. Once this file is found, it translates it into plaintext and parses through it to find certain key words that link to information that will be needed for the json to add a new quiz, question, and answer. Once all of the information has been formatted correctly and submitted, the information is added into the database and shown in the table.

Currently, the admin portal is not even remotely at a point that could be considered finished. It still needs to have the results page for the Clinical Imaging as well as the reset of the pages for the other VR projects that fall under the purview of our sponsor.

Level Design

Our current idea is to have an introduction room for the player to familiarize themselves with the controls, the main lab room where the player can look at the CT scans, and lastly a quiz room where the user will be able to test their knowledge. Planning the levels include: interactions with objects, the scale of the objects so it feels like a realistic lab, and how the player is going to move around the lab.

Tutorial Room

Overview

The tutorial room will include the basics of using our VR app and movements like moving, swiping, clicking buttons, and grabbing objects.

User Interactions

In this area the user will be presented with a table of objects where they can learn and practice how to grab an object in front of them and how to set the object back down. The user can repeat this action as many times as they need to get comfortable with the mechanic. There will be another table in the room to represent the CT scans and 3D models, this area is less about the actual scan/model and more for the user to learn how to navigate the scans/models mechanics with the different planes, different views of each. This mechanic will most likely be the most difficult to grasp so the user will be able to mess around with it for as long as they want. Finally, the user can walk to the door and enter the next room.

Design

This room will look like a medical room with key items like tables and a door leading to the next room. There will be other items within the room as we see fit to fill in the space but they will not be interactive. Objects such as a folder of papers, clipboard, etc. will be available to practice picking up. There will be a monitor/projection for the CT scans to be displayed. We will be mindful of the sizing for the scans so that the user will be able to fully interact with them without the display being limited. However as this is a tutorial it may not be as full sized as the display in the main lab area. Lastly,

there will be a door leading to the next room. The door will be closed and the user will need to interact with it in order to proceed to the next room. For a better user experience we have decided to have the environment have a lot of light so everything is bright and easy to see.

Other

This room will have relaxing background music playing in order to provide a more immersive experience. For customization the user will be able to adjust the volume for the background music.

This room will also display the credits on the wall including all group members names and any thanks as needed.

Lab Room

Overview

This room is where the main user interactions take place. The user will be able to navigate through the room and interact with stations in order to swipe through the CT scans and explore 3D models.

User Interactions

Once the user has moved into the main area they will be faced with the CT Scans and 3D models. They will be able to interact with the projection in order to trigger the mechanic. They will be able to swipe through CT scans and choose which to study. Each CT scan will be represented in the axial, sagittal, and coronal plains, and the user will be

able to interact with the CT scan directly to switch between slices. There will also be a 3D model representation of the CT scan, which will show the inner slice corresponding to the current view. The user will be able to switch between different CT scans using a menu with radio buttons. The screen will also display the model of where the user is in relation to the body so they can orient themselves.

Design

This room will be designed as a medical imaging environment. We will have tables displaying anatomical models and CT scans around the room for environmental purposes. In the back of the room we will include other environmental objects as needed such as plants, cabinets, and trash cans. The area to view the CT scans will include a table/desk, scattered 3D models for display, CT scans around for display and then the main projection/monitor area which is the main area for viewing and user interactions. We aim to make this area as big as possible without compromising user comfort, we want them to be able to utilize the freedom of VR but not too huge that it's difficult or inconvenient to operate. The overall design within the room will remain similar to the tutorial level in order to preserve the sense of cohesiveness throughout the app (ie; similar lighting levels, colors).

Other

This room will also have background music in order to enhance the player experience. For customization, the volume of the audio will be adjustable in the menu.

Quiz Room

Overview

This room is where the user will be able to test their knowledge of CT scans. This will include a variety of quizzes (multiple choice, fill in the blank, short answer) that the user will be able to answer and receive immediate feedback.

User Interactions

In this room the user will be able to interact with a whiteboard in order to enter the quiz UI. In this UI they will be given the CT scan and 3D model side by side a question. The user will be able to answer the question and then press a "Submit" button in order to receive feedback. If the user gets it correct it will signal with a checkmark and a "Correct." If the user gets it wrong it will signal with an "x" and an "Incorrect". Additionally, they will receive feedback on why they got it wrong. This will pop up over the question part of the UI and be able to close to see the original question as needed. Finally, the user will be able to exit the quiz or continue to another question.

Design

The room will be designed as a classroom. We will include desks and chairs as well as a whiteboard at the front of the room. There will be plants included in order to give it a more realistic environmental feel.

Other

This room will have background music in order to enhance the player experience. In comparison to the other rooms, we understand some people quiz better in silent rooms therefore it will be acknowledged that the user can change the volume or mute the background music in the menu.

Room Design Changes

As we moved through the project we realized there were some adjustments we needed in order for the rooms to fit what we needed. To begin, when we put the actual headsets on (versus viewing in the Play Mode in Unity), we found the scaling of the rooms to be off from what we imagined it to be. When we began to add in objects we were not testing on the actual headsets and just basing the sizing off of play mode and through that we thought the rooms were too small and thus we made the rooms bigger. After we began testing in VR through the headsets we realized the objects we added were really big and since we had changed the room size to match the objects the whole room just seemed very unrealistically big. After we realized this sizing discrepancy we began to resize the objects, using the headsets more frequently to ensure the objects mirrored objects in real life. With this, we were able to size down objects and eventually size down the rooms. We wanted to make the environment friendly for users to play sitting or standing so we also were mindful of not making objects too tall or making the board too difficult to see if you were sitting.

In addition to general scaling changes to objects we also needed to remodel how some of the rooms were sized. In the beginning the lab room was pretty small (even in our concept ideas) and the quiz room also took on a more square shape than rectangular. However, as our ideas for how to

display the CT images and 3D models changed we needed to expand the lab room in order to fit a side by side for the normal and abnormal CT images. This later also applied to our quiz room as we decided to separate the multiple choice quiz and the drag and drop quiz. In the quiz room we decided to also have two side by side whiteboards in order for the user to have quick access to whatever they wanted to do.

Other small changes throughout our process was the design to the walls which was created by us to have a more comfortable setting instead of just bare default white walls. While we went for a more comfortable environment within the tutorial and quiz room we stuck with a more medical feel for the lab room with the tiled floors. Additionally we needed to add or subtract windows for the walls based on if we stuck something on that wall or not. Lastly, we wanted to include baked lighting for our rooms just to give it a bit more home-y feel, however we learned that should be one of the last things you do since we made a bunch of object changes after our initial baked lighting.

In our tutorial room we also made some changes to better suit how the player would be interacting with things. Originally, we planned to have two tables and a general welcome board. We wanted two tables, one with things to practice interacting with and another with a computer and tablet. Further in our development we realized we had no real use for the computer and tablet so that was removed and the objects to practice picking up were centered. We also decided to make the welcome board more interactive and to have it display a general VR tutorial. This tutorial explains what the buttons do on the controller as well as explaining the accessibility settings you can turn on and off in our menu. For a first time user starting in this room helps them orient themselves with the controllers but it's also bypassable if the user is returning or already knows how to use VR.

Some other small changes included adding in a poster on the wall to represent the planes of the body. We also included a clinical PEARL for the user to reference. Both were given to us by our sponsor. We added the planes poster to the left of the CT scans board so the user would only need to turn their head slightly to reference it.

Layout

Initial Layout

The initial design was made in Figma but since Figma is meant for 2D user interface designs, the team mainly imported images and menus from Meta's own Figma account. The first page of the Figure shows the initial plan for the layout of the virtual reality lab. The first thing the users spawn in is the tutorial/ introduction room that will explain the features and the options for the users to use while in the application. The next room will be where the user can cycle through the CT scans and the 3D model assets to study from. Then the user has a choice between going to the quiz room next, where they can take quizzes to practice from, or the credits room, where the team and each member's roles will be shown.

After showing and discussing the design with Melissa Cowan it was decided to combine both the credit room and the tutorial room together. In the initial tutorial room there will be a section that will act as the credits section showing our names and each of our roles.

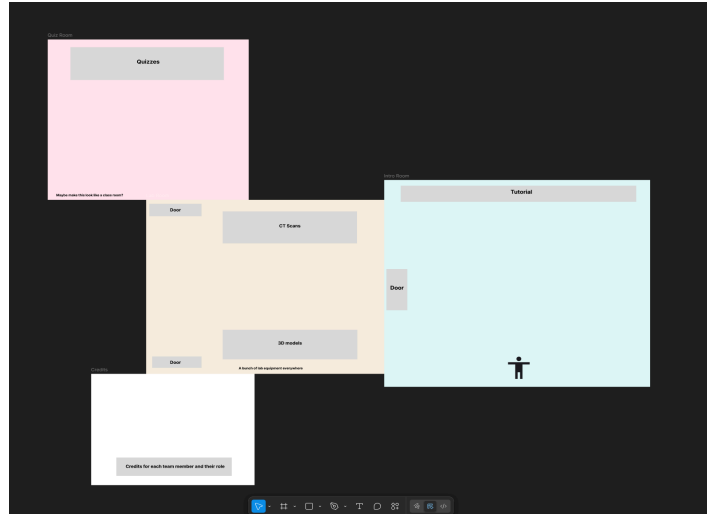


Figure 15: Initial Layout of the App - This was the original Figma layout design that we pitched to the sponsor (created by Arianna).

The next page on Figma shows the assets uploaded from Meta's Figma which gives a good example of the potential menus, interactions, and environments that are standard in a Meta environment. It shows the possible menus for changing the settings and pulling out features such as the ruler or the labeler. Also it shows the controller and affordances, pointers and cursors, that the users will have to interact with in the virtual reality environment. Additionally, one of the environments shows the potential menu to pick different organs, 3D models or CT scans for the main lab room.

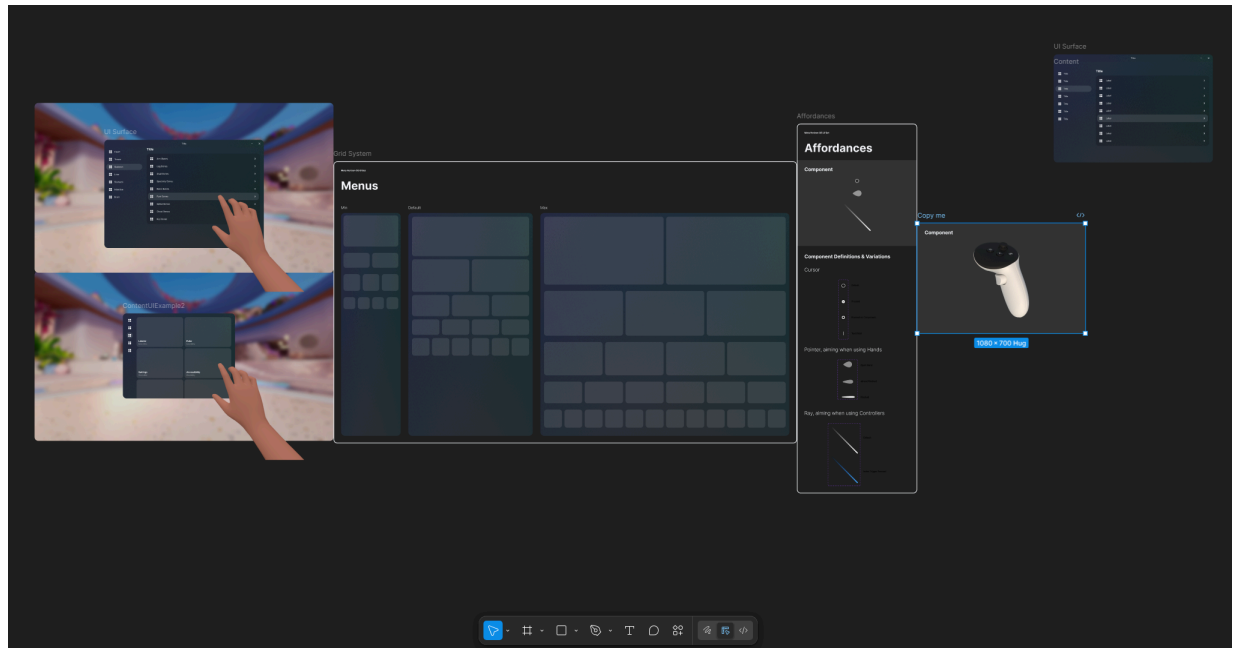


Figure 16: Basic UI layout - This is the initial plan for the UI of the menus and affordances (created by Arianna).

The below figure is the final room design created in Unity which includes functional walls, floor and ceiling, and door frames. This is our base design with no textures added to the walls and no assets placed inside. The layout is very similar to our initial figma design with a few design changes to better optimize the player experience. The very far right room is where the player will begin when they first get into the game. They will do the tutorial and then proceed to the main lab room to the left. This room is where most of the interactions and learning take place as the user will be able to explore CT scans and 3D models in one section of the room as well as practicing measuring the models with a virtual ruler in another section of the room. Lastly, when the player is ready they can proceed to the last room where they can take quizzes.

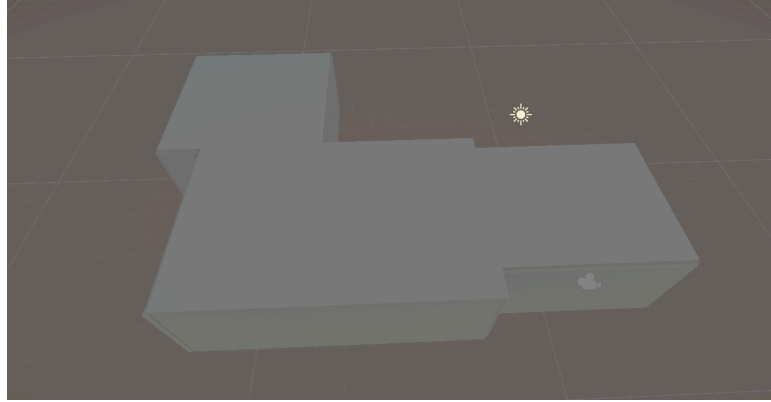


Figure 17: Room Design in Unity - After showing the layout plan to the sponsor she made some changes and then we created the final version in Unity (taken by Zoe).

The rooms have been scaled to an appropriate size so there is enough room to navigate but not too big that the user finds moving through the rooms too time consuming. In our original design we had the CT scans and 3D models in separate areas of the main lab area but our sponsor pointed out it might give the users a headache to have to go back and forth between the two areas so we decided to alter the interior of the rooms to better suit a VR experience. We decided on a relatively simple room layout so it is convenient for the player to locate the areas they want and we can dedicate more time to ensuring the interior has all the features the user will need to succeed.

Our next steps for the room design will be adding texture to the walls, adding in an actual door the user can interact with in order to move room to room, as well as actually adding in assets/objects to the room for both interactive features and environmental design. For interactive features we want to add in tables to hold the items, monitors to view the scans, a whiteboard for the quiz room and so on. For environmental features we want to add things like skeletons, trashcans, clocks, cabinets, etc. to really make it feel like a realistic environment. In the next semester we will also want to implement lighting within the rooms, based off of other VR games we have

demoed, we decided a bright environment is optimal in order to ensure users can properly see around the room and we believe the bright lighting is also common for medical offices so it fits the room environment.

Story Board

In order to ensure we were all on the same page for the vision of the project, our sponsor recommended us to create a storyboard using a slideshow. In our slideshow we included our color scheme for the UI and for the general environment. We also included the general design ideas for the rooms including environmental objects, and a general idea for our CT scan/3D model UI.

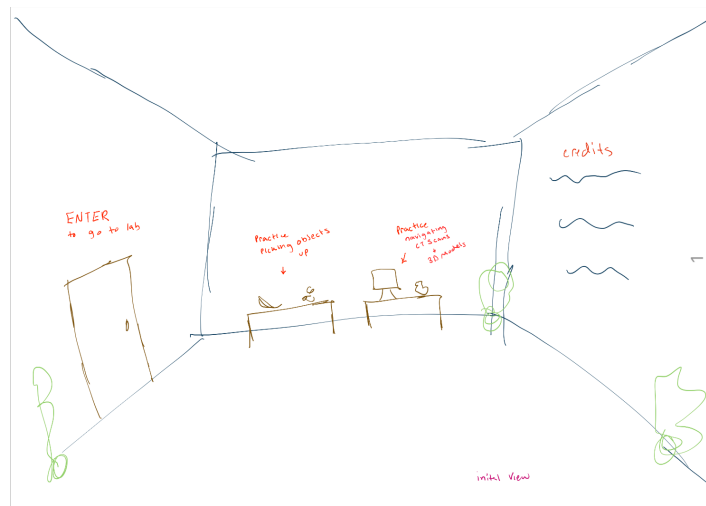


Figure 18: Storyboard for Tutorial Room (created by Zoe)

Above is our storyboard of the tutorial room, while it is a very rough sketch of the idea, it shows all of the important details that we need to know. We included some of the environmental objects we think we can use to fill the space such as plants. Additionally, we have shown where we want to include the credits, a display on the wall, out of the way but someplace that the user will be able to actually see. Lastly, we have the main tutorial

area where the user will be able to learn how to use features within our app. This we have decided to place right in front of the player when they first enter the game as it is the main feature the user should be focused on. We have also included the door as it's the only way the user can enter the next room.

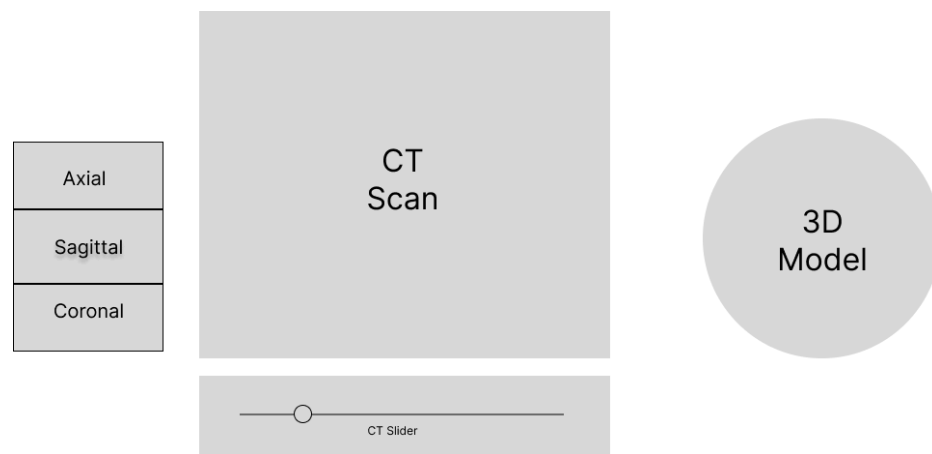


Figure 19: Initial UI for scans and models (Created by Kyle)

The above figure displays our initial idea for the UI for the CT scans. We realized there were a lot of different features we would need to include for the CT scans in order for it to be beneficial to medical students. So we have included the main scan viewer and an area for the 3D model near it so the user will be able to see both at the same time without it being a strain to find or see. Our sponsor had mentioned that even though we want to utilize the space in VR, it's important not to make it too uncomfortable for the user to actually use the features (having to turn their head a lot or move around for small things).

In this design we have the slider and the buttons for the different planes. This ensures the user will be able to switch between views in order to get the full experience of CT scans. However, we were showing our sponsor she mentioned it is better if the user would be able to see everything at once, so we have decided to implement more of a grid view

with 4 images showing at once, three will be sliced parts and one will display the whole model. Additionally she requested we add in a model with the planes to show where they are so they will be able to orient themselves better. We plan to implement all of these changes into the final UI design next semester once we begin to code.

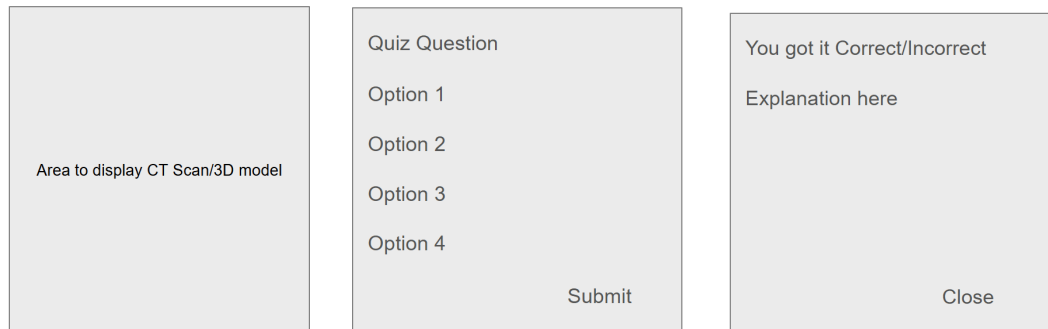


Figure 20: Initial quiz UI from storyboard (created by Zoe)

This figure shows a very rough idea of how we want the quiz UI to look like. On the right is the area for the scans, in the middle is how we plan for the quiz to look like. We will display the question at the top and it will be followed by the radio buttons of options which will be maxed out at 4 options, finally there will be a submit button when the user is ready to check their answer. On the left is what will appear overtop the original quiz question box. It will let the user know if they got the question right or wrong and then an explanation will appear. We have also included a close button for the user to return to the original page. After the user has received the feedback the submit button will be turned into a "Next" button so they can move to the next question.

The storyboard included a lot of our original ideas and what we thought would work well but once we begin the actual process of creating in VR we expect plans to change and evolve. Not only will we be testing the application but we will be showing our sponsor our updates and getting

feedback. Additionally we plan to have actual medical students test our application and based on that we can also adjust designs or add in features they may suggest.

Integration

Ensuring that there is proper integration between the level design and UI is imperative to achieving an overall cohesive application design and layout. The main focus for this will be on implementing the 3D models into the VR lab that match with what is happening in the UI and keeping to an understandable and obviously medical feel.

Style Guide

Introduction

The style guide that was provided to us by our sponsor includes important information about how we can keep ourselves consistent and understandable throughout the project, and allows for any future teams to come in and be able to understand where all of the files are located and what everything is meant to be and do. We intend to follow this style guide for every programming aspect that we include in the final product. There are multiple different sections of the guide that cover different parts of the project. The major sections cover the Introduction, Project, Asset Naming, Scripting, and Custom Tools.

Project

The project section of the guide covers how to handle version control, organization, and folders. Version control is essential for contributing to the project and should always be used. Each push to the Git repository must be properly named and tagged using a Release.Update.Patch structure. The Unity project's version should also be updated via Edit > Project Settings > Player > Version, and every version push should include a changelog update. A Release marks full project completion or major overhauls, an Update introduces new features or minor reworks, and a Patch covers bug fixes or small improvements. For incomplete features, an experimental suffix is used, where the State (alpha or beta) indicates development progress, and the Index tracks iterations. Once a feature is finalized, the experimental tag is dropped, continuing with the standard versioning format.

Moving on to overall organization, the guide explains that assets should be organized by scope > asset type group > asset type > category > room. Scope is the root folders under Assets that are determined based on where assets come from and how they are used, such as Packages and Samples. The asset type group and asset type can both be derived from the name; the asset type group contains the asset types that are able to be grouped together in some sort of relation while asset type is simply the asset type folder. The category is where in the project the asset is being used in a general system. And finally, the room is simply what room the assets are physically used in, though in the scope of this project, this lowest level will most likely not be used as we are not working in multiple rooms, though we may keep it to allow for easier room additions in the future.

In the project folders section, the guide explains how each folder is expected to follow the Pascal Case (where the title has no spaces and all

beginning letters capitalized), except in the case of any imported folders. In the case of importing folders, we are expected to keep the original naming convention that it came with. The same applies to cases where folders are automatically generated.

Asset Naming

In the asset naming section, it contains the structure and modifiers for how assets are named. The structure is explained to be similar to the folder naming convention in how we are expected to use Pascal Case, except in the case of prefabs, which are expected to use Title Case. To provide more clarity, when naming an asset there are five parts that should be followed: prefix, base name, variant, index, and suffix, with each part being separated by an underscore. It is expected that not all five parts are needed, but there should be the base name at a minimum. The prefixes and suffixes should always be present if applicable and are defined further in modifiers. The variants section should only appear if the asset is a variant of the original asset and not an original itself. If the asset does not need a more descriptive name, possibly because there are multiple different types of the same base asset, the index is used to number them starting at 1.

The modifiers part of the asset naming section covers the different asset types' prefix and suffix naming conventions and lists them out for convenience, showing the prefix and/or suffix next to the asset type. It covers different asset groups such as: general, rendering, and texture assets.

Scripting

The scripting section covers scripts, classes, variables, and methods and gives instructions on how to handle each. Scripts attempt to follow the style of the Unity source code but due to inconsistencies, clearly defined rules for script naming and overall management have been put in place. Scripts should only ever contain one public type, though they may have multiple non-public types in addition. The script names should always be in Pascal Case and be named to match their public type. Digging deeper into script handling gives information on what to do with interfaces, enumerations, attributes, comments, and the debug log.

The class part of the scripting section deals with classes and how they should be handled, such as class members should always be defined with an access modifier and follow the C# preferred modifier order. The members should be organized in the script according to their variables and methods. They should be grouped together in this fashion and further organized by their access modifiers.

The variables include constant (const) fields, static fields, serialized fields, fields, events, and properties. Const fields should use the prefix `k_` and Pascal Case, while static fields use the prefix `s_` and Pascal Case. Regular fields should be private or protected, use the prefix `m_`, Pascal Case, and the readonly keyword where applicable. Boolean fields should start with a question word. Fields meant to appear in Unity's inspector should use the `[SerializeField]` attribute on private or protected fields instead of making them public, and should be ordered by importance and grouped using `[Header("...")]` with spacing for clarity. Events should be preferred over direct method calls to reduce dependencies and improve performance. They must use the event Action type (not `UnityEvent`), be named in camel Case, and be instance-based (never static). Listeners should be added in `OnEnable`

and removed in `OnDisable`, avoiding `Awake`. Always check for null before invoking an event and name listener methods using the `On` prefix (e.g., `OnSomeEvent`). When dealing with `UnityEvents`, always add and remove listeners through code rather than the inspector to prevent reference loss. For passing parameters, use lambda expressions. Properties should also be in camel Case and used to expose private/protected fields. Lambda expressions are preferred for simple getters and setters, and properties can store references to external scripts' members for convenience.

The methods include constructors, event functions, static methods, and methods. Constructors should generally be avoided in classes that inherit from Unity types, as Unity manages their creation—initialization should instead occur in `Awake`, `Start`, or a custom `Initialize()` method. Constructors are appropriate for non-Unity classes. Unity's event functions should be marked private, used in contextually appropriate situations, and ordered according to Unity's execution sequence. Static methods are encouraged when instance data isn't required. All methods should follow Pascal Case naming, with event listeners using an `On` prefix. Methods returning a bool should pose a question, and methods with no parameters that return a value may be better as properties. For time-based logic, prefer `async UniTask` or `UniTaskVoid` over `IEnumerator` coroutines, as `UniTask` offers performance advantages through its lightweight structure, generic return types, and efficient memory usage.

Custom Tools

The custom tools section covers extension methods, animator event handler, editor scene loader, and inspector button. Extension methods in C# allow developers to enhance existing types by adding new functionality, improving code readability, and making processes more concise. These

methods should be placed in static classes named after the type they extend, with "Extensions" appended. The methods themselves must also be static, with the first parameter specifying the target type and beginning with the `this` keyword. Extension methods can be especially powerful when used with the `IEnumerable` interface and LINQ, and they can be chained together as long as the return types are compatible.

The animator event handler (*AnimatorEventHandler*) is used to enable scripts to interact with an Animator component by responding to specific animation states. To use it, the *AnimatorEventHandler* must be added as a behavior to the Animator Controller asset. Since Animator Controllers function as state machines, *AnimatorEventHandler* extends the *StateMachineBehaviour* class. However, the *AnimatorStateInfo* provided by these methods does not include the state name as a string, only as an integer *shortNameHash*. To compare states by name, developers should use *Animator.StringToHash* to convert the state name into its hashed form.

Focusing on the editor scene loader (*EditorSceneLoader*), to minimize duplicated objects and improve scene organization, the application uses a dynamic scene loading system that separates core systems into their own persistent scene. However, since Unity normally replaces the current scene when opening a new one, developers would need to manually load the system's scene each time. To streamline this, the *EditorSceneLoader* intercepts Unity's asset-opening behavior and ensures that the system's scene is automatically loaded whenever any scene is opened. Additionally, an empty scene called "All" acts as a trigger for the script to load all scenes from the build settings at once, ordered by their build index—useful for editing the full map or making broad changes. Scenes can be added to this list through File > Build Profiles > Scene List.

The inspector button attribute in Unity is a useful tool for testing gameplay features directly from the Inspector without needing to play through the game or recompile each time. It creates a clickable button in the Inspector based on the method it decorates, named after that method and triggering it when clicked. It can optionally be enabled outside of play mode, though caution is advised since some methods may not work properly in the editor context and should remain runtime-only. For methods intended to run in the editor, preprocessor directives should be used. To organize these buttons visually, the [*MethodHeader*] attribute can be used to add headers above methods.

Build / Prototype / Test Plan

Testing Strategy Overview

Our testing approach is structured in multiple phases to ensure that the Clinical Imaging CT VR application meets its goals of accuracy, usability, and educational value. The testing process will involve both internal QA and hands-on feedback from first-year medical students, who are the target users of the application.

Phase 1: Internal Prototyping and QA (June–August)

During early development sprints, internal testing will focus on:

- Functionality Testing:
Ensuring individual features (UI menus, object interactions, quiz logic, settings toggles) work as intended.

- VR Performance Testing:
Measuring frame rates, input latency, and smoothness of object rendering to confirm the experience meets comfort standards.
- Accessibility Testing:
Verifying that UI elements are legible, subtitles display properly, and comfort settings (e.g., teleportation, snap turning) reduce motion sickness risk.
- Developer Walkthroughs:
Team members will rotate roles in playing through the app as if they were students to identify bugs, UI friction, and areas of confusion.

All bugs and usability issues will be logged and tracked through our Jira system. These will be addressed in each sprint based on severity and priority.

Phase 2: Controlled Pilot Test with Medical Students (September–October)

Once a stable prototype has been built with core MVP features in place, we will begin user testing with first-year medical students from the UCF College of Medicine.

Recruitment and Setup

- We will work with our sponsor, Dr. Melissa Cowan, to recruit volunteers from the target student population.
- Headsets will be provided on-site, and students will participate in supervised sessions.

Testing Procedure

- Each student will go through a guided onboarding session using the tutorial scene.
- They will then explore the lab freely, complete a short CT identification task, and take a sample quiz.
- Observers (team members) will record:
 - Completion time
 - Errors in interaction
 - Confusion or misuse of UI
 - Motion sickness or discomfort reports
- Students will complete a short survey at the end of the session, evaluating:
 - Ease of use
 - Comfort
 - Educational clarity
 - Feature usefulness
 - Suggestions for improvement
- Optional: Think-aloud feedback may be collected during gameplay, with consent.

Phase 3: Iterative Improvement and Final Testing (October–November)

Based on medical student feedback, we will:

- Refine interaction logic
- Improve clarity and flow of instructions
- Optimize comfort settings and accessibility features
- Fix identified bugs or pain points

We will then conduct a final usability test with a small group of returning and new testers to validate improvements.

Design

Throughout the virtual reality application the intention will be to replicate a real lab so students feel it's a familiar and comfortable environment to study in. In order to achieve this, the team will be using lab table and lab chair models throughout the virtual environment. One potential idea is to replicate the look of the new UCF College of Medicine student lab. A virtual walkthrough of the new building has been provided to the team, replicating it would allow the students to feel they are in a familiar learning environment and could potentially encourage more studying through the application. The color palette for the user interface will stick to the default look and feel of Meta's UI, with white text on a black background that makes it easier to read in VR as opposed to black on white. Any color that we use as indicators for buttons will be tested in greyscale to see if the contrast is great enough. The color palette for the rooms will mainly consist of brown wood furniture and a teal blue that takes after Lake Nona Hospital. In the

future this may be changed to a more UCF black and gold palette, but as of this writing we will be using teal blue. We will also be incorporating green plants in the rooms so as to not let the color scheme become too boring.

An important part of this virtual application would be an in-depth tutorial that goes over all the functions and features so that the users can use the app to its fullest. The tutorial will be in the room the user first lands in once they enter the game, this ensures new users will not miss it. The tutorial will include many user interactions the user will need to know in order to proceed through our game. The following will be included in the tutorial:

- How to pick up and set down an object
- How to interact with an object (that does not involve picking it up)
- How to use the ruler feature
- How to enter another room through the door
- How to access the game menus

The tutorial will be presented by words in front of the user near the object or feature they are looking at/near. Additionally, the tutorial will be present each time the user enters the game but if they are experienced they have the option to bypass the room by simply moving through the door to the main lab area.

For the actual 3D model viewer, one of UCF College of Medicine's previous medical virtual reality applications includes a 3D model of a heart with different options to view it and the ability to manipulate the model as the user sees fit. Since this aligns very closely with how this project needs to have its 3D models viewed, the team will be taking inspiration from their system and making some improvements/ altering it so that it works for more

than just a heart model. This will most likely be changing it so that the users have options to choose different parts of the body, and then giving them options to choose different specific organs in the body. The end goal will be to give the users the option to look at different CT scanned organs through a 3D model and allow the user to change which layer is being viewed, similar to the 3D slicer that is commonly used by medical professionals to view CT scans.

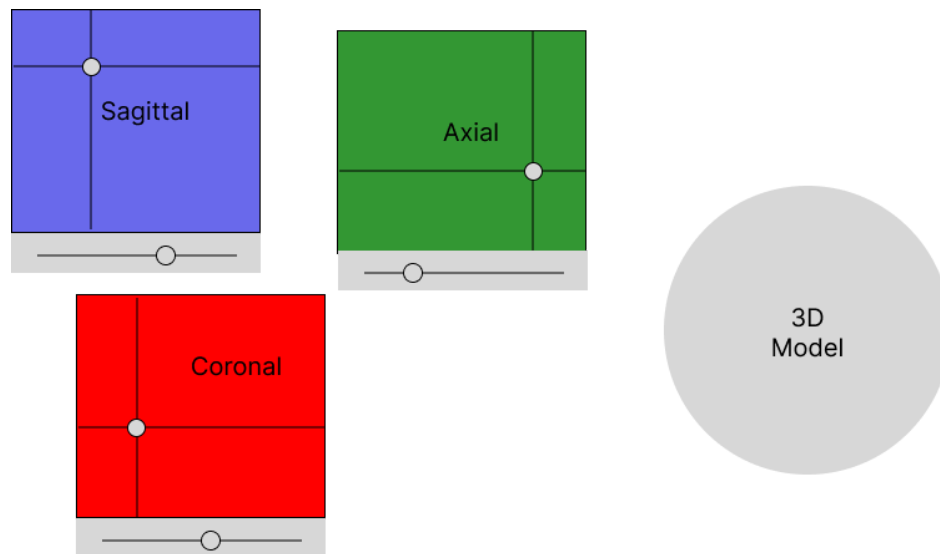


Figure 21: Abstract model of user's view while viewing CT scans (created by Kyle)

The CT scans can be viewed in the same room but as a slideshow on a wall, maybe the team can implement a projector to make it seem like the CT scan viewing is being projected on a wall. As a possibility, the team can put a computer by the projector which can act as the menu to choose between different normal and abnormal CT scans.

The next room will be where the users can pick and take different quizzes regarding CT scans. This can look like a computer, to choose the kind of quiz, and then similar to the CT scan slideshow there can be a projection on the wall with the quiz questions and answers. As a fun addition, the team can make the quizzes look similar to Kahoot! questions but without the

music since it's copyrighted and in this instance it would not fall under fair use. For this room in particular, the team can set it up to either look like a classroom with desks and a whiteboard. Especially since one of the stretch goals would be to add a 3D pen, the whiteboard would give the students space to write down different ideas or rule out certain answers while taking the quizzes.

Interactions and Interactables

The most important thing in making a virtual reality feel like the player is really moving around the space, are the things that the player can interact with. They not only have to be responsive to the players movements and actions, they also need to feel like realistic objects. In other words, if a user picks up an object and decides to throw it then the object needs to actually have the physics to be thrown and have interactions with other objects it may come into contact with, like bouncing off a wall. If the object just clips through the wall then it breaks the realistic illusion and feels like poor design.

All objects in the scene should have a basic level of interaction except for the furniture such as tables, which will not be movable. For example, a banana on a counter should have the ability to be picked up, placed, or thrown by the user. Although something like banana interactions are not required as part of the minimal viable product, the addition of random interactable objects will add to the immersion and let users feel like the virtual environment is much more realistic. Besides random interactable objects the scene will have interactable tutorials which will give an explanation of the CT scan viewer and how to use it, as well as, how to take the quizzes in the quiz room. Additionally, since a lot of players will most likely have never used a Quest headset, or potentially any VR headset, it's

important the interactable tutorial includes a basic explanation for movement and button layout with a place for the users to practice picking up objects, pressing buttons, and typing on a keyboard if written responses are required for the quiz section. The biggest interaction in the entire virtual environment will be the 3D model and CT scan viewer. The initial concept is the CT scan viewer itself will be composed of four screens that the user can change the view of, as in what plane, by interacting with buttons. For the 3D models, the user will be able to interact with the model by moving its position, changing the slice of the model, labeling the model, and measuring it. Finally, the quiz room interactions will be like taking any quiz except by choosing the correct answer with the virtual pointer on the controller or by typing the correct answer with the virtual keyboard.

Administration

Budget/Assets

Each of us has been given a Visual Studio Professional subscription through May 31st, 2026. Included in this subscription is a variety of benefits, including access to Visual Studio Professional, but most importantly this also includes a \$50 monthly credit for Microsoft Azure. Among the seven of us, this should be more than enough to run any testing servers during the duration of our project's development, especially since the majority of this project is locally hosted. In regards to budgeting, this subscription is priced by Microsoft at a cost of \$99.99 per user/month for an annual subscription, although it's very likely that these were purchased at a reduced cost from either educational or bulk discounts.

We also have each been granted access to one Meta Quest 3 headset per person, as well as an additional Meta Quest 2 headset to ensure the

project is compatible with both. Each Meta Quest 3 headset could be valued at \$499.99 when purchased new, while the Meta Quest 2 headset was initially valued at \$299.99. However, these headsets have been used for previous development projects, and so a better estimate might instead value each headset at a monthly cost of \$29.99 per month, which is the cost of financing a Meta Quest 3. Using the same price ratio as their purchase prices, the Meta Quest 2 headset could be valued at roughly \$17.99 per month. Our total project costs and budget will be estimated using a 6-month timeframe in the table below.

Asset	Value/Person/Month	Total Monthly Value	Total Value
Visual Studio Professional	\$99.99	\$699.93	\$4199.58
Meta Quest 3 Headsets	\$29.99	\$209.93	\$1259.58
Meta Quest 2 Headset		\$17.99	\$107.94
Total Budget		\$927.85	\$5567.10

Figure 22: Table of Budget from Combined Assets

Backlogs

Our product backlog includes all of the tasks we need in order to complete all the goals we have for this app. Our current sprint backlog includes the tasks we plan to accomplish within the current sprint. In order to keep track of our tasks we have decided to use JIRA for backlog and sprint planning.

Final Summer Sprint

Final Sprint Dates: July 14 - July 27

Our current and final sprint for the summer semester is focused on creating the skeleton of the app in order to prepare ourselves for Fall semester. This will essentially be our badly done demo that will do basic interactions and have basic features. We consider the skeleton to be: room layouts with functional walls, basic UI for the menus and player view, and basic interactions between the player and objects within the rooms. We will also add in assets like doors and the basic assets needed for interactions like tables and monitors.

Task	Status
Create functional Walls	Done

Add in some 3D models	Done
Add in some CT Scans	Done
Quest 2 and 3 Systems Testing	Done
Create initial menu UI	To Do
Confirm color palates for UI	Done
Complete Final Design Doc	Done
Add in door Assets	In Progress
Add in basic Assets	In Progress
Create Quiz UI	Done

Figure 23: G04 Current Sprint Backlog

Product Backlog

The product backlog contains any task that we will eventually need to complete within our timeline but is not being worked on in our current sprint.

☐	Type	Key	Summary	Status	Comments
☐	☒	G04-39 🔗 G04-15	Initialize Azure DevOps Repo	TO DO	Add comment
☐	☒	G04-39 🔗 G04-16	Set up Azure	TO DO	1 comment
☐	☒	G04-40 🔗 G04-17	Plan Program Outline	TO DO	Add comment
☐	☒	G04-10 🔗 G04-18	Initial UI Design - Figma	TO DO	Add comment
☐	☒	G04-40 🔗 G04-19	Research VR Interfaces	TO DO	Add comment
☐	☒	G04-10 🔗 G04-28	Environment Design	TO DO	Add comment
☐	☒	G04-40 🔗 G04-21	Final Additions	TO DO	Add comment
☐	☒	G04-39 🔗 G04-22	Quest 2 and Quest 3 Systems Testing	TO DO	Add comment
☐	☒	G04-38 🔗 G04-23	Add Reach Goals	TO DO	Add comment
☐	☒	G04-10 🔗 G04-24	VR Options Menu	TO DO	Add comment
☐	☒	G04-40 🔗 G04-58	Practice presentation	TO DO	Add comment
☐	☒	G04-40 🔗 G04-25	Present	TO DO	Add comment
☐	☒	G04-40 🔗 G04-26	Create Project v2	TO DO	Add comment

Figure 24: Initial product backlog in JIRA (taken by Zoe)

Above is a sample of our JIRA backlog for our project, it is not a completed list of everything we need to do yet as we assume there will be bug fixes and other issues that pop up along the way. The above figure contains a very simple list of tasks we will eventually need to complete, so below is a more detailed and focused list of tasks in our backlog:

- Design the rooms to resemble a clinical environment
- Create an introduction scene
- Create a credits scene
- Add in objects for the user to interact with
- Add objects with interactivity into the rooms
- Add functionality to tutorial scene
- Add functionality to CT scans and 3D models
- Add virtual ruler and any other tools into app with functionality

- Add in Accessibility options
- Add Background audio/music
- Create settings menu with options for audio, brightness, subtitles etc.
- Create practice questions UI
- Ensure practice questions functionality
- Create library of practice questions
- Import CT scans, 3D models
- Implement voice narration
- Ensure proper scaling
- Give to Medical Students for testing
- Test application on both Quest 2 and Quest 3

Sprint Map

In order to keep track of all of the tasks that need to be completed and by when for our application, we have decided to implement 2-week sprints. During each of these sprints we have set goals that we expect to complete within that 2-week period. By using this sprint map, we should have time to complete the MVP of the app, test the base app, and then possibly add in some of our stretch goals before our final systems testing and presentations.

Sprint	Dates	Goals

Sprint 1	June 16 - June 29	Sprint Prep, Initial Setup
Sprint 2	June 30 - July 13	App layout (level design, UI, etc.), Prelim doc
Sprint 3 (current)	July 14 - July 27	Setup skeleton of App (room layouts, basic UI, basic interactions), Final design doc
N/A	<i>July 28 - August 17</i>	<i>Semester Break*</i>
Sprint 4	August 18 - August 24	Basic gameplay (Tutorial, CT scans, quizzes) and menus
Sprint 5	August 25 - September 7	Integrate database, add in ruler tool, ensure appropriate scaling for models
Sprint 6	September 8 - September 21	Add voice narration and additional MVP features for CT scans (drag/drop labeling) Initial Medical student testing (perhaps)
Sprint 7	September 22 -	Add in subtitles, check for

	October 5	accessibility, add in environmental features(background music), begin stretch goals, fix any issues from Medical student testing in previous sprint
Sprint 8	October 6 - October 19	Focus on stretch goals (TBD)
Sprint 9	October 20 - November 2	Finishing touches on app, Team testing, Final medical student testing
Sprint 10	November 3 - November 16	Prep for Design Review
Sprint 11	November 17 - November 30	Prep for Final Presentation

Figure 25: Sprint Map

*If we are not caught up to our goals we will add in the work needed to be done here

Milestone Chart

Below is our Milestone chart from JIRA with each of our sprints labeled. Even though our semester started earlier than mid-June, we only learnt about sprints/scrums around mid-June and thus our Sprints began in June and followed a two week timeline (with the exception of the first sprint). Some of our major summer milestones included: receiving the Meta Quest headsets, creating the joint Unity space and creating a basic room layout.

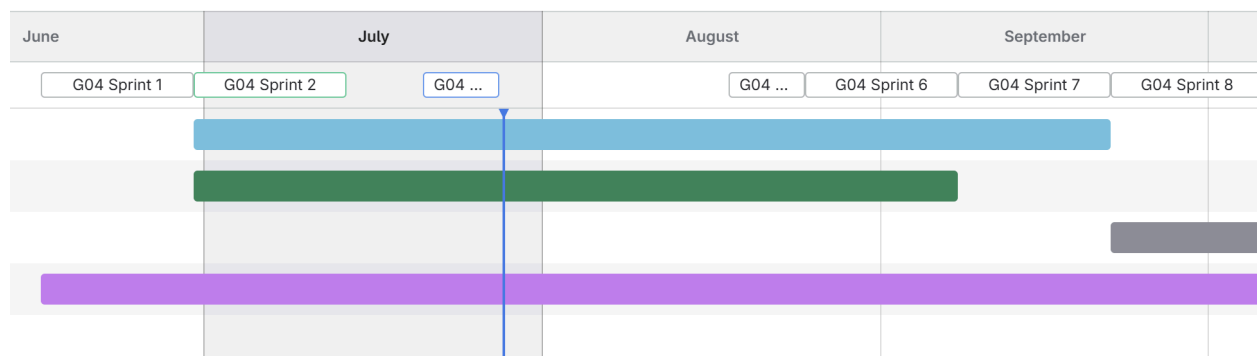


Figure 26: Milestone Chart from JIRA (taken by Zoe)

Conclusion

Summary

To summarize what we are intending on building and what we have completed thus far, we are making a Clinical Imaging CT Virtual Reality application designed to be used with Meta Quest 2 and Meta Quest 3 headsets. Our project will consist of two main parts, the quizzes and the CT scans. We are intending on receiving the quizzes from the UCF College of Medicine and thus only have to focus on designing how the quizzes will be displayed and interacted with. We will also be receiving CT scans and

possibly pictures of cadavers also provided by the UCF College of Medicine. Our primary focus with these will be implementing a ruler and allowing users to differentiate healthy CT scans versus unhealthy CT scans and be able to identify what is wrong in the unhealthy CT scan. This will be our primary goal for this project along with making it run well and designing it to our sponsor's liking.

Results Analysis

Although our project is still ongoing, we have finalized our design and have decided to assess how we have managed to navigate the design process thus far. We will be reviewing our progress and any issues that we managed to run into and how we dealt with them. We hope that by seeing what went right and what went wrong, we will be able to analyze these instances and use it to improve our teamwork for the upcoming semester where we will be moving on to the next development stage by building the project.

This semester we have proven to ourselves that we are good at keeping up on documentation. We have made our development steps in our sprint maps and have kept to them - excluding unforeseen incidents that have prevented items from getting completed. We have made notes on every sponsor meeting to better understand what is expected from us and have kept in regular communication with her in case of any miscommunication. Our dedication to writing everything down has only helped us as there have been instances where we have forgotten what needs to be done or what, exactly, it was that our sponsor wanted from us. Having these clear records of expectation has only benefitted us in the long run while we were creating our development process and when we started to design our project. Not only does it help with reminders, but our

record-keeping also keeps us accountable for who does what when we end up assigning tasks to certain people, we know who failed and who completed their tasks. This allows us to open further communication with those who didn't meet the mark and question them as well as to offer assistance with completing it.

As mentioned already, our team was very good at establishing and maintaining communication with each other and our sponsor. We had weekly scheduled meetings as a team with occasionally other, impromptu meetings designed to clear up any miscommunication or issues that we were having as well. We also had weekly meetings with our sponsor where we would talk about what we got done for that week and what our sponsor would want us to do going into the next week. In addition to meetings, we have kept in constant contact with each other over Discord with multiple channels to keep all of our information and communication easy to understand and find later on if it needed to be referenced. Communication with our sponsor out of meetings consisted of emailing and messaging over Microsoft Teams. With this ease of access to not only each other but also our sponsor, we made sure to keep up to date and keep everything accurate to the best of our abilities.

Another thing that we accomplished was keeping to the schedule that we had set earlier in the semester in order to get a working bare-bones prototype up and running by the time of our check-in. Our sponsor was very happy with our progress because of this and helped us to finalize some details now that she had a better understanding of our vision versus hers. The feedback that we got allowed us to further finalize our project design that we plan on sticking to in the upcoming semester. Overall, we are proud to say that we stuck to our timeline quite well and even advanced farther than we intended to in the end.

Though we had many good things going for us, we also had many challenges along the way. The most disruptive to our progress were technical difficulties that we faced when integrating our technical pipeline for the multiple development softwares we are using to assist us in our project development. There have been known issues with creating conflicts when multiple people are working at once that we will need to figure out a system for. There were also issues in operating system compatibility with the software that we are using, leading to missed deadlines and setbacks. These issues were unfortunate and annoying to deal with and will be fully addressed and handled in the upcoming semester, and hopefully we will be able to prevent issues like this from happening again.

Another thing that went wrong was miscommunication. While our overall communication between ourselves and our sponsor were amazing, there were times that different interpretations between our team and our sponsor about what was expected of us arose, especially near the beginning. There were instances where classroom deadlines did not match up with our sponsor deadlines, leading to our sponsor insisting that we slow down in order to better plan, which led the team feeling frustrated and anxious to get things done. There were also times where our sponsor asked for something and after never really elaborated to a point that we were able to fully grasp what she wanted. This happened multiple times before we were able to figure out that it was a mismatch of understanding what we were needing to do. Issues like this also occurred amongst each other, impromptu meetings not reaching all of the members and scheduling some things last minute that not everyone could attend.

These issues happened frequently and noticeably enough that we intend on implementing a number of improvements to better facilitate an easier project development process next semester. First, we plan on creating a more structured process for design approvals. Before anything major is

implemented in the main project, it must get approved by the majority of the team and at least mentioned to the sponsor about what was pushed in the next meeting with her. This will ensure that everyone is aware of what is currently working in the project, and what still needs to get done, as well as being able to make sure that the correct person is notified if a bug or glitch were to occur due to the import. Next, we plan to keep meeting together each week, in person, for the upcoming semester where we will be highlighting what we did that week and any issues that had not been mentioned in the Discord chat. We will be making sure that everyone has the same understanding of what our sponsor expects of us and if there is no clear consensus then we will reach out to our sponsor for clarification.

Finally, the biggest lesson that we have taken from this design process is the importance of balancing speed with the actual process. We fell into the temptation of moving on when not everything was fully fleshed out at times and this led to issues down the road. We've learned that investing the time into more planning and reviewing with everyone on the team as well as our sponsor giving approval before we move on will lead to better results and less hesitance at what we are doing, as well as being more prepared for any compatibility issues as we will have planned out where everything is supposed to work. As we move toward the project building and implementation, we hope that our improvements will lead to better and more streamlined development. We believe that these changes will significantly improve our ability to deliver a better overall project and will help us grow as developers.

Final Performance Analysis and Graphs



Figure 27 - Frame Rate Graph over time

Generally, the app runs at a target frame rate of 72 FPS. 72 FPS is a reasonable target framerate for standalone VR apps, although it's lower than the preferred target of 90 FPS expected for PCVR. There are occasional frame dips as elements are loaded in, although this isn't very noticeable and isn't likely a significant issue, especially over a longer use time. The main performance issue is upon loading in and popping out a 3D model, which can be seen in the graph above - the framerate falls to about 56-60 FPS while in the room.

A drop to 56-60 FPS could have a negative impact on people who frequently experience virtual sickness, as if they turn or move quickly any timing disconnect could exacerbate their symptoms. That being said, this room doesn't expect much movement or turning, so ideally the impact won't be that noticeable. It's also high enough and consistent enough that it should hopefully still be comfortable enough for most people - if we were experiencing sudden spikes down to sub-30 FPS or frame tearing, as we did

with our original more detailed models, this would be more of an issue. As it is, for people with virtual sickness, relying on accessibility comfort settings will hopefully smooth out their experience.

Hopefully, in future updates it will be possible to improve performance without hurting the visuals of the rest of the application. It's clear that the application is being GPU-throttled by the highly detailed models. The level of detail for the models was ultimately a compromise between clarity and performance, and it'll be difficult to downscale it further without hurting the overall experience. However, it might be possible to add additional models at lower levels of quality and switch between them using LOD settings, allowing for a higher level of detail only when the user is close enough for it to be noticeable.

Insights Gained

As stated before, we came across a lot of issues while developing. Below are how we addressed issues and how we changed our development processes to avoid causing even more issues.

Frontend

One of the most basic but important lessons we learned was to begin testing on the headset as soon as possible. Even for just exploring the room set up, I found it's super important to get a feel for scale within the headset and of course the best way to know if something is realistically sized is to just experience it yourself. Additionally, viewing the application within the actual headset allows you to catch small details about the environment or gameplay that you might not have caught from the Unity side of things. For example, in our application the outside environment was not working properly and would appear a bright yellow if you looked through the

windows. For a while this problem was left untouched because it looked normal in unity, both in the editor and gamemode. As we got more into development and were using the headsets we were constantly reminded that we needed to fix that and thus it wasn't left that bright yellow for our demos.

Another lesson would be to form an integration plan. As a team we had some general merge conflicts that were swiftly fixed in the beginning. However, at one point in mid development I accidentally corrupted my scene and then passed that corrupted scene onto another group member when we merged. Up til that point we had a couple scenes going on but no clear distinction on which each of us should be working in, leading to some conflicts because I forgot to pull from master branch and has pushed my recent changes. After this mishap, I had to just abandon my changes and remake a branch off of the current master branch. From that point on I made a point to pull from master always, and then to duplicate the master scene to make my own scene and work just in that. When we integrated as a team we could then merge all our changes together.

A lot of our assets we got premade from our sponsor and her 3D modeler but there were some other assets we had to go out and find on our own. In the beginning I didn't quite understand what it was exactly that I needed to download and while it wasn't that big of a deal it created some inconveniences towards the end. I would only download the prefab or objects of the assets and leave behind the material/texture that went along with the object. This meant when actually placing the object in the scene it would not look good and I would either use material from another asset or eventually I realized my mistake and I was able to go back and grab that texture/materials I needed.

Version Control

We learned having different scenes in Unity can help us when it comes to integrations. Integrations typically fail when multiple people are working on one scene but in different branches, so when the branches are merged the scene will have different sets of items and will result in a completely corrupted scene. After that hiccup we decided everyone would get a different scene based on the work they are assigned and name it their first name+date_created. To merge scenes we will merge individual items over manually onto a main master scene, which again is where the player will be spending most of their time so it's important to handle these integrations properly. It seems like a long and arduous process but it was the simplest solution we could think of that would work for our project so we wouldn't constantly be losing data.

Make your own scene in Unity! This is one of the best tips we can provide. Instead of working on one scene try to have each task or each member make their own scene and then during integrations pick and choose what you want seen in master. To make it easier to merge the scenes when integrating, each person should work on a task or part of the scene that is separate from what other people are working on. For example, if somebody adjusts the scaling of a room in their scene, and another person moves or adds objects to that room in their own scene, it will be difficult to combine, since the position of the objects won't match the scaling of the new room.

When performing an integration, one person can then combine all the branches with updates to push, and then each scene should be combined to make a new master scene. To make it easier, you should start with whichever scene has the most changes (since you then won't need to apply those changes). This will be your integration master scene. After this, you can load multiple scenes at the same time - I suggest duplicating your

hierarchy and using the lock button above to select different elements between each scene. Ideally, it should be as easy as dragging the changed objects by their parent into your integration master scene. Once all the changes from a given scene are implemented, you can unload the changed scene (don't save changes to that scene, just in case you made any mistakes) and move on to the next.

Once all changes have been made, make sure every member with changes approves the new integration branch before you merge it back into master. This can be handled using Azure DevOps' pull request feature. Once the changes are approved, you can safely delete the personal scenes, everyone can merge the newest master branch into their working branches, and duplicate the new master scene into a personal scene. It's very helpful to integrate often and make sure everybody works off the most updated material - if they don't, you can eventually run into conflicts and lose progress.

Besides working in separate scenes, it's also possible to save specific collections of objects as a prefab - just move your parent object from the hierarchy to somewhere in your assets folder below - ideally in a folder specific for prefabs like it, so they're easy to find. You can then modify the prefab in isolation or in context, and once you're satisfied with those changes, you can press apply all to make those changes to those prefabs, regardless of what scene they're in. However, if the prefab in a scene currently has any overrides, changes will not be made unless you either choose to apply (saving the overrides in that scene) or revert (using the currently saved version of the prefab) the current overrides. Prefabs can be a little tricky to use since they don't allow you to remove elements without unpacking them and you need to remember to handle overrides, but it will be very helpful if you can properly modularize your project. You need to make sure only one person at a time is ever making changes to a given

prefab - you'll run into the same issues with merging prefabs as you would with merging unity scenes. Remember, if you don't handle any overrides in your scene, your prefabs won't update properly.

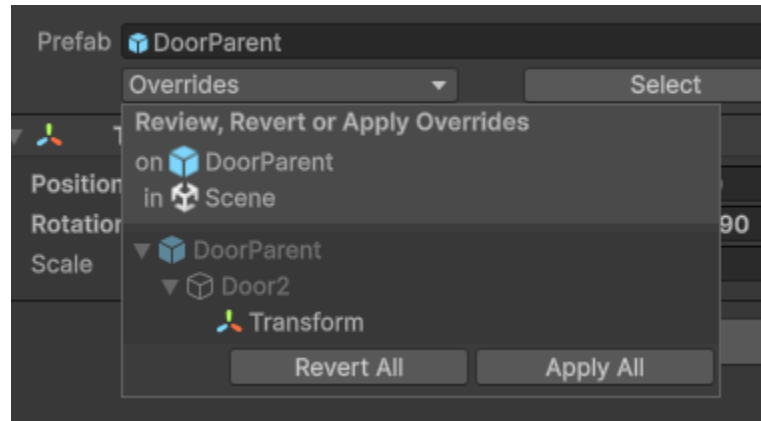


Figure 28 - Prefab Revert All and Apply All options

In addition to prefabs, you also need to handle version handling for your scripts. Unlike with Unity scenes, scripts will update and merge just like any other code - parallel changes are handled just fine. Similarly to prefabs, scripts will automatically update in every scene that they're in. However, this means you must use EXTREME caution when updating any scripts that are currently being used as components with any changeable properties - changing a script can mean you break whatever's currently working in other people's scenes. That's why, ideally, you should use your scripted components in objects that are part of and which exclusively target objects in the same prefab - that way, updates will update both the changes to your script and to your prefab's component, resulting in a seamless integration. Whenever this isn't the case, you should ideally copy over all objects that have components referencing each other together when integrating - if you don't, you'll need to manually reselect the properties for the component that weren't carried over.

Ideally, with proper planning and modularizing, integrations will take no time at all. If all changes are confined to prefabs and scripts, you won't even need to manually merge separate scenes. However, I'd like to emphasize some critical pitfalls that can lead to major issues:

- Do not EVER touch the master scene or another person's scene except to duplicate it and to save as at the end of an integration push. If two people work on the same scene at the same time, you will get a merge error. You'll be unable to modify the scene in Unity, and you'll need to open the .scene file in a code editor where it will be nearly impossible to parse. Someone is ultimately going to lose progress at this point.
- Do not make changes to another person's prefab when they are also working on it. Similar to above, you can get merge errors.
- Do not make copy-paste changes to large chunks of scripted code. This is general Git advice, but it still applies here - if you copy and paste large changes or save a file separately (maybe you're dealing with a bug, and you want to use an old version of the code that doesn't have those bugs, or somebody sent a WIP version of the newest code) your version control will assume that the newly saved code is all recent changes, potentially resulting in a merge occurring where the correct code is overwritten.
- Make sure you pull the most recent code after every integration, even if your work wasn't integrated. If someone runs into an error with their scene as a result, an error would have occurred when their branch was next integrated, and it's best to handle any individual issues on your own time instead of at a group meeting/integration, which can potentially cause delays and additional meetings.

- When handling merge conflicts (not Unity scenes, but C# scripts or package files), always check the code manually. If you're unsure which is the most recent, double check with the person who made the changes, and if they're unsure, err on the side of additions rather than deletions - once you make a decision and delete something, version control can make it hard to get back.

Building the APK

Building the APK for the headsets is a fairly simple process, but there are a lot of potential options to change in Unity, so this will cover each of the steps for what you should and shouldn't change when making a new build.

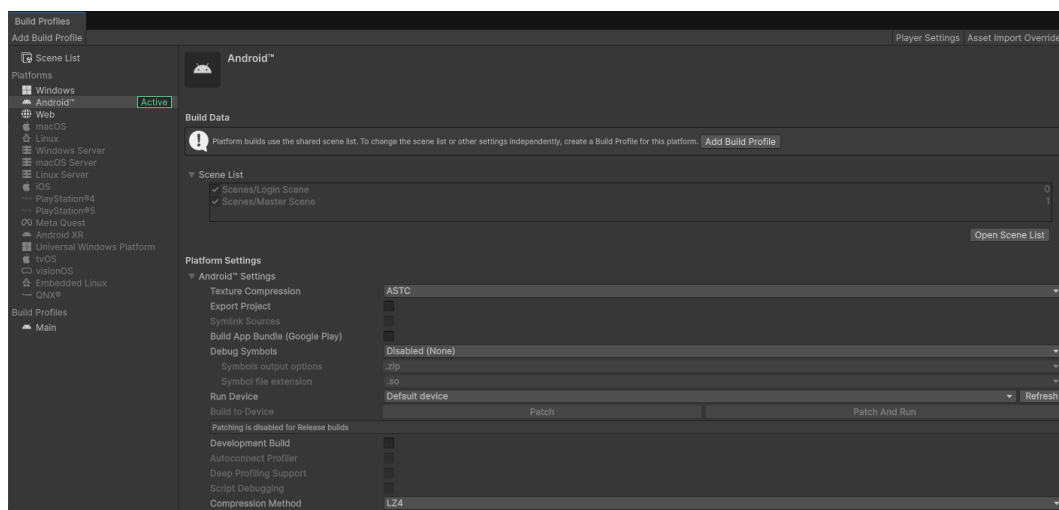


Figure 29 - The Build Profiles Window with our current settings

First, go to the File - Build Profiles button. We generally worked off the Android build profile under platforms, although it's possible to make your own. On this page we have the scene list - this probably won't change between builds unless you're planning on removing/replacing the login scene. Our texture compression settings is ASTC, which is recommended for the Meta Quest. Besides this, everything on the page is set to their defaults.

Before you build, you'll want to adjust version control settings in player settings, seen in the top right of the menu shown above.

You should now be in the player settings menu, which you can also reach from the project settings menu. Company name and product name should remain unchanged between versions, but you should increment the version counter each time. It is VERY important that you also go to the Other Settings dropdown menu in the Android settings menu and increment the Bundle Version Code every time you make a new APK to be automatically pushed to COM headsets - this is the internal versioning system that Meta will check when pushing out a new build - if the number isn't higher, it won't actually update the APKs.

The two other changes you might want to make here are to the default icon (the icon that shows up in the quest library) and the splash image (what shows up . If you save a new image as a Texture 2D somewhere in the assets folder, you'll be able to use that as either the icon or the splash image by selecting it. These are generally going to be the only settings you're going to want to change.

In addition to the above, if you want to make changes to any graphics options in order to make tweaks to performance, you should go to the Quality section, also in the Project Settings UI. For an Android build you should be using the Mobile level Universal Render Pipeline Asset. If you double click on the Render Pipeline Asset object below, you can open it in the inspector. Between the settings on the Quality page and the settings in the inspector, there are plenty of options to strike a balance between performance and graphics. However, I'd like to point out some of the following options:

- Global Mipmap limit should be kept at Full Resolution. The clinical pearls will become unreadable if they're overly mipmapped.
- Anti-Aliasing is ideally set at MSAA 4x - this is the recommended level for the Quest 3, and you ideally shouldn't be running any other Anti-Aliasing in other settings. However, you can experiment with lowering this.
- Post-processing should remain checked in the Main Camera settings, but the post-processing options here aren't important - if you want to make changes to post-processing you should adjust the object in the main hierarchy. The currently enabled components in that object should all remain as they are since they're used for visual settings, but you can feel free to add any additional settings as desired.

Now that you've made any necessary adjustments to your build and graphics settings, you'll also want to adjust your lighting settings. Since we've made the decision to use baked lighting in order to improve performance, if you've made any major changes to scenes or moved things around, you'll at the very least want to generate new lighting in order to account for the shadows of any moved objects. You can reach the lighting menu by going Window - Rendering - Lighting. Again, there are many different settings available for you to change - we've generally tried to prioritize options that improve performance but which still look good on the Quest 3. Most importantly, you'll want to go to the arrow to the right of the Generate Lighting button - you should clear the previously baked data before you generate the lighting again. After this, you'll then press the Generate Lighting option - this should take several minutes, and there should be an indication of how far along it is on the bottom right of the main window.

At this point your APK should finally be ready for building. WAIT! Before you build the APK, make sure you disable the XR Device Simulator in the hierarchy - this is immensely useful for testing on the PC, but it will ruin your APK build. Go back to the Build Profiles window and select build - from here you can choose what to name and where to save your APK file - this name isn't going to be the name once it's installed on the headset (that will be the Product Name in the player settings window), so feel free to name it according to its version or date for ease of use and version control. Again, this process will take several minutes, so make sure you double check everything is right before starting.

Once the APK build is finished, you're ready to deploy. However, you shouldn't deploy to all of the headsets before you do a quick bug check - for example, if you left the XR Device Simulator enabled, you will be completely immobile when you run the APK, which is an easy fix if caught immediately. However, if you push an APK using an application like the Meta Quest Developer Hub, pushing a new APK means you'll also have to increment your Bundle Version Code, and everyone will need to wait for a new deployment to install. If you instead test your APK using SideQuest first, however, you should catch any obvious bugs immediately and you shouldn't need to go through the additional hassle that multiple deployments would bring.

Open SideQuest, making sure your headset is plugged into your computer by USB. If you're having issues, SideQuest has setup instructions next to the headset indicator. Make sure your headset is on and that developer mode is enabled in the options menu - if it's your first time connecting to that PC, you'll also need to go into your headset to confirm that you want to allow USB debugging. Once you have the green light, use the 'Install APK from folder' button. SideQuest will show a loading bar and tell you when it's finished. At this point you can put on your headset - applications installed by SideQuest will generally be in the Unknown Sources

folder in your library. If you're happy with how your application looks and runs, then you can add it to your Teams APK folder and deploy it for the rest of your team - you've successfully built a new version of your APK.

Notes for Future Developers

- Backend is shared between projects, everything should be labeled per project and other projects should not be touched.
- Keep security in mind at all times as the projects will be hosted on UCF servers.
- All the files that were worked on independently are located in a blue teams folder labeled COM_VR_App. The frontend Admin Portal and the backend that connects to it are located inside.
- Please keep data size in mind for the database, i.e. varchar over nvarchar unless you need the emojis.
- The framework for the login, session, and quizzes connected to the backend is all there and should work properly.
- The backend server is currently not up and will have to be tested before endpoints can be replaced in Unity scripts.
- Reminder that the backend can only be connected on UCF internet.

Ideas for Future Developers

Each of us worked very hard on this project and we did our best to implement all sorts of user friendly features within our allotted time. We would love to see the torch carried forward to other Senior Design teams to continue building upon this project. That being said, here are some suggestions and ideas we think would be good to implement in the future

based on what we have learned, seen, and received as feedback from others.

First and foremost, regarding the multiple-choice quizzes and drag and drop game we think it would be a great idea if future developers could expand so there are more quizzes and games for the students to practice with. We have already set up the framework that would allow an administrator, in this case Melissa Cowan, to go in and add or adjust the multiple-choice quizzes but we unfortunately could not get this connected to the UCF COM database because of firewall issues so this connection could be one of the first things a future team of developers could address since the framework has been set up. We also could not get this same framework set up to work for drag and drop quizzes since those work with images and labels but maybe a future team could find a way to allow an administrator to upload their desired images and labels for students to have additional drag and drop interactions besides the one we implemented. In the same realm, connecting to the UCF database to allow quizzes to autograde and automatically get updated on a student's canvas would be an amazing addition to this project. That way a student can take a quiz or interact with a drag and drop game and the grade they receive inside the scene and actually be used in the student's curriculum.

Besides additions and adjustments to the quizzes and game interactions, incorporating more CT scans and 3D models would be a good idea for future developers to consider. Maybe instead of just focusing on adding more CT scans, future development teams can consider adding different forms of medical imaging such as X-Rays, MRIs, or Ultrasounds. The process of creating 3D models with different medical imaging might also be different but it would still be a good thing to add so the VR App can help first-year medical students with all sorts of imaging recognition. We are not sure how the curriculum is for second-year medical students, but maybe by

adding other medical imaging forms can also increase the difficulty of the educational material and allow for not only first years, but also second year medical students to use the app to help them learn. Broadening the target audience for this VR application would be a great priority because then it'll increase users which equates to more students studying this material and potentially helping them.

On the topic of broadening the target audience for our Clinical Imaging CT VR App, this app can be ported to be used on other VR headsets. We always had this idea in mind so we stuck to the use of OpenXR assets during the creation process. Since OpenXR is the standard API for cross-platform virtual reality applications it would be a lot easier making this application usable on headsets like the Apple Vision Pro or even the Steam Frame. Ideally a system like the Steam Frame would be the perfect candidate for an application such as this one. As discussed in earlier sections such as "Insight Gained" our biggest drawback came from how much the in-unit GPU for the Meta Quest headsets could run. Since one of our main goals for the project was to allow students who may not have the best computers to be able to check out the headset with the UCF College of Medicine and study to their hearts desire, we needed to limit the polycount of the models, making them unfortunately less detailed. The Steam Frame, as of the writing of this paper, would run like its own untethered computer, which would be perfect for this heavy application. If not, then another potential would be turning this app into an executable that can run off a PC. The advantages to this would be better frame rate and higher polycount/ higher resolution 3D models, but the drawback would be that you would need a pretty beefy computer to run the .exe file version which is why we did not focus on this stretch goal.

For future installments of this application we would recommend the following changes for improving the quality of the rooms:

- Many of the walls are built using basic cube objects but this means the doorways have to be split into several different objects making it difficult to place materials onto these walls. In case of future wallpaper changes (and just for quality of life) we would recommend finding doorway assets and replacing those separated walls with one wall. As it stands, the wallpaper for sections with windows/doors had to be made with a separate quad object over the wall.
- Another improvement for the walls would be to ensure that there can be different material/texture on either side. In our building the rooms do not share similar wallpaper designs and in this version when the material is placed on the wall it will show up on both sides. In the future we would not want that so each room can have their own distinct style.
- In this current version we do have the proper wall and window asset but the scaling still feels a bit off. It works for this first version but in the future it would be a nice quality change. Additionally, this wall and window asset does not respond to our custom material well as it appears in vertical stripes instead of the horizontal it should be. As a result we had to use a quad overlay to recreate our wallpaper, in the future this should be fixed for a better quality.

Some recommendations for the quiz in terms of the UI:

- In the current quiz, once the user clicks an answer choice it automatically submits that as the answer, a quality of life improvement would be to have a submit button just in case they accidentally click an answer or they want more time to think.

- Along the same lines, another improvement would be adding a “view question” type button so the user could go back and see the question/answer they just answered. Currently if the user wanted to go back and see the question again they would just need to restart the question which is inconvenient.
- To lean more into the educational aspect of the application, when the user takes the quiz it would be nice if it would be able to suggest what they need to work on based on past quizzes. We want the students to be able to learn and improve their skills and this feature would greatly improve their ability to do that.

Recommendations to the quiz in terms of the database:

- Include more details in the quiz response table to be able to have more information on what happened during a quiz, such as which questions were marked right or wrong and how much time was spent per question instead of the entire quiz. This will allow for better analytics to be given back to the teachers. A suggestion to do this would be to create a new question response table and include the additional metrics there and link it to the quiz response id.
- Rewrite the column value for timeSpent to be in TIME instead of INT for more clarity and potential analytics.

Another improvement would be in regards to the login room:

- The current version of this application starts the user in a separate scene for the login. This login scene is pretty basic and not very immersive in context to the rest of the application. In the future, the application would feel more cohesive if the login was moved to a room that matched the medical feel of the rest of the building. For example,

creating a waiting room type environment where the user must sign in before proceeding into the tutorial room.

Improvements to be made to the admin portal:

- Fix the edit and delete buttons in the student dashboard section.
- Populate the quiz results page with the quiz results table (with the possible question results table in a similar manner to the quiz view).
- Add in a way to keep track of who is allowed access to the admin portal, whether creating a new admin table or adding a new admin column to students and renaming it to users. With this, delete the hardcoded login credentials and use what's in the table.
- Find a way to link Canvas quizzes and students to the database, possibly multiple different classes and use the student class section to differentiate which students see what.

Student Testing Feedback

Our target audience for this application is first-year medical students, specifically this product was made for the UCF College of Medicine students. With this in mind, we were able to conduct some student testing with some first-year medical students at the UCF College of Medicine and get their feedback on what they thought of the Clinical Imaging suite. Most importantly, they really enjoyed the application! But of course like most first versions of applications, it was not perfect. They were interested in seeing if the CT scans themselves could be popped out and moved around the scene. I imagine they envisioned something similar to how the menu can be grabbed and moved around the scene but with the CT scans on the board. Additionally they thought each individual organ and structure should be modeled and isolatable. Originally, we did have the app that way but since it

was not the main focus of the app we ended up moving in the different direction, instead making major structures like the skeleton and the medical devices isolatable alone. We did not test having every organ modeled and isolatable in an app version so we are not even sure if the headset would be able to handle it in the first place. But if this wants to be implemented for future versions of the app then the process on that model creation is written out. One of our favorite ideas from the students was adding a patient case breakdown for each patient in the scene. Since these are real CT scans from real patients UCF College of Medicine should have a breakdown report of their history and what their medically implantable device is. We did not have enough time to implement this but maybe a clipboard on a table by the board indicating the patient case would be a great addition.

Of the things we currently already have implemented, the students stated they wanted to see more drag and drop interactions. We included a simple brain scan for students to practice with but this interaction is the only one we have in the scene, but even just this one brain drag and drop game was showing students they did not really know how to identify every part of the brain. This shows how the application can be used as a great studying tool and the students wanted to see more drag and drop interactions. With this in mind, more drag and drop interactions, more multiple choice quizzes, and maybe even some additional studying games can be added in future versions.

Lastly, although this was not explicitly stated, some students did suffer some motion sickness while going through the application. We had already implemented a bunch of motion sickness accessibility features in the app since we knew motion sickness was common while using VR headsets. But after the student testing we realized the motion sickness options might not be obvious enough to find. After that we decided to include it in the voice narration, on the board, and even as a glowing symbol on the controller so

the users can have an easier time locating the options button and testing out/ using the different motion sickness settings we've included in the app.

Acknowledgements

Sponsor Assistance

This project would not be possible without the invaluable support of our sponsor, Melissa Cowan. Our group holds weekly meetings with her each Friday where she provides us insight into how our project is going and answers any questions we may have to the best of her ability. Because of her background in working with other senior design projects, specifically those that work with virtual reality in partnership with the UCF College of Medicine, she has already been able to give us the same resources as past teams as well as their contact information in the case that she is unable to answer a question that is more technical. She has worked so hard to provide us with the necessary materials and information to be able to get this project fully fleshed out and started, and will undoubtedly continue to provide us with support and assistance in the future. We thank her for her assistance.

Additional Assistance

Additional assistance that we have access to includes that of Mariana Lobeto, a 3D modeler that our sponsor has put us in contact with if we are in need of 3D models to be made for our project and we are unable to do it ourselves. As of the end of summer semester we have not needed her direct

assistance, however we have also been given access to her previous assets created for other teams (Dr. Hemo and CardiaVR). Once the fall semester begins and we start seriously building our project we may have need of her assistance in creating new models.

Facilities and Equipment

Due to this project being a senior design project, we have been given access to the senior design lab that is located on the UCF main campus in the HEC building. We give thanks to the UCF College of Medicine for providing the necessary equipment in order to work in VR. They have provided Meta Quest 2 and Meta Quest 3 headsets to aid in the development and testing for this project. These headsets are only on loan from the UCF College of Medicine for the duration of this senior design project and we are expected to keep them in working condition.

Bibliography

Unity Learn Pathways

"Unity Essentials Pathway." *Unity Learn*, Unity Technologies, n.d., learn.unity.com/pathway/unity-essentials. Accessed 26 June 2025.

"VR Development Pathway." *Unity Learn*, Unity Technologies, n.d., learn.unity.com/pathway/vr-development. Accessed 26 June 2025.

Meta Horizon Develop

"Unity." *Meta Horizon Developer Documentation*, Meta, n.d.,
developers.meta.com/horizon/develop/unity. Accessed 26 June 2025.

UCF Accessibility Policy

"Digital Accessibility." UCF Student Accessibility Services. University of Central Florida, n.d. Web. Accessed 26 June 2025.
<https://sas.sswb.ucf.edu/campus-accessibility/digital-accessibility/>.

Figma

"Clinical Imaging CT VR App." *Figma*, designed by Group 4,
<https://www.figma.com/design/qT6uu2DK4G9jn6A6u2cv3u/Clinical-Imaging-CT-VR-App?node-id=0-1&p=f&t=iSj4q2tZXedSebMc-0>. Accessed 7 July 2025.

Anatomical Plane Diagram

"1.4D: Body Planes and Sections" LibreTexts, n.d.,
[https://med.libretexts.org/Bookshelves/Anatomy_and_Physiology/Anatomy_and_Physiology_\(Boundless\)/1%3A Introduction to Anatomy and Physiology/1.4%3A Mapping the Body/1.4D%3A Body Planes and Sections](https://med.libretexts.org/Bookshelves/Anatomy_and_Physiology/Anatomy_and_Physiology_(Boundless)/1%3A_Introduction_to_Anatomy_and_Physiology/1.4%3A_Mapping_the_Body/1.4D%3A_Body_Planes_and_Sections).
Accessed 28 July 2025.

Figure 5: Covid 19 Pattern Recognition

<https://www.nature.com/articles/s41598-023-47183-9>

Accessed 28 July 2025

Figure 6: NIH Stroke Scale

<https://www.aclsmedicaltraining.com/nih-stroke-scale/>

Accessed 27 July 2025

Appendix

Figure 5. *Ensemble classification of integrated CT scan datasets in detecting COVID-19 using feature fusion from contourlet transform and CNN* is shared as OA (Open access).

Figure 6: NIH Stroke Scale CC BY-NC-ND 4.0

Figure 14. *1.4D: Body Planes and Sections* is shared under a CC BY-SA license and was authored, remixed, and/or curated by LibreTexts.